

电 子 科 技 大 学

UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

专业学位硕士学位论文

MASTER THESIS FOR PROFESSIONAL DEGREE



论文题目 千万门级 FPGA 中 PCIE 模块的研究

专业学位类别 工 程 硕 士

学 号 201852021711

作 者 姓 名 张明铭

指 导 教 师 王治国 教 授

分类号 _____ 密级 _____

UDC ^{注 1} _____

学 位 论 文

千万门级 FPGA 中 PCIE 模块的研究

(题名和副题名)

张明铭

(作者姓名)

指导教师 王治国 教 授
电子科技大学 成 都

(姓名、职称、单位名称)

申请学位级别 硕士 专业学位类别 工 程 硕 士

工程领域名称 电子与通信工程

提交论文日期 2021.4.1 论文答辩日期 2021.5.25

学位授予单位和日期 电子科技大学 2021 年 6 月

答辩委员会主席 _____

评阅人 _____

注 1: 注明《国际十进分类法 UDC》的类号。

Research on PCIE Module in Ten Million Gate FPGA

A Master Thesis Submitted to

University of Electronic Science and Technology of China

Discipline: **Master of Engineering**

Author: **Mingming Zhang**

Supervisor: **Prof. Zhiguo Wang**

School: **School of Optoelectronic Science and Engineering**

摘 要

总线(Bus)是通信系统中各功能组件信息交互的通讯网络,是高性能通信器件的关键技术之一。在高速数据处理系统中,系统总线的性能瓶颈是限制系统性能的重要因素之一。PCI Express 总线技术是第三代系统总线技术,是目前高速设备支持的主流 IO 总线。PCI Express 总线技术采用了串行差分信号传输的模式、端对端的传输架构,具有高频、高带宽、低功耗等性能优势。在高性能 FPGA 芯片中集成 PCI Express 总线接口能够有效提高芯片在高速数据传输环境下的系统性能,针对这一领域的研究具有重要的工程意义。

本文基于一项千万门级FPGA芯片设计工程,研究了高性能FPGA器件中PCIE模块集成的技术路线。本文设计的芯片是一款自主研发国内领先的千万门级高性能FPGA器件,其采用了业界先进的28 nm工艺,集成的晶体管规模达到上亿级,芯片包含可编程单元达到693120个,集成了三个PCIE IP核,支持PCI Express3.0规范,支持最多8个传输通道、单通道最大8 GT/s的传输速率。文中所设计的FPGA芯片采用了业内经典的GRM结构,简化了PCIE IP核的集成工作。作者基于软件提供的链接约束关系,在FPGA芯片的顶层将PCIE IP核周围的功能模块的输入输出端口和FPGA中对应的互连资源的端口进行对应链接,在电路网表文件上实现了PCIE IP核的集成。文中提出了一套高效的PCIE核功能验证方法;作者基于Verilog语言搭建了一套符合灰盒验证思想的验证平台。开发的验证平台通过TASK函数在平台内封装了可重用的PCI Express协议验证向量指令集;作者通过对PCIE IP底层网表进行行为建模,在VCS软件中实现了快速仿真验证;整体验证环境采用了端对端的电路仿真模型,通过上游设备发送相关的事务激励,由下游设备接收激励数据包,产生响应;仿真完成后通过调用关键内部信号和IP核外部输入输出端口的信号,分析功能结果。芯片流片后,作者基于芯片验证阶段的工作,开发了FPGA芯片的应用测试平台,完成了对流片的FPGA样片的应用测试,实现了对FPGA芯片的PCI Express接口及其配套的CMT、SERDES、CLB、SRAM等模块调用;通过带宽示波器取样,得到了实时的信号传输质量眼图和误码率分析图,在实际测试环境中验证了FPGA芯片中PCI Express接口的可靠性。

本文成功的完成了FPGA芯片中PCIE模块的集成、验证以及流片后的应用测试工作;搭建了支撑验证工作的验证平台,最终验证的功能覆盖率达到了100%;流片后的FPGA芯片经过样片应用测试,其PCIE接口在实际工作环境中,在8 GT/s、5 GT/s、2.5 GT/s三种传输模式下传输质量眼图的眼宽和眼高分别为125 ps、400 mV,

200 ps、400 mV, 400 ps, 400 mV; 在 10^{-12} 误码率下的眼宽均大于 0.6 个 UI; 以上数据表明 FPGA 芯片的 PCIE 接口在三种模式的信号传输质量符合设计要求。

关键词: PCI Express, FPGA, 总线技术, 功能覆盖率

ABSTRACT

The bus is a communication network for the information exchange of various functional components in the communication system and one of the key technologies of high-performance communication devices. In high-speed data processing systems, the system bus always becomes the bottleneck which limits the system performance. PCI Express bus technology is the third-generation system bus technology and the mainstream IO bus supported by current high-speed devices. PCI Express bus technology adopts serial differential signal transmission mode and end-to-end transmission architecture, and it has performance advantages such as high frequency, high bandwidth and low power consumption. Integrating the PCI Express bus interface in a high-performance chip can effectively improve the system performance of the chip in a high-speed data transmission environment. Research in this field has important engineering significance.

Based on a project of FPGA chip design with tens of millions of gates, this paper studies the technical route of PCIE module integration in high-performance FPGA devices. The chip designed in this article is a self-developed domestically leading high-performance FPGA device with tens of millions of gates using the industry's advanced 28nm process. The amount of integrated transistors reaches hundreds of millions, and the chip contains 693,120 programmable units and three PCIE IP cores. Each PCIE core supports PCI Express 3.0 specification which supports up to 8 transmission channels with a single channel transmission rate of 8GT/s max. The FPGA chip design in this article uses the classic GRM structure, which simplifies the integration of the PCIE IP core. Based on the link constraint relationship provided by the software, the author links the input and output ports of the functional modules around the PCIE IP core with the corresponding interconnection resource ports in the FPGA on the top layer of the FPGA chip, and realizes the PCIE IP integration on the circuit netlist file. The author builds a verification platform based on gray-box verification using Verilog. The reusable PCI Express protocol verification vector instruction set is encapsulated in the platform through TASK function. Through behavior modeling of PCIE IP, fast simulation verification is realized in VCS software. The entire verification environment adopts an end-to-end circuit simulation model, and

relevant transaction stimulus are sent through upstream devices, then downstream devices receive stimulus data packs and generate responses. Once the simulation is completed, the function result can be analyzed by calling key internal signals and signals from the external input and output ports of the IP core. After the chip is taped out, the author completes the application test of the taped FPGA using the FPGA chip application test platform. The application test project realizes the calling of the PCI Express interface of the FPGA chip and its supporting CMT, SERDES, CLB, SRAM and other modules. Then real-time signal transmission quality eye diagrams and bit error rate analysis diagrams are obtained by a bandwidth oscilloscope. After that the reliability of the PCI Express interface in the FPGA chip is verified through actual tests.

This article successfully completes the integration and verification of the PCIE module in the FPGA chip as well as the application test after the tape-out. A verification platform is built to support the verification work, and the functional coverage of the final verification reaches 100%. In the test process of the PCIE interface of the FPGA chip after the tape-out, the eye widths and eye heights of the transmission quality eye diagram are 125ps and 400mV in 8GT/s mode, 200ps and 400mV in 5GT/s mode together with 400ps and 400mV in 2.5GT/s mode. The eye width under 10^{-12} bit error rate is greater than 0.6 UI. Statistics above prove that the signal transmission quality of the PCIE interface of the FPGA chip in the three modes meets the design requirements.

Keywords: PCI Express, FPGA, bus technology, function coverage

目 录

第一章 绪论	1
1.1 论文研究背景与意义	1
1.2 国内外研究现状和发展态势	2
1.3 本文的组织结构及内容安排	3
第二章 高速总线技术与千万门级高性能现场可编程门阵列器件	5
2.1 高性能现场可编程门阵列	5
2.2 PCI EXPRESS 高速串行总线概论	6
2.2.1 PCI EXPRESS 的经典拓扑架构概述	6
2.2.2 PCI EXPRESS 内部的分层结构	7
2.2.3 PCI EXPRESS 的包	10
2.2.4 链路训练和初始化	13
2.3 本章小结	15
第三章 千万门级 FPGA 的设计与 PCI EXPRESS 模块的集成	16
3.1 FPGA 芯片的结构介绍	16
3.1.1 典型 FPGA 芯片的工艺	16
3.1.2 典型 FPGA 芯片的架构	16
3.2 高性能 FPGA 芯片结构与 PCI EXPRESS 模块的集成	18
3.2.1 PCI EXPRESS 模块的互连资源	20
3.2.2 PCI EXPRESS 核的结构	20
3.2.3 功能配置模块	23
3.2.4 输入输出互联模块	25
3.2.5 FPGA 逻辑	26
3.2.6 CMT 模块	26
3.2.7 SERDES 模块	27
3.3 本章小结	27
第四章 PCI EXPRESS 核的验证方法研究	28
4.1 通用验证平台的架构	28
4.1.1 验证仿真电路环境	28

4.1.2 验证环境的结构	29
4.2 验证激励向量介绍	31
4.3 验证结果分析	33
4.3.1 PIC EXPRESS 系统的初始化和链路训练的验证	33
4.3.2 基本事务的验证	37
4.3.3 时钟和复位仿真	41
4.3.4 电源管理的验证	43
4.3.5 中断功能的验证	44
4.3.6 LATENCY TOLERANCE REPORTING 消息的验证	48
4.3.7 ALTERNATIVE ROUTING-ID INTERPRETATION 功能验证	49
4.3.8 OPTIMIZED BUFFER FLUSH/FILL 消息的验证	51
4.3.9 基地址寄存器的验证	52
4.3.10 极性倒置的验证	54
4.4 本章小结	56
第五章 千万门级 FPGA 中 PCI EXPRESS 接口应用测试	58
5.1 架构设计	58
5.2 子模块设计	59
5.2.1 JTAG 接口模块	59
5.2.2 EP 模块	62
5.2.3 RP 模块的设计	64
5.3 应用测试流程简介	68
5.4 应用工程的建立	69
5.4.1 约束策略	69
5.4.2 布局布线结果	69
5.4.3 时序报告和功耗分析	71
5.5 应用平台的搭建和测试	72
5.5.1 功能应用	72
5.5.2 不同模式下的传输质量分析	75
5.6 本章小结	79
第六章 总结与展望	80
6.1 文章总结	80
6.2 未来展望	81

致 谢.....	82
参考文献	83
附录	86

第一章 绪论

1.1 论文研究背景与意义

现场可编程门阵列（Field Programmable Gate Array, FPGA）是一种流片后可以通过工具二次编程实现某一些具体功能的芯片技术。其与中央处理器芯片（Central Processing Unit, CPU）、存储器芯片、数字信号处理芯片（Digital Signal Processing, DSP）统称为四大通用集成电路芯片^[1]。现场可编程门阵列器件（下称 FPGA 芯片）由早期的可编程逻辑阵列（Programmable Array Logic, PAL）、通用逻辑阵列（Generic Array Logic, GAL）、可编程逻辑器件（Programmable Logic Device, PLD）等编程器件发展衍生而来。FPGA 芯片有效缓解了传统可编程器件可编程逻辑资源不足的瓶颈，同时弥补了全定制电路（Application Specific Integrated Circuit, ASIC）芯片开发周期长、单一产品前期研发成本投入高等缺陷^[2, 3]。

FPGA 芯片的主要使用场景分为两类。一类是采用 FPGA 芯片设计半定制电路芯片和 ASIC 芯片。FPGA 芯片中提供了大量可编程的逻辑资源，通过软件开发和开发工具，设计人员可以在 FPGA 开发平台上进行某款新品的前期开发设计、验证、测试，基于 FPGA 芯片的可反复擦除、编程烧写的特点，通过 FPGA 芯片进行设计的研发成本低、容错度高；同时，采用 FPGA 芯片进行专用定制电路的开发，在开发工具内可提供完整的综合、布线算法和优化策略，省去了传统设计时后端设计的大量工程周期^[4, 5]；采用 FPGA 芯片进行某一新型芯片前期的产品研发已是业内主流研发手段^[6]。

FPGA 的另一类使用场景是异构并行计算领域^[7, 8]。近年来，随着人工智能、深度学习、大数据、云计算等基于大算力的新兴技术兴起，硬件领域算力供应方向的研究不断发展。一方面，FPGA 芯片中有大量可编程逻辑资源可以通过开发工具完成对先进算法的晶体管级实现，高效的算力供给能力使得其在并行计算领域得到了空前的关注；另一方面，传统的算法实现 ASIC 芯片一款芯片只能实现一种算法，而 FPGA 芯片对于算法的实现更灵活。由于 FPGA 具备可以重复编程的特点，算法的迭代可以通过 EDA 软件及时调整 FPGA 内部的晶体管级的逻辑关系实现，FPGA 芯片有助于高效测试硬件算法的性能。在这些新兴领域初期研究阶段，高性能 FPGA 芯片有着不可替代的重要作用^[3]；同时由于 ASIC 芯片在异构并行计算领域实现过程中研发成本高、周期长，FPGA 芯片技术在这一方面具备独特的优势。

FPGA 芯片在算力供应领域需要处理大量的数据，同时需要一些定制单元提供数据处理的控制逻辑，这就要求 FPGA 芯片内部有一套高性能的数据互连通路结构进行数据的传递，在传统通信设备中这一结构叫做系统总线。实际上，内部互连总线的效率已经成为了限制 FPGA 芯片计算性能的主要瓶颈之一，高性能 PCI Express (Peripheral Component Interconnection Express, PCI Express) 总线在 FPGA 芯片中的集成技术应运而生。

PCI Express 是第三代 IO 总线技术，由 PCI-SIG 组织在 2002 年发布。PCI Express 总线的提出是总线技术发展历程的一次跨越式进步，其系统架构、链接方式、信号载体等多方面相较于上两代总线发生了根本性的变革^[9]。表 1-1 是各代 IO 总线的基础信息介绍。

表 1-1 各代 IO 总线的迭代情况^[10]

总线代数	总线名称	描述			
		最大吞吐量	传输速率/ 频率	最大支持 通道数	传输方式
第三代	PCI Express5.0	63.0 GB/s	32 GT/s	16lane	串行
	PCI Express4.0	31.5 GB/s	16 GT/s	16lane	串行
	PCI Express3.0	15.8 GB/s	8 GT/s	16lane	串行
	PCI Express2.0	8 GB/s	5 GT/s	16lane	串行
	PCI Express1.0	4 GB/s	2.5 GT/s	16lane	串行
第二代	PCI-X	2 GB/s	533 Mhz	-	64 位并行
	PCI	533 MB/s	66.6 Mhz	-	64 位并行
第一代	VESA	133 MB/s	33 Mhz	-	32 位并行
	ISA	8 MB/s	4.7 Mhz	-	8 位并行

随着版本的迭代，PCI Express 总线采用的变革性的串行端对端传输架构，使得其性能在近年来呈指数级增长。最新发布的 PCI Express5.0 协议中，总线带宽突破 63.0 GB/s，单通道传输速率最高可达 32 GT/s。在不久的将来，总线带宽的容量将远远超过高性能 FPGA 器件的传输需求。总线技术的发展，补齐了高性能器件的数据交互的短板，为今后超高速数据处理和传输器件的研究发展提供了总线技术的支持^[2, 11, 12]。

1.2 国内外研究现状和发展态势

随着高性能并行计算对于人工智能和大数据等新兴技术行业的重大意义，高

性能 FPGA 芯片的发展近年来步入了快车道。主流的 FPGA 厂商的 FPGA 产品基本突破千万门级，支持 PCI Express 串行接口技术，工艺尺寸突破 28 nm。例如，FPGA 市场的龙部厂商，赛灵思（Xilinx）公司（2020 年 10 月被 AMD 公司收购）的 7 系列 FPGA 产品：Virtex、Kintex、Artix、Spanrtan 均采用 28 nm 的先进工艺，Virtex7 型和 Kintex7 型 FPGA 芯片均搭载了 PCI Express 串行接口技术；2019 年 4 月，美国 Intel 公司发布了 10 nm 工艺的 FPGA 芯片 Agilex，其搭载了 PCI Express5.0 总线，是目前搭载的串行接口技术最先进的量产 FPGA 芯片；国内的相关研究起步较晚，公开发布的产品中，上海安陆科技的 Eagle 系列 FPGA 芯片采用了 55 nm 的低功耗工艺技术，搭载了 1 Gbps 的 LVDS 差分传输接口技术；西安智多晶公司的 Seal5000 系列 FPGA 采用了 28 nm 的先进工艺，支持 1.2 Gbps 的 DDR3，支持 1.2 Gbps 的 LVDS 传输接口；深圳紫光同创公司发布的 Titan 系列高性能 FPGA 使用的是 40 nm 的工艺技术，是为数不多支持 PCI Express 接口的 FPGA 产品，其搭载了基于 PCI Express2.0 规范的串行接口，单通道最大传输速率达到 5 GT/s。

1.3 本文的组织结构及内容安排

本文基于实习公司的千万门级 FPGA 设计研发项目中的 PCIE 模块的集成验证工作，完成了 PCI Express 定制 IP 在 FPGA 芯片中的集成工作和基于 PCI Express3.0 协议的 IP 核的功能验证工作；在芯片流片后，作者使用实习公司可重用的通用验证应用测试平台，实现了使用设计的 FPGA 芯片样片中 PCI Express 接口访问 DDR3 存储颗粒的应用测试工作。

本文的章节结构安排如下：

第一章 本章对作者所做工作的工程背景、研究意义、研究现状进行了大致的描述。

第二章 本章是对本文所涉及的主要器件的基本理论知识进行了较为充分的论述，有助于对于下文中的工作内容提供理论支持。

第三章 本章是本文研究的 FPGA 芯片的整体工艺架构分析和 PCI Express Core 的集成工作的论述。

第四章 本章讲述了对于定制 PCI Express Core 的直接验证方法的研究，搭建了通用的验证平台，对 PCI Express Core 功能的研究进行了详细的分析。

第五章 本章是在 FPGA 芯片流片后，参考第四章完成的验证平台相关的工作，完成了一款基于高性能 FPGA 芯片的支持 PCI Express3.0 协议的 DDR3 读写访问的应用场景设计，完成了芯片的冒烟测试，同时收集了芯片集成的 PCI Express 接口的信号传输质量，最终结果符合设计目标的要求。

第六章 本章总结了本文所做工作解决的困难和其工程意义，同时对研究方法中的不足和后续研究工作的展开提出了意见。

第二章 高速总线技术与千万门级高性能现场可编程门阵列器件

随着摩尔定律的发展，集成电路的工艺技术在过去半个世纪快速迭代，相关的 IC 产品性能的逐步提高^[11]。现场可编程门阵列器件（下称 FPGA）的规模与性能在过去三十年得到了极大的提高，其需要处理的数据量也呈指数级增长。随着高性能 FPGA 芯片内部数据流量的增加，其对高速总线的性能与可靠性要求亦逐步提高^[13]。

总线技术是连接计算系统各个计算单元的基本结构。在一个计算系统中，数据总线连接了系统中所有的逻辑计算单元和存储单元，所有参与处理的数据在系统内部通过高速总线传递。随着工艺技术的发展，一般计算机系统中计算单元的计算能力和存储单元的存储能力都得到了极大的提高。在这种背景下，总线技术成为了制约器件性能的关键一环。随着产品的迭代升级，计算系统都需要配套高总线带宽、快传输速率、高可靠性的总线技术。

2.1 高性能现场可编程门阵列

现场可编程门阵列由可编程逻辑资源、内部互连资源、可编程输入输出端口、专有定制 IP 资源四部分组成。其中专有定制资源包含多种为提升 FPGA 专项性能的不同功能的定制 IP，例如提升运算性能的 DSP 模块、时钟产生的 GCLK 模块、块存储单元 BRAM 模块、高速串行接口 PCI Express 模块。

高性能 FPGA 芯片内部集成了大量的逻辑资源，同时提供了大量可供调用的专用 IP。高性能 FPGA 的开发就是整合使用这些资源，形成某一种具体的功能；形成的功能越复杂，使用的资源越多。高性能 FPGA 技术的发展伴随着内部逻辑资源的丰富，现在业内最顶级的 FPGA 芯片已经达到了千万门级的规模，晶体管数量达到数亿个，逻辑单元数量最多可以达到数十万。

同时，FPGA 的开发使用需要配套的软件开发工具，对于 FPGA 芯片内部的资源调用是通过 EDA 软件进行的。业内的主流软件工具有 ISE、Vivado 等，对 FPGA 的开发通过硬件编程语言在软件中实现，软件根据其内部的 FPGA 资源库对硬件编程代码进行综合、布线，将最终的实现的方案数据通过 bit-stream 的形式保存，FPGA 芯片通过数据配置完成和软件工具的交互；FPGA 完成配置的过程即为读取 bit-stream 的过程，FPGA 内置的配置机制可以将所有的 bit-stream 信号映射到芯片内部的配置点存储单元、布局布线资源等内部资源中，通过逻辑组合，实现软件中形成的功能。

2.2 PCI Express 高速串行总线概论

本节将全面的介绍 PCI Express 高速串行接口技术，分别从其拓扑架构、分层结构、数据包等方向进行剖析。以此对后文 PCI Express IP core 的集成应用和验证测试工程提供理论指导。

2.2.1 PCI Express 的经典拓扑架构概述

PCI Express 总线技术是由 PCI-SIG 提出的第三代 I/O 总线^[2]。区别于前两代 I/O 总线，PCI Express 总线提出了全新的拓扑架构，如图 2-1 所示。PCI Express 的经典拓扑架构要求链路采用高速差分串行传输的模式，端对端的设备连接方式（多设备使用 Switch 结构连接），这使得新一代的总线技术能够提供更大的总线带宽和更高的时钟频率，使得系统内部的信号传递有了更高的效率和可靠性。

由图 2-1 所示，在 PCI Express 的经典拓扑结构中，其组件为中央处理器单元、根复合体（Root Complex, RC）、交换开关（Switch）、桥接设备、存储器（Memory）、终端设备（End Point）。同时，根据 PCI Express 协议规定，PCI Express 总线系统需要向前兼容上一代总线设备，所以系统中需要挂载 PCI Express to PCI/PCI-X 桥接设备（下称桥设备）。通过桥设备，系统可挂载支持上一代 I/O 总线的 PCI、PCI-X 设备^[14, 15]。

在 PCI Express 经典拓扑架构中，根复合体和交换开关是两个核心的组件，下文对其进行详细说明。

2.2.1.1 根复合体

根复合体（Root Complex）在系统中可以外接终端设备、桥接设备、存储器设备和中央处理器^[16]。其在 PCI Express 总线系统中需要完成存储器域和 I/O 总线域的地址转换和物理信号转换。一般来说，在 PCI Express 体系架构中，终端设备和 CPU 无法完成直接的通信，终端设备对 CPU 的访问实质上都是对根复合体的访问，在根复合体中，相应的处理单元将有底层终端设备路由的事务信息进行处理翻译，将和 CPU 相关的信号进行翻译然后发送给 CPU，这样来完成一次终端设备对 CPU 的访问^[17-19]；同时，在其他一些特设的应用领域，可以外加一些特殊的应用功能。根复合体的设计，是 PCI Express 系统设计中最为复杂和相对混乱的一块；在不同的应用系统环境中，设计人员都可以根据需求对根复合体进行相应的设计支持。

2.2.1.2 交换开关

交换开关（Switch）的主要作用是拓展链路设备数量^[20]。由于 PCI Express 协

议规定了 PCI Express 总线的端对端传输模式，这样使得在一条 PCI Express 链路上，有且仅有两个设备可以采用这种端对端的模式进行通信。交换开关的加入，使得一条 PCI Express 链路可以挂载多个设备。

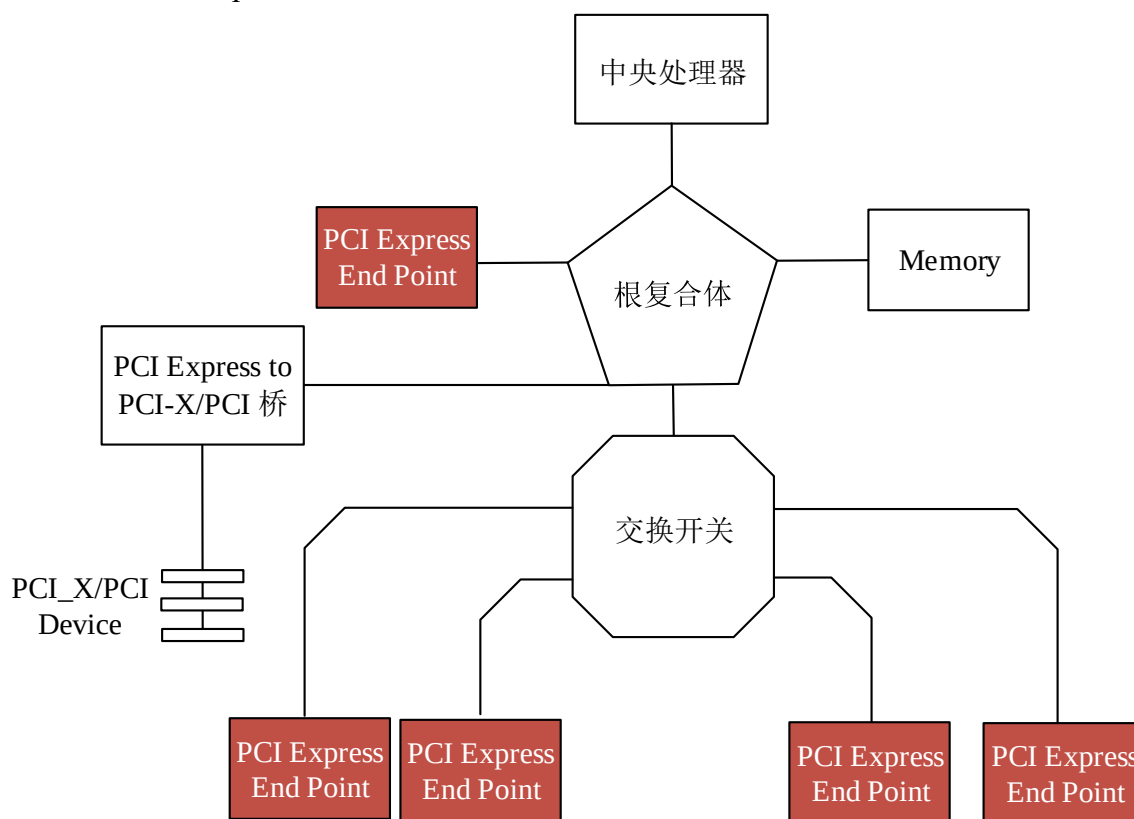


图 2-1 PCI Express 经典拓扑架构^[10]

一个标准的交换开关由一个上游端口和多个下游端口组成，其内部结构如下图 2-2 所示。交换开关的实质是由多个 Virtual P2P Bridge 组成，Virtual P2P Bridge 的上游是 Virtual Bus，下游是 PCI Express 端口^[21, 22]。对于交换开关来说，其内部 Virtual P2P Bridge 的数量决定了其链路拓展能力。不过，交换开关内部的 Virtual P2P Bridge 的数量并不是可无限增加的，因为其还需要考虑总线传输质量的问题。在 PCI Express 体系架构中，总线资源是以第一级交换开关做仲裁分配的，如果某一个交换开关的挂载资源太多，其挂载的资源所享有的总线资源就越少，同时对仲裁算法的设计和 TC/VC 映射规则的设计也会带来麻烦。

2.2.2 PCI Express 内部的分层结构

根据 PCI Express 协议规范要求，PCI Express 的端点 IP Core 采用了分层的设计结构。如图 2-3 所示，由上至下分别为设备核心、事务层、数据链路层和物理层。

其中，PCI Express IP Core 内包含事务层、数据链路层和物理层^[23]。

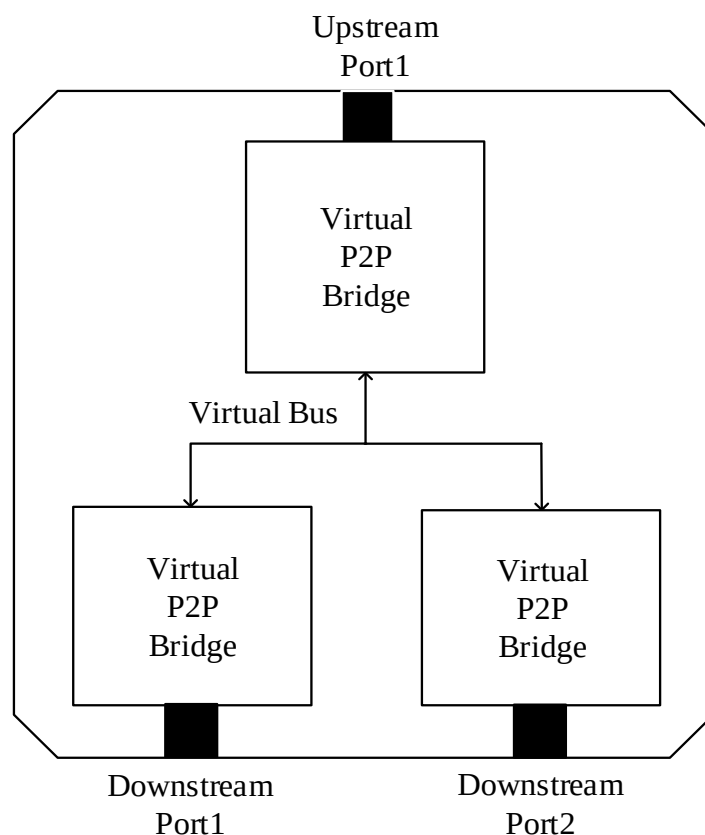


图 2-2 交换开关内部结构

2.2.2.1 设备核心

设备核心是 PCI Express Core 之外的组成部分，是大部分 PCI Express 事务发起的起点和响应终点。设备核心的主要功能有：和设备软件相结合，为事务包的产生提供必要的信息，包括事务类型、地址数据、事务数据等，这些信息将被送往 PCI Express 内部的事务层；设备核心同样要接收来自事务层的信息，包括事务包的类型、事务包的地址数据、接收的数据量等信息。

2.2.2.2 事务层

在 PCI Express 的分层结构中，事务层分为发送部分、接收部分、流控制和电源管理部分。事务层一方面负责接收系统内部（主要是设备核心）发送过来的请求信息对其进行分析，整理组织成 PCI Express 规范格式中的 TLP(Transaction Layer Package, TLP) 数据包，另一方面，事务层还有对称的处理方式：事务层会将传递过来的 TLP 包进行分解、拆除，将 TLP 中有效信息进行翻译处理，并将涉及设备核心功能的信息发送给对应的设备核心，由设备核心来执行 TLP 包相关功能请

求。

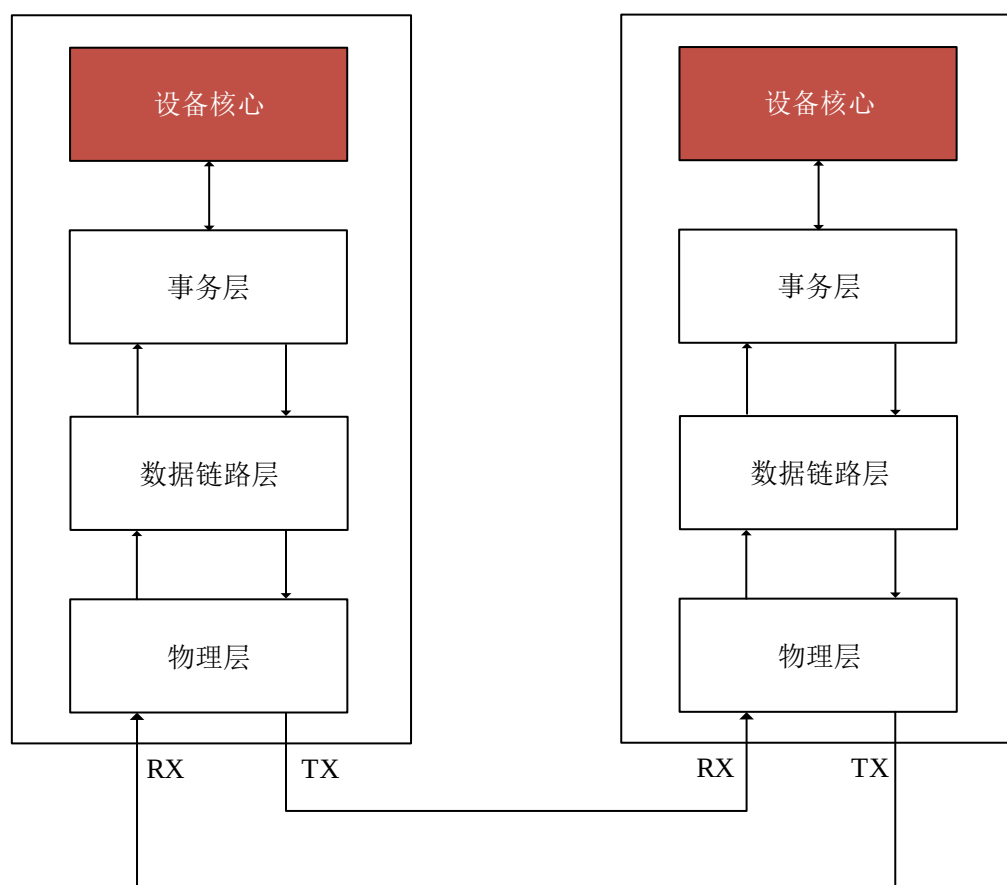


图 2-3 PCI Express 的分层结构

此外，事务层还会执行 PCI Express 流控制、服务质量和仲裁相关的业务，优化数据在 PCI Express 链路传输效率和传输质量；电源管理功能是事务层镶嵌的另一个重要功能，是芯片低功耗设计的具体实现。其具体行为是：事务层协调系统的各个模块在某些情况下进入非全功耗的系统状态，对系统工作状态进行自主处理，从而达到系统的功耗降低。

2.2.2.3 数据链路层

数据链路层是连接事务层和物理层的中间层，其主要作用是保证事务包高效、可靠的传输到对端设备。数据链路层需要有严格的错误检查机制、数据备份、重传机制，来保证数据传递过程中不出错、不丢失。

数据链路层通过收发 DLLP 数据包实现链路管理的功能，包括 TLP 响应、电源管理和流控制信息交互。数据链路层产生的 DLLP 包在一个链路中的两个相邻设备之间链路层传递。所以数据链路层的功能实现一般和对端设备的工作状态相

关，对端设备的数据接收情况会直接影响当前设备数据链路层的工作状态。

在数据链路层中执行数据的传递的时候，从事务层传递过来的 TLP 事务包会被加上序列号检错的链路层校验码 LCRC (Link Cyclical Redundancy Check)，然后通过链路中的 ACK/NAK (Acknowledge/Negative Acknowledge) 传输协议，完成数据在链路层的准确传输^[24]。ACK/NAK 的协议实现较为复杂，其本质是上游设备的握手行为，只有当数据在对端设备准确接收，当前设备才会正常发包，一旦链路传递过程中出现误码，链路层的重传机制就会介入，中断事务层发包，同时通过查询序列号向对端设备重新发送接收失败的数据包^[25]。

数据链路层是 PCI Express 体系中保障数据完整性和系统工作稳定性的重要结构，它在模块设计时其占用的资源最多、设计复杂度最大。而在验证阶段，这一层功能的实现很少需要特殊实现，观察事务层和物理层的端口数据即可实现对数据链路层功能的大部分覆盖^[26]。

2.2.2.4 物理层

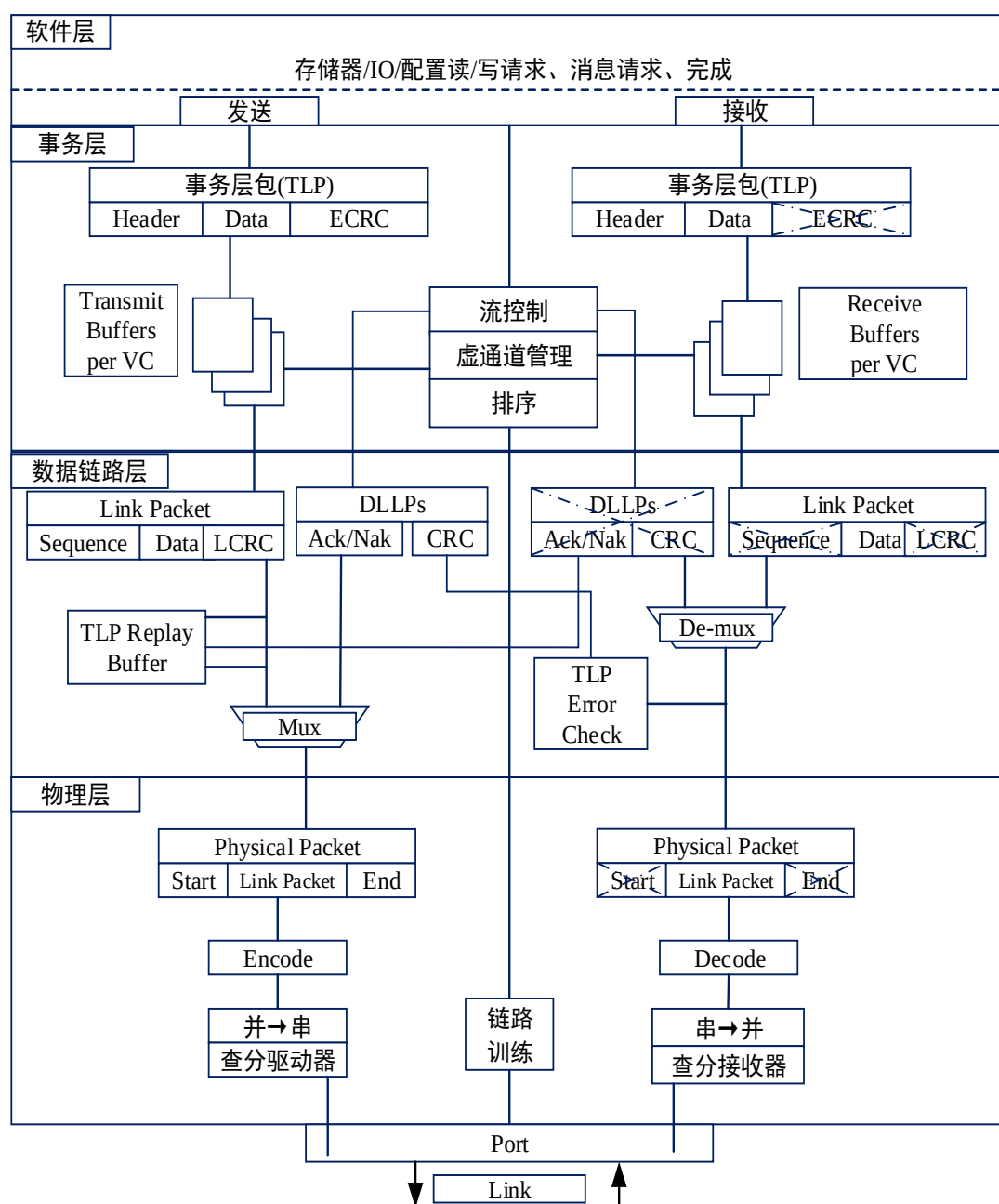
物理层是 PCI Express Core 的最外层，负责连接数据链路层和对端设备。在广义的定义中，物理层包括逻辑物理层和电气物理层。物理层的功能包括给接收的 TLP、DLLP 增加物理层标志位数据，传递接收的各类事务包到对端设备；物理层会产生和拆解物理层包 (Physic Layer Packet, PLP) 数据包，通过 PLP 包实现物理层的功能，包括链路训练、系统初始化、时钟补偿等^[27, 28]。

2.2.3 PCI Express 的包

在 PCI Express 的协议规范中要求，系统中的各类事务均以事务包的形式传递。由上文所述的 PCI Express 的层次结构，我们将各个层次中的各类事务包的关系汇总如图 2-4 所示。

如下表 2-1 所示，软件层中提供了基本的请求类型，包括存储器请求、IO 请求、配置读/写请求和完成请求。根据地址空间、传送方式和读写类型可对各类请求分类。

Non-Posted 传送和 Posted 传送的区别在于是否需要等待反馈的完成数据。对于 Non-Posted 传送类型的请求事务，发送端在数据包发送后需要等待接收端的回复，同理，接收端收到 Non-Posted 的请求事务，需要立即反馈一个完成事务包给到对端设备；而对于 Posted 传送方式，发送设备在发出 Posted 包后，这个事务在当前设备就已经执行完成；接收设备在收到 Posted 包后，根据包中信息完成相应的请求，这个事务即执行完毕。

图 2-4 PCI Express 层次结构中的包^[10]

2.2.3.1 事务层包

在事务层中，PCI Express Core 需要根据设备核心传递的数据产生 PCI Express 的基本数据包——事务层包（Transaction Layer Package，TLP）。TLP 是 PCI Express 体系架构中最基本的数据包单元，它存在于整个 PCI Express 事务传递过程中^[10, 29]。表 2-2 说明了 TLP 包的组成部分及其作用。

表 2-1 PCI Express 请求类型分类^[10]

事务类型	读写类型	传送方式	说明
Memory	Read	Non-Posted	在存储器寄存器中读取数据
	Write	Posted	往存储器空间中写入数据
IO	Read	Non-Posted	在 IO 空间读取数据
	Write	Non-Posted	在 IO 空间写入数据
Configuration	Read	Non-Posted	在配置空间读取数据
	Write	Non-Posted	在配置空间写入数据
Message	None	Posted	由设备发出的专用信号，如中断信号、

TLP 包中，其包头内容是最丰富的。其格式随着事务类型变化而变化，TLP 包头的通用字段如下图 2-5 所示。

2.2.3.2 数据链路层包

在数据链路层中，除了需要传递由事务层发送过来的事务包，还会产生数据链路层包—DLLP (Data Link Layer Packet)。DLLP 包根据功能分为流控制 DLLP、ACK/NAK DLLP、电源管理 DLLP 以及厂商定义 DLLP 包^[30]。DLLP 的出现都是以支持数据链路层的功能为目的。DLLP 包的基本通用结构如下图 2-6 所示。

表 2-2 TLP 包的结构及说明

事务包的结构	容量	产生	拆解	说明
包头 (Header)	3 DW or 4 DW	设备核心提供信息，事务层组成 TLP	事务层	不同事务包的包头格式不同，由事务类型、接收地址、数据大小等关键信号决定
数据 (Data)	0-1024 DW	设备核心提供信息，事务层组成 TLP	终端设备	可选字段，有效载荷为 0--1024DW
摘要 (Digest)	1 DW	事务层	事务层	选字段，大小为 1DW
序列号 (Sequence)	1 DW	数据链路层	数据链路层	链路中传递的事务包的标记，ACK/NAK 协议的重要承载
LCRC	16 bit	数据链路层	数据链路层	数据链路层的错误检查机制
帧开始 (STP)	1 DW	物理层	物理层	物理层数据包的开始标志位
帧结束 (END)	1 DW	物理层	物理层	物理层数据包的结束标志位

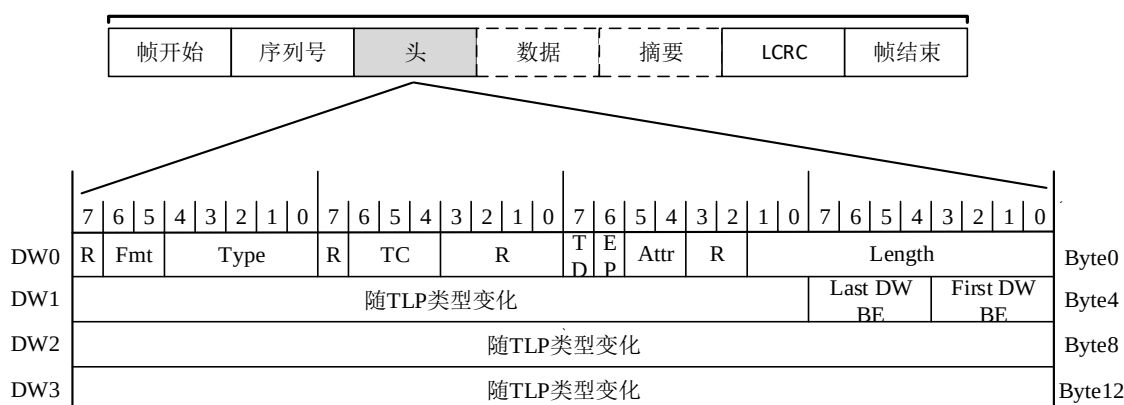


图 2-5 TLP 包的通用头字段



图 2-6 DLLP 包的通用包格式

区别于 TLP 包，DLLP 不带数据，只负责完成某项具体的功能，所以包的结构更为简单；DLLP 包只在相邻的两个设备之间传递，所有 DLLP 包不需要路由，包中无地址信息^[31, 32]。

2.2.3.3 物理层包

在物理层中，数据包一部分是传递由事务层和数据链路层的数据包对于其施加开始帧和结束帧，还有一部分完成物理层工作的数据包。物理层包是一种由物理层硬件自主产生的数据包，其存在形式为有序集（Ordered Set）。物理层包主要完成物理层的链路训练和时钟偏差补偿等功能^[33, 34]。

2.2.4 链路训练和初始化

系统的初始化是指在系统刚上电时，通过自主产生的基本复位（冷复位）使整个 PCI Express 系统内的所有设备的状态机、配置信号、寄存器、可配置端口等可更改的值初始成缺省状态。初始化要求设备能够产生这样一个能全系统识别的复位信号，要求设备要有准确的复位机制。

在 PCI Express 完成复位后，链路即开始进行对于链路的初始化，主要包括链路初始化和链路定向。链路初始化和定向配置是初始化设备的内部配置寄存器、

物理层端口状态机和链路的物理控制机制，使设备后续可以进行数据传输，这一过程也称为链路训练^[35, 36]。链路训练的主要内容有：

1) 配置链路宽度和链路速度

PCI Express3.0 规范要求 PCI Express Core 支持的链路宽度为 X1 至 X16，支持的单通道链路速率为 2.5GT/s 至 8GT/s；在系统配置链路宽度和链路速率时，会根据链路两端设备的最小项进行约束匹配。例如，一端设备支持 2.5GT/s、X8 通道的传输模式、另一端设备支持最大 8GT/s、X1 通道的传输模式，在经过链路初始化后，这个端对端传输的系统会以 2.5GT/s、X1 通道的传输模式进行数据传输^[26]。

2) 链路训练状态机

链路训练由物理层的硬件模块来执行，以状态机的来管理执行这个过程。图 2-7 展示了链路训练的状态机转移图。链路训练过程一共有 11 个状态：检测(Detect)、轮询(Polling)、配置(Configuration)、恢复(Recover)、L0、L0s、L1、L2、热复位(Hot Reset)、回环(Loop Back)和禁用(Disable)。这些状态对应系统在不同状态时物理层的反馈，根据具体事务类链路状态机可按图中所示分为五类。即，链路定向状态：检测、轮询、配置；链路重新定向状态：恢复；电源管理状态：L0、L2；活动电源状态：L1；其他：禁用、外部回环、热复位^[37]。

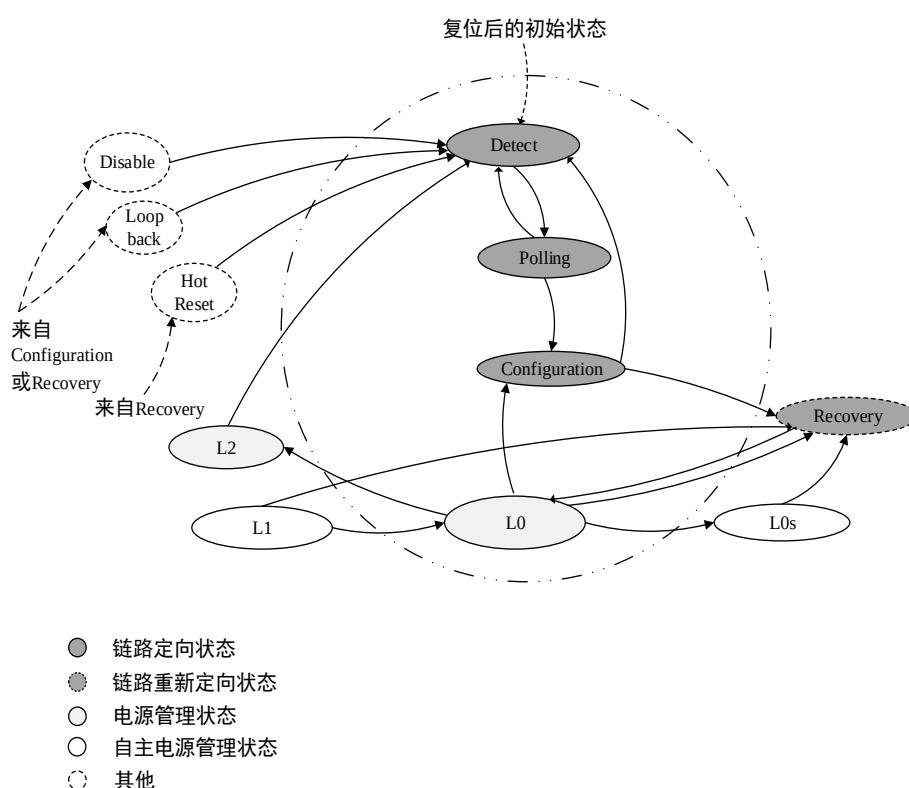


图 2-7 链路训练状态机^[11, 38]

2.3 本章小结

本章先介绍了高性能 FPGA 器件一些基本理论知识、开发流程，为下文 FPGA 的架构设计提供理论支撑；然后本章重点介绍了 PCI Express 总线协议的理论知识，从拓扑架构、分层结构、数据包等方面对 PCI Express 这一总线规范进行了详细的介绍，通过对其理论的知识的阐述，一方面为后期 FPGA 芯片中的集成提供理论依据，另一方面也为第四章 PCI Express 核的验证、第五章基于 PCI Express 规范的应用工程提供了重要的理论参照。

第三章 千万门级 FPGA 的设计与 PCI Express 模块的集成

3.1 FPGA 芯片的结构介绍

3.1.1 典型 FPGA 芯片的工艺

根据 FPGA 芯片工艺的不同,主要分为熔丝与反熔丝技术 FPGA、SRAM(Static Random-Access Memory) 工艺 FPGA 和 FLASH 工艺 FPGA 三种。

熔丝与反熔丝技术是一由两个金属连线的微型非晶硅柱构成,非编程情况下由于非晶硅绝缘,熔丝断开,当编程信号到来,非晶硅在大电压或大电流变为电阻极小的导体,金属熔丝通路^[39]。熔丝的本质相当于开关,由于反熔丝技术开关打开无法关断,编程操作不可逆,所以采用反熔丝技术的 FPGA 芯片都是只能一次编程,在开发使用过程中成本较高。其优势是此类 FPGA 芯片在完成编程配置后抗辐射性好,因此,反熔丝型 FPGA 芯片在 HKHT、核工业等特殊使用环境下具有巨大的使用价值^[40]。

SRAM 型 FPGA 芯片的特点是可以多次重复配置。开发人员只需要在工作时将配置数据配置进 FPGA 中的 SRAM 中,芯片的功能就能实现。在完成工作后,对 SRAM 的数据进行清除,或者进行覆盖,FPGA 即可实现下一次的工作^[39, 41, 42];同时,业内储存技术的高速发展,一定程度上推动了高性能 SRAM 工艺的 FPGA 高速发展。SRAM 型 FPGA 的缺点是掉电数据丢失,无法保存数据。为了让 FPGA 芯片正常工作,一般需要在外部增加配置模块用于配置、保存编程数据。由于 SRAM 技术相对成熟且可靠性高,目前国内外主流 FPGA 厂商的高性能产品往往优先使用 SRAM 型 FPGA 技术,本文研究的千万门级 FPGA 项目亦聚焦这一技术领域。

FLASH 型 FPGA 芯片技术具备了 SRAM 型 FPGA 可反复烧写的优点,同时亦具备反熔丝型 FPGA 抵抗辐射等特点,且编程工作时不用外挂配置模块^[18, 43],但是由于此类 FPGA 的生产工序和工艺相较 SRAM 型 FPGA 更多、更为复杂,使得研发周期和成本大大提高,相关配套的工艺技术不完善,目前市面上 FLASH 型 FPGA 已经不占主流,本文不再过多赘述。

3.1.2 典型 FPGA 芯片的架构

根据内部互连资源的不同,FPGA 芯片可分为行式结构、岛式结构和互连矩阵(General Routing Matrix, GRM)结构^[41, 44]。行式结构是业内早期提出的基础的

FPGA 结构。由于逻辑资源较少，在行式结构布局的 FPGA 芯片中，各个模块的逻辑资源以行的形式布局在芯片内部，行与行通过布局布线资源链接。如图 3-1 所示。

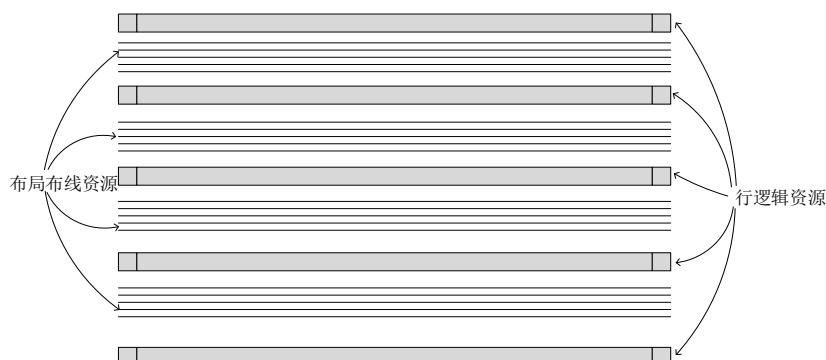


图 3-1 行式架构 FPGA 布局布线图

这种布线是基于早期的 ASIC 专有定制电路的版图结构进行定制的，有着能够精确预估芯片路径延时、布局布线资源结构简单、布局布线优化易于检查等优势。但是由于这种布局方式对于纵向的布局基于反熔丝编程技术的 FPGA 芯片设计与研究，在大规模 FPGA 芯片设计时，需要考虑大量的纵向布局，行式架构无法适应可多次编程使用的 FPGA 芯片发展，取而代之的是岛式布局设计的 FPGA 架构。

岛式 FPGA 架构是一种让芯片内部逻辑资源以二维矩阵形式分布的设计思想。其具体形式为：逻辑资源以二维矩阵的形式分布，布局布线资源在逻辑资源周围以网格状分布^[41, 45]。如图 3-2 所示。

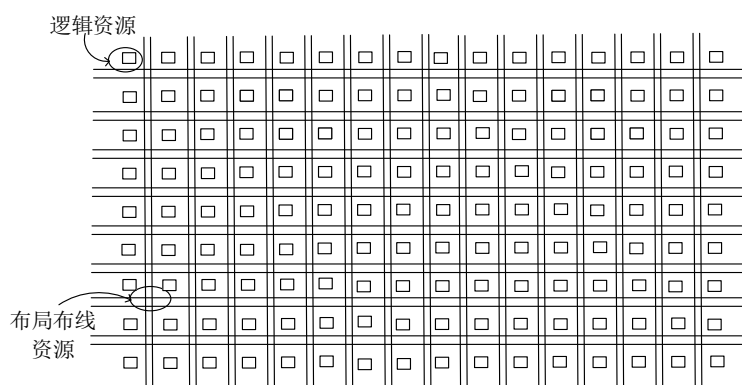


图 3-2 岛式架构 FPGA 布局布线图

随着 FPGA 芯片的发展，岛式布局的 FPGA 产品不足以满足部分高性能产品的要求，业界提出了一种更加丰富的布局布线结构——通用互连矩阵（General Routing Matrix, GRM）。GRM 结构的核心是可重复的 CLB 与 GRM 构成的基本单元，这个单元也叫 TILE，内部结构如图 3-3b 所示。本文所研究的 FPGA 也采用了此种结构。

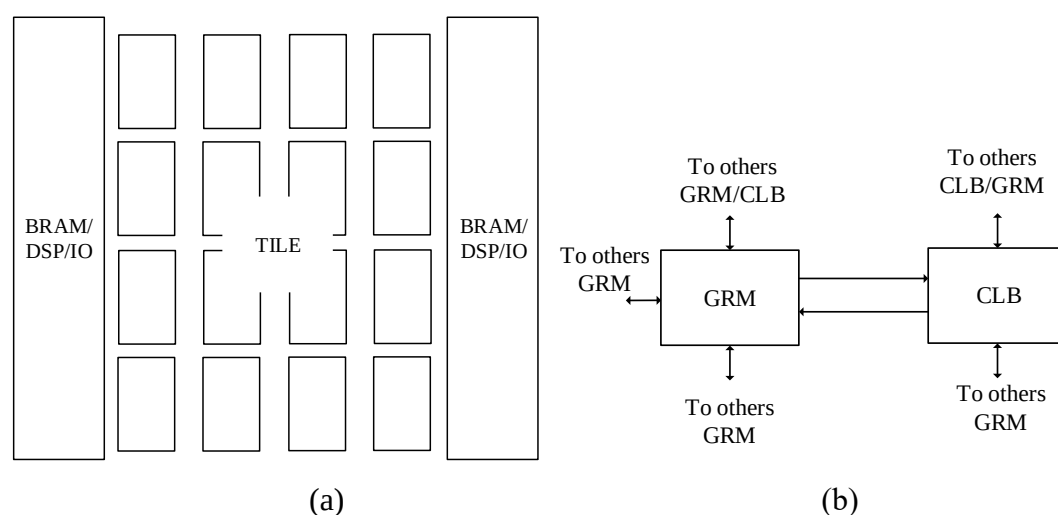


图 3-3 基于 GRM 的 FPGA 架构图。(a) 基于 TILE 单元的 FPGA 架构图；(b) 基于 TILE 单元的 FPGA 架构图

由图 3-3a 所示，在 GRM 结构中，FPGA 由基本单元 TILE 组成。TILE 之间互连，外围的 TILE 可以连接 TILE，也可以连接 BRAM、DSP、IO 等其他资源。TILE 是 GRM 架构中的核心单元，其内部包含一个可编程逻辑块单元（CLB）和通用互联矩阵（GRM）。由于 CLB 是可编程器件，其具体功能和连线可以根据功能进行软件配置；GRM 是专用的互连资源模块，其内部包含各类的开关矩阵，进行其周围组分间的互连。由这两部分资源组成的 TILE 单元具有很高的互连性。使用这种架构的 FPGA 芯片的可编程性能良好，单次编程可编程逻辑资源率高，实现相应功能的逻辑走线算法可更替性高。

3.2 高性能 FPGA 芯片结构与 PCI Express 模块的集成

本文所设计的高性能 FPGA 芯片采用了上文中介绍的基于 TILE 的 FPGA 整体架构，其内部集成的基础模块包括可编程逻辑模块（Configurable Logic Block, CLB）、布局布线资源、嵌入式块随机存取存储器（Block Random Access Memory, BRAM）、时钟管理单元和某些具有特定功能的集成模块（Intellectual Property Core, IP Core）等。基于 TILE 的 FPGA 整体架构的 IP Core 集成方式更加开放，因为该架构下电路可以提供给软件更加丰富的布线资源。由于本文所集成的 PCI Express IP Core 是一个定制电路的 IP，其集成方式通过在 FPGA 的顶层进行子模块例化的方式进行集成。集成的工作是根据 PCI Express 的各个端口功能分析报告和 FPGA 内部的接线端口的说明进行连接，作者通过相应的脚本软件进行了端口连接关系的快速提取，支持了 IP Core 的集成工作。IP 集成后的 FPGA 整体资源分布图如下

3.2.1 PCI Express 模块的互连资源

前文我们讲述了本文所研究的高性能千万门级 FPGA 芯片的架构和工艺技术。由于项目方案要求，本次项目集成了支持 PCI Express3.0 协议的 PCI Express 接口模块。在传统的大型专用定制芯片中，对于 PCI Express 的配置需要阅读大量的相关设计手册和协议，才能完成对 PCI Express 核的某单一功能的配置；需要切换新的传输速度、通道数时，又需要对其配置方式进行新的重组，在一大型系统中集成 PCI Express 核需要耗费极大的精力与研发成本。而在本文研究的高性能 FPGA 芯片中对于 PCI Express 核的集成工作相对更加容易完成，研发人员需要整理 IP 核所需要的资源，例如可编程逻辑单元（CLB），时钟处理单元(Clock Management Tile, CMT)等，将 FPGA 中的功能单元通过互连资源的输入输出进行连接，即可完成 PCI Express 核的集成。用户需要的配置功能将在 FPGA 配套的软件中直接产生，软件可以将实现相关功能需要的资源根据其内部的优化算法进行自动化优化约束，同时将相应的核中的配置点的值设定正确，免去了人为去核验 PCI Express 核不同功能配置点值的复杂冗余的工作。

本文首先从宏观总体上介绍本次项目中 PCI Express 核相关的配套资源情况和相应的集成连接的方式。如图 3-5，图中所示为 PCI Express 集成块的结构图，主要功能模块由 PCI Express 核、PCIE 功能配置单元、FPGA 逻辑单元、时钟处理单元、高速串行链路单元组成，PCI Express 核与其他功能模块之间通过输入输出互联模块进行数据交互。下文会对各配套资源进行简略介绍。

3.2.2 PCI Express 核的结构

3.2.2.1 PCIE 核的描述

本次项目使用了 Xilinx 公司提供的基于 PCI Express 3.0 版本协议的定制 IP 核。该 PCIE 核模块囊括了 PCIE 协议规定主体功能，内部包含事务层，数据链路层以及物理逻辑层。PCIE 核与 SERDES 模块一起组成的 PCIE 集成解决方案支持 1-lane、2-lane、4-lane、8-lane 传输，每个通道传输速率可达 2.5 GT/s(Gen1)、5 GT/s(Gen2)、8 GT/s(Gen3)。

3.2.2.2 PCI Express 核的集成与外围端口设计

PCI Express 核内部是由门电路搭建的支持完整的 PCIE3 协议规范的电路，外围挂载了一些方便用户读取访问的功能接口，如图 3-6 所示。这部分作为一个整体以 PCIE 模块的形式存在。

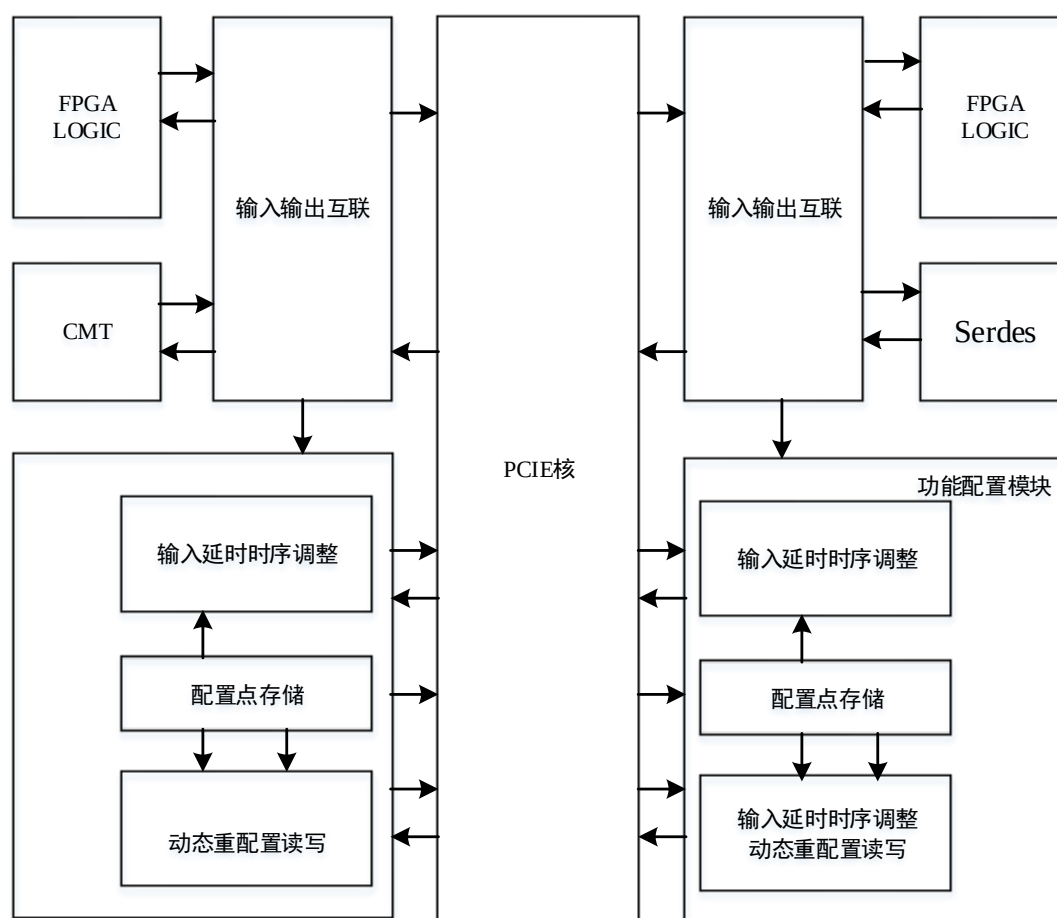


图 3-5 PCIE 集成模块结构图

PCI Express3.0 的核集成了多种功能、类型的外部事务接口,根据功能的不同,我们将其分为 AXI_Stream 接口、配置接口、PCIE 链路接口、时钟复位等其他功能性的接口。

AXI_Stream 接口类是 PCI Express Core 事务层和用户应用层的交互的接口。由于 PCI Express 事务层的事务接口和状态管理接口众多,通过核的事务层接口让用户应用层与 PCI Express 核进行消息传递,需要开发人员详尽了解每一个核端口的功能定义,这会对 IP 核开发人员的工作造成极大的困难。PCI Express IP 解决方案在核外部封装了 AXI_Stream 端口,通过将大部分的状态管理端口进行统一管理,用户只需要在输入端给到事务层事务包的主要内容,数据通过核中集成的 AXI 转 PCIE 桥接模块,相应的事务包数据和状态数据可以根据相应的规则完成配置转换,这极大地简化了相关的工作。表 3-1 给出了 AXI_Stream 接口详细信息。

配置类接口是连接 PCI Express 的各项配置功能和外部配置点的端口。配置接口的状态与 PCI Express 核的功能模式、传输速率等息息相关。配置接口类包含配

置管理接口、配置状态接口、配置接收消息接口、配置发送消息接口、配置控制接口、配置中断控制接口等。表 3-2 给出相关接口的详细说明。

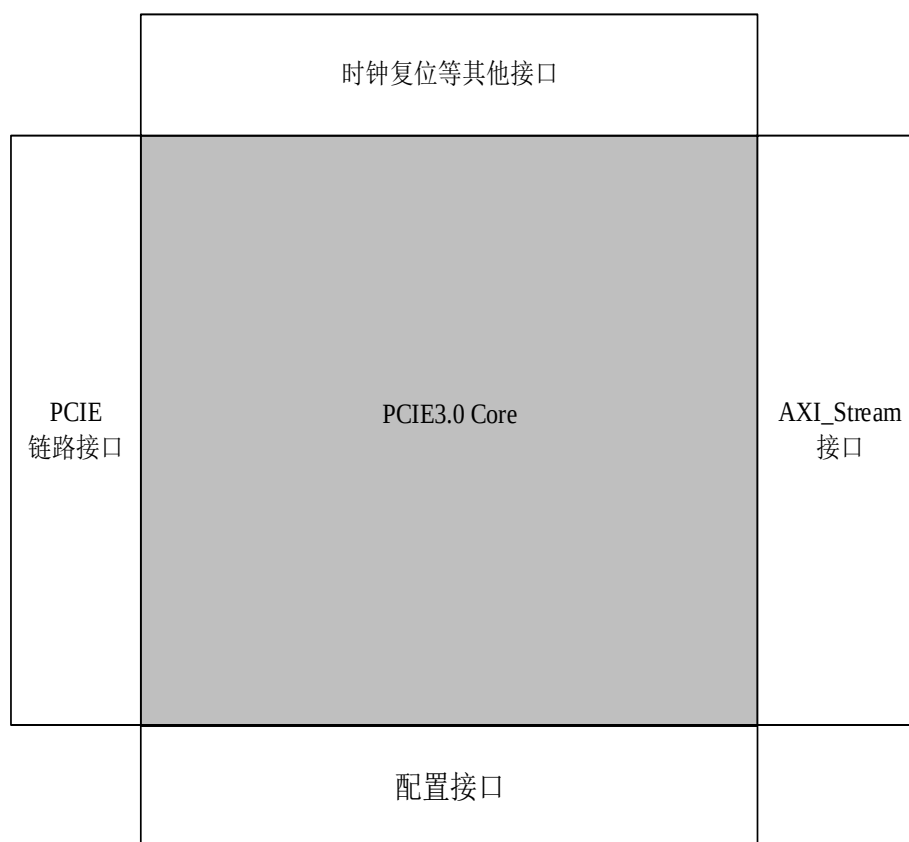


图 3-6 PCI Express 核的顶层接口

PCI Express 链路接口是连接 SERDES 串行高速总线的接口。PCI Express(PCI_EXP)接口包含多条通路，每个通路上有发送和接收两个方向，每个方向上由一对差分信号组成。即一个 PCI Express 通路由一对发送差分信号(pci_exp_txp, pci_exp_txn)和一接收差分信号(pci_exp_rxp, pci_exp_rxn)组成。若仅支持×1 通路模式，则通路 0 响应，PCI Express 链路接口 pci_exp_rxp[0], pci_exp_rxn[0]有效；若支持×2 通路模式，则通路 0-1 响应，PCI Express 链路接口 pci_exp_rxp[0: 1], pci_exp_rxn[0: 1]有效；若支持×4 通路模式，则通路 0-3 响应，PCI Express 链路接口 pci_exp_rxp[0: 3], pci_exp_rxn[0: 3]有效；若支持×8 通路模式，则通路 0-7 响应，PCI Express 链路接口 pci_exp_rxp[0: 7], pci_exp_rxn[0: 7]有效。

其他接口包含 PCI Express 的外部时钟和复位接口、流控制相关的接口以及一些特殊定制的功能接口，与 PCIE 核的复位、外部时钟输入、流控制等功能相关。

表 3-1 AXI_Stream 接口类的详细描述

接口分类	接口类型	说明
AXI_Stream 接口	完成者请求接口	以 m_axis_cq_*命名, 用户应用层可通过完成者请求 (CQ) 接口来转发链路上所有的接收请求。
	完成者完成接口	以 s_axis_cc_*命名, 用户应用层使用完成者完成(CC)接口来发送完成者请求。可以将所有的非转发请求作为分离的事务, 即在发送完成请求的同时, 在 CQ 接口可以继续接收新的请求。
	请求者请求接口	以 s_axis_rq_*命名, 用户应用层使用请求者请求(RQ)接口来产生传送至远程 PCIe 设备的请求。
	请求者完成接口	以 s_axis_rc_*命名, PCIe 核使用请求者完成接口来呈现有链路接收来的响应你的请求的完成。

3.2.3 功能配置模块

功能配置模块包含三部分, 即定义 PCIe 核功能的配置点存储模块, 动态重配置读写模块, 输入信号延时调整模块。

3.2.3.1 配置点储存模块

配置点存储电路主要是将软件对 PCIe 核的设置以配置点形式存储在 SRAM 中。SRAM 是 FPGA 中存放配置的模块, 六管 SRAM 的结构如图 3-7 所示。SRAM 模块由 16 根 WL 线控制, 分上下两个 ROW, 每个 ROW 中各 8 根, 每个 ROW 均有 4 根 WL 控制对 PCIe 核的功能进行配置, 4 根 WL 控制 PCIe 核的输入延时大小。

表 3-2 配置类接口详细说明

配置接口	配置管理接口	以 <code>cfg_mgm_*</code> 命名, 配置管理接口用来对配置空间寄存器进行读写。
	配置状态接口	以 <code>cfg_*_status</code> 命名, 配置状态接口提供已配置核的信息, 如经过协商的链路宽度和速度、核的电源状态和配置错误信息。
	配置接收消息接口	以 <code>cfg_msg_received_*</code> 命名, 用户应用层可从配置接收消息接口得到一些链路的可解码信息, 与数据相关的参数, 已接收的消息类型等信息。
	配置发送消息接口	以 <code>cfg_msg_transmit_*</code> 命名, 用户应用层可使用配置发送消息接口来发送消息, 用户应用层提供消息的类型和数据信息, 核会已一个 Done 信号作为回应。
	配置流控接口	以 <code>cfg_fc_*</code> 命名, 用来转发、处理各类流控制相关的信用量数据
	配置控制接口	配置控制接口给了用户应用层和核很宽的范围来进行信息交换。用户应用层使用这些接口来设置配置空间; 指出已发生的可校正和不可校正的错误; 设置设备串码 (Serial Number); 设置下游总线号, 设备号, 功能号; 接收每一个功能的配置信息等。
	配置中断控制接口	以 <code>cfg_interrupt_*</code> 命名, 配置中断控制接口允许用户应用层设置传统 PCIe 中断、MSI 中断或者 MSI-X 中断。核给出配置中断的发送和失败状态。

3.2.3.2 输入延时时序调整模块

输入延时时序调整电路主要完成对输入到 PCIE 核的端口信号进行有选择的延时, 由信号延时电路和选择器组成。延时电路完成信号延时, 然后送入选择器中, 由配置点决定是选择直接输出原信号还是延时后的信号。其电路结构示意图如图 3-8 所示。

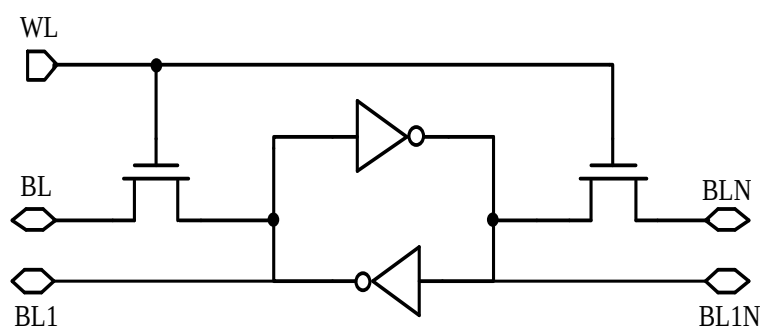


图 3-7 配置 SRAM 单元

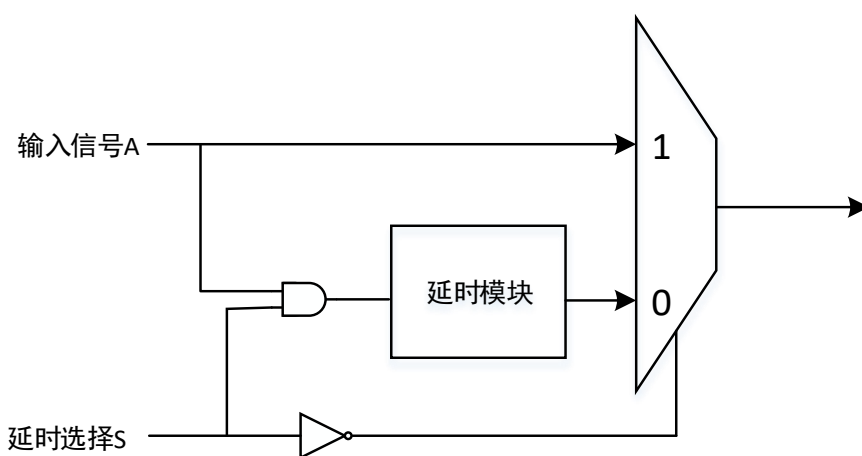


图 3-8 输入延时模块结构图

3.2.3.3 动态重配置读写模块

动态重配置读写是 PCIE 核根据需要，在 bit 流下载完成后，对 PCIE 核功能进行更改的一种途径。用户逻辑将重配置数据、重配置地址等传入由 PCIE 核，核进行地址译码后传入动态重配置读写模块，来改写存在配置点存储模块相应的值，同样读的过程是用户逻辑将动态重配置读地址通过 PCIE 核发送至动态重配置读写模块，动态重配置读写模块将对应地址里的配置点值通过读端口传至 PCIE 核，再传送至用户逻辑。

3.2.4 输入输出互联模块

PCI Express 模块和外部设备的交流可以调用 FPGA 中的输入输出端口。FPGA 的输入输出端口属于全局可以配置资源，用户在调用资源的时候在物理约束的时候说明需要调用的输入输出端口的物理位置，电平约束的要求，系统可以通过软件自动布局布线，完成输入输出模块和功能模块的连接工作。输入输出模块主要

完成各模块数据交互功能。

3.2.5 FPGA 逻辑

FPGA 逻辑包含与 PCIE 核临近的 CLB 及配置 SRAM 资源。可编程逻辑单元是实现用户自定义时序逻辑、组合逻辑功能的主要模块。可编程逻辑单元可以通过互联矩阵实现互联，也可以通过互联阵列和产品内其他资源连接。可编程逻辑单元的复杂度和数量是产品可编程能力的重要参数。配置 SRAM 资源和 FPGA 产品的架构关系密切，架构直接决定了配置电路实现的系统组合。其基本功能就是对 SRAM 阵列写入不同的数据组合控制内部的逻辑电路来满足用户设计的功能，而 FPGA 能够实现何种功能是由其配置比特流文件来决定的。

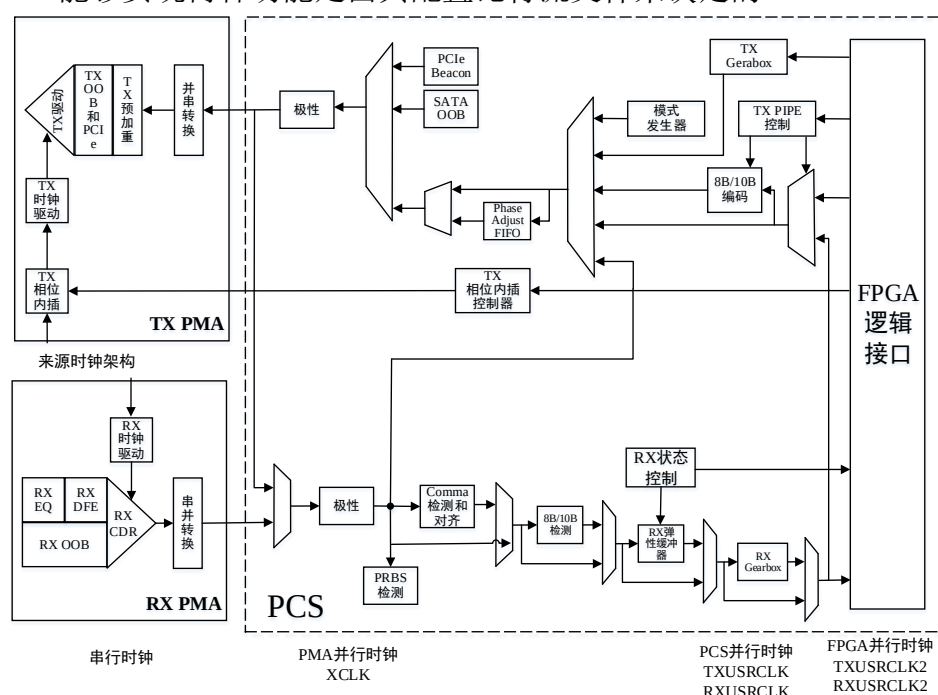


图 3-9 Serdes 收发器系统功能框图

3.2.6 CMT 模块

FPGA 芯片的各个功能模块的时钟激励可以由外部输入输出给进，也可以由内部的 CMT 单元给出。时钟管理模块是 FPGA 内部负责芯片的供给的核心模块。CMT 模块可以根据芯片需求提供给 FPGA 芯片的全局时钟资源与区域时钟资源。PCIE 核的时钟信号可以由 CMT 模块在芯片内部产生，也可以使用核上的时钟输入输出端口进行外部时钟的输入，这是用户可选的功能。

3.2.7 SERDES 模块

每个高速串行接口模块（SERDES）含发送端和接收端，分为物理编码子层（PCS）和物理媒介接入层（PMA）两部分，其具体结构如图 3-9 所示；其中 PCS 主要实现对数据的编码、解码及各种协议控制等功能，PMA 主要实现数据的并串转换和串并转换。本文所使用的 SERDES 可以实现从 500Mb/s 到 8.5 Gb/s 的链路线速率。

3.3 本章小结

本章首先介绍了典型 FPGA 芯片的架构，并且根据实际情况，确定了本次项目设计的 FPGA 芯片的架构体系。根据项目总体的资源配置情况，制定了本次子项目千万门级 FPGA 芯片中 PCI Express Core 的集成方案；然后对 PCI Express Core 集成相关的资源进行了详细的介绍。本章是本次研究工作的重要组成部分，是后文模块功能验证和应用开发的基础。

第四章 PCI Express 核的验证方法研究

芯片验证是在芯片前端研发设计过程中对待测设备 (Device Under Test, DUT) 与设计方案要求中的行为一致性、处理错误的处理方式合理性、不同模式下功能的全面性进行检验证明的一种通用方法^[46]。芯片验证环节是芯片设计完成后交付厂商流片前的重要环节,是全芯片设计周期中最为耗时费力的一环。同时,在 FPGA 芯片研发的全周期流程中,验证环节的数据资料和芯片生产后的应用测试环节有一定的通用性。为了缩短项目研发周期,如何在芯片验证阶段的快速高效的搭建出一个满足项目要求、适合芯片设计生产测试全周期使用、能够充分验证 DUT 功能且重用性高的验证平台成为了验证环节中最为关键的工作。本章将介绍一种基于 PCI Express3.0 协议,可通用于芯片验证阶段的行为仿真功能校验的仿真平台,验证平台为芯片测试阶段测试工程搭建提供有效支撑。

4.1 通用验证平台的架构

4.1.1 验证仿真电路环境

根据 PCI Express 协议拓扑架构规定,PCI Express 的工作电路环境是一个复杂的系统,一个最简单的链路环境需要两个 PCI Express 核,一端作为根联合体 (Root Complex),另一端作为终端设备端 (Endpoint)。

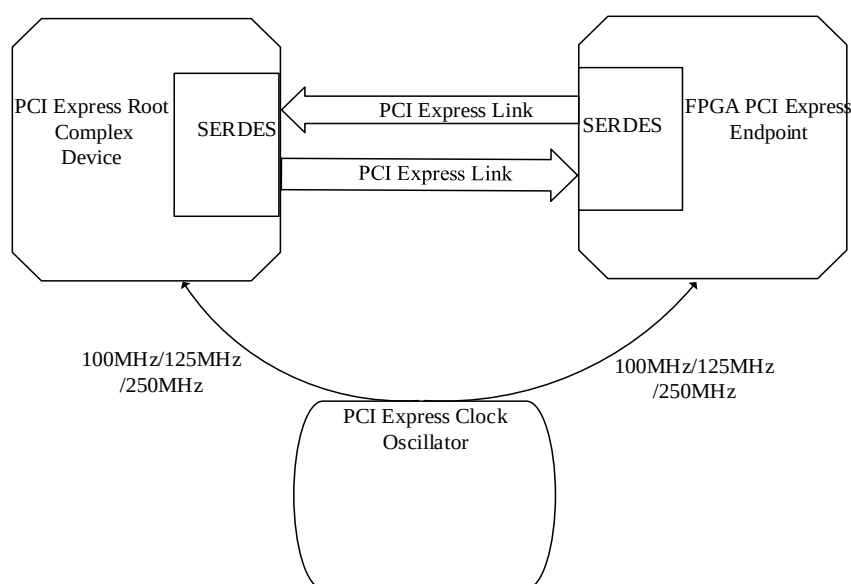


图 4-1 验证仿真电路环境

本次验证所设计的仿真环境就是按照这样的结构组成,如图 4-1 所示,左上是电路环境中的根复合体端,内含一个完整的 PCI Express IP 核以及相应的配置资源,其物理层接口高速 Serdes 模块通过与 PCI Express 链路链接,从而实现与对端设备的链接通信;右上是电路环境中的终端设备端,和根复合体一样,终端设备中需要匹配完整的 PCI Express IP 核和高速 Serdes 模块,同时根据验证功能需要配置相应的配置终端,如可编程输入输出(Programmed Input Output, PIO)设备,传统 PCI 设备等。本次仿真使用全数字仿真环境,仿真软件使用 VCS 软件,波形文件查看使用 Verdi 软件。

4.1.2 验证环境的结构

根据上文对验证电路架构的分析,我们需要进一步来梳理对于整个系统验证平台(TestBench)的搭建。图 4-2 是本次项目中使用的验证平台的基本结构。PCIE_Agent 模块是整个验证平台的核心,其主要功能是发送完整的覆盖整个项目要求和协议规定的功能的激励 TLP 包,同时根据下发至仿真环境激励收集相应返回的 TLP 包,实现整个仿真的信号闭环处理。

PCIE_Agent 由 PCIE_Driver、PCIE_Monitor、PCIE_CFG 三个子模块组成,接下来分别介绍各部分的功能。

4.1.2.1 PCIE_Driver 模块

在这个子模块中,主要封装了仿真过程中需要使用的所有的激励 TLP 包,并且将每一个仿真行为封装成 TASK 块。在后期进行仿真时,PCIE_Driver 根据仿真指令信息,调用相应的 TASK,组成一个完整了 PIC Express 事务集,从而完成一次功能完整的激励仿真。所以,在搭建验证平台的时候,对于 PCIE_Driver 模块的搭建是整个项目工作中的重点。对这一模块的工作方式,笔者进行了一些经验性的整理:首先,需要深入了解、熟悉此次项目需要验证的功能点;然后根据 PCI Express3.0 协议,将需要在仿真过程中完成了发包的类型进行整理,撰写相关的 TASK;分析不同用例使用的 TASK,对有相同或类似功能包的 TASK 进行删减、重命名和重组,尽量减少 PCIE_Driver 库中 TASK 的数量;最后,在项目进行过程中,根据工程的要求和项目进度,及时更改、更新 PCIE_Driver 中 TASK 库。

4.1.2.2 PCIE_Monitor 模块

在仿真过程中,一次完整的仿真激励可能会包含激励的发送和接收(比如 PCI Express 各类读请求事务,就会返回一个完成事务包);所以,在验证平台中需要有

一个组件来接收这部分反馈回来的数据事务包，PCIE_Monitor 子模块即完成这一部分的功能。

4.1.2.3 PCIE_CFG 模块

PCIE_CFG 存放的是在某一种工作模式下，系统对 PCI Express Core 的配置信息，由于 PCI Express 支持多种工作模式（例如 Gen1X1、Gen2X8，不同工作模式对应不同的工作传输通道数和单通道的传输速率），更改此模块的数据可以达到更改系统工作模式的目的。在本项目，这一模块由 FPGA 的配套软件直接生成，有效提升了工作效率。

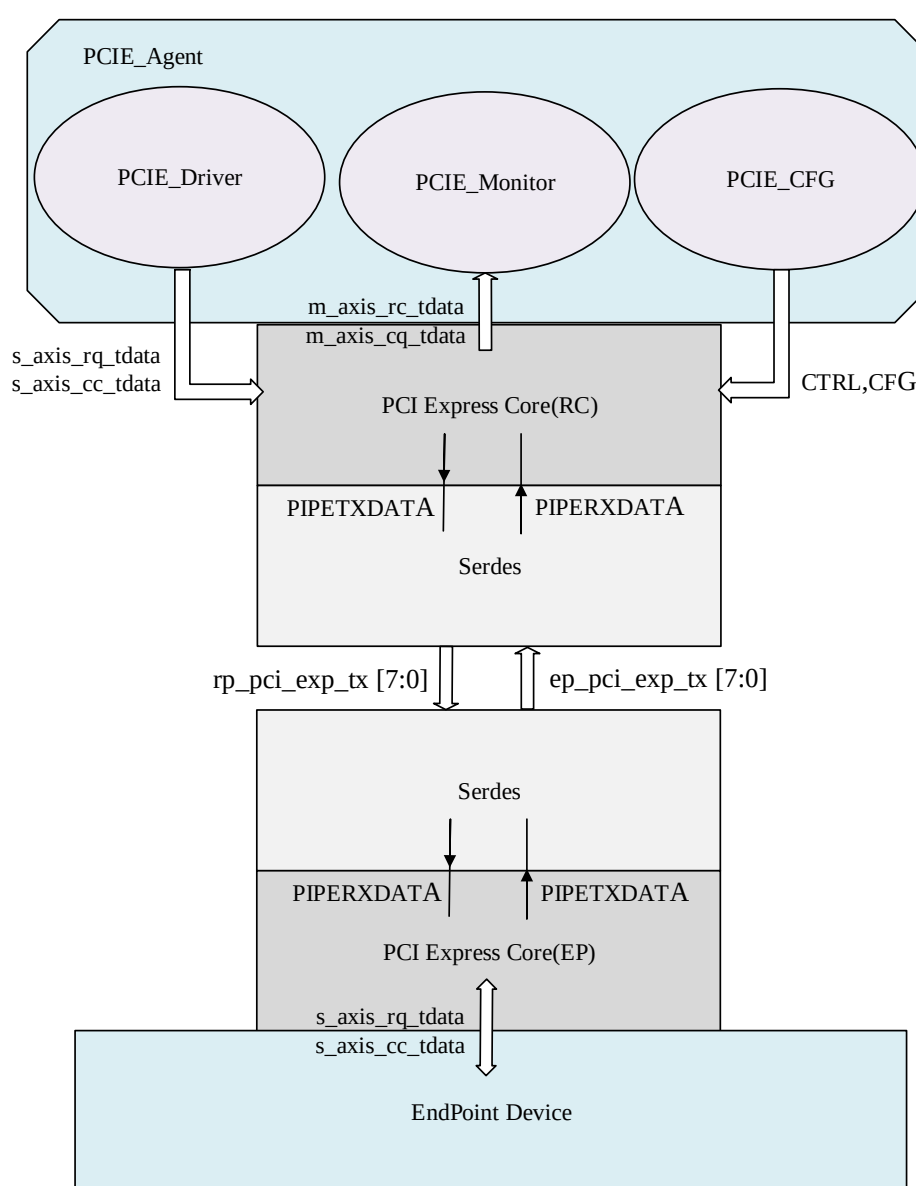


图 4-2 验证平台基本结构

4.2 验证激励向量介绍

本次仿真为了实现 PCIE3.0 协议的覆盖,构建了大量的向量。具体向量的构建包括三个基础向量 (Base_Specification) sample_smoke_test0、sample_smoke_test1 和 pio_writeReadBack_test0, 还有单独构建的特殊向量如读取寄存器属性值的管理寄存器向量, 以及在基础向量上构建的测试向量, 具体仿真向量的详细规划如下。

Base_Specification: 基础测试向量组, 包括了 sample_smoke_test0、sample_smoke_test1 和 pio_writeReadBack_test0 三个子向量。sample_smoke_test0 向量的操作为执行系统初始化, 等到系统初始化完成后由上游设备发送一帧配置读的事务包, 系统将会反馈一个带设备的器件/供应厂商 ID 号完成事务包, 这个 ID 号是由用户定义的, 通过比对系统回读和用户定义的 ID 是否一致, 判断这一操作在系统内是否正确执行; sample_smoke_test1 向量的操作为执行系统初始化, 然后同时发送一个配置读 TLP 事务包, 一个带数据的完成 TLP 事务包, 两个任务并行进行; pio_writeReadBack_test0 为程控输入输出 PIO 设备的写任务以及回读, 具体包括了: 系统初始化, 基地址空间 (Base Address Register, BAR) 初始化, IO 读写, 32 bit、64 bit 存储器 (memory) 读写。以上三个向量连续仿真可以覆盖的功能有: 通用数据包头字段规则, 有数据负载的 TLP 规则, TLP 摘要规则, 路由和寻址规则, 处理层描述符, 存储、配置请求规则, 完成规则, 请求处理规则, 完成处理规则, 事务顺序规则, 遵循读事务的更新顺序和粒度, 写事务提供的更新顺序和粒度, 虚拟通道证明, TC 到 VC 映射, VC 和 TC 规则, 流控制规则, 数据链接层控制和管理状态机制规则, 流控制初始化协议, 流控制初始化状态机制规则, DLLP, 数据链接层包的规则, 数据完整性, LCRC, 序列号, 和重复管理 (TLP 发送器), 2.5GT/s 和 5.0GT/s 数据传输率编码, 8.0GT/s 和更高数据传输率编码, 链接初始化和训练, 链接训练和 LTSSM 类型, 链接训练和状态情况规则, 时钟容差补偿, 设备同步, 同步支持等。

pio_test: pio_test 向量包括了 pio_testByteEnables_test0、pio_memTestDataBus、pio_memTestAddrBus、pio_memTestDevice、pio_readConfigReg_test0、pio_tlp_test0、pio_timeoutFailureExpected、pio_barRange_Test0。该向量也是一连串的程控输入输出测试向量, 需要单独仿真。pio_test 验证了关于 PCIE 以及 PIO 设计的一些功能, 该向量可以覆盖的功能有第一/最后 DW 字节启用规则, 错误发送, 完成超时机制等。

Advanced_Error_Reporting: 高级错误报告向量主要是为了验证 PCIE 的错误处理, 错误分类等功能, 是在基础向量的基础上植入不同类型的错误, 配置一些错误寄存器和相关使能端口进行仿真的。该向量可以覆盖的功能有错误信号和记录,

处理, 研究范围, 错误分类, 错误信号, 错误记录, 设备错误序列发送和记录应用, 错误消息控制, 内部错误。主要验证了有关错误报告这方面的功能。

Alternative Routing-ID Interpretation: 可选 ID 路由实现功能是通过更改请求者 ID 向量是在基础向量上配置有关 ARI 的功能寄存器以及修改输入任务激励实现仿真的, 此向量覆盖了设备核的 ARI 功能。

INTx_Interrupt: 传统中断的仿真使用的是传统端点的模式, PCIE 端点不支此功能, 仿真激励是在环境中修改了相关控制文件的激励以及配置相关寄存器任务能够覆盖的功能传统中断。

MSI_Interrupt: 在原有的中断向量环境上构建 MSI 中断向量的控制指令和 MSI 中断任务, 覆盖 MSI 中断功能。

IO 事务: IO 向量的仿真是在传统端点模式下进行的仿真, 需要构建传统设备的仿真环境。PCIE 端点不支持 IO 事务, IO 仿真调用了 pio_writeReadBack_test0, 其中 IO 的读写向量覆盖了 IO 事务的功能。

laneX1_8G: 该向量是在基础向量上修改通道属性值进行的向量仿真, 覆盖了单通道的传输功能。

laneX2_8G: 该向量是在基础向量上修改通道属性值进行的向量仿真, 覆盖 2 通道的传输功能。

laneX4_8G: 向量是在基础向量上修改通道属性值进行的向量仿真, 覆盖 4 通道的传输功能。

laneX8_2.5G: 向量是在基础向量上修改传输速率属性值进行的向量仿真, 覆盖了 2.5GT/S 的数据传输功能。

laneX8_5G: 向量是在基础向量上修改传输速率属性值进行的向量仿真, 覆盖了 5GT/S 的数据传输功能。

laneX8_8G: 向量是在基础向量上修改传输速率属性值进行的向量仿真, 覆盖了 8GT/S 的数据传输功能。

Polarity: 该向量在原有的环境上修改顶层文件构建 PCIE 通道错误连接的向量, 覆盖 PCIE 的极性倒置功能。

PowerManagement: 此事务在基础向量上通过软件层对 PCIE 的电源管理寄存器配置, 增加新的向量事务, 覆盖了链接状态功耗管理、PCI-PM 软件协调机制、PM 软件控制链接功耗管理状态、功耗管理事件机制、辅助功耗支持、辅助功耗使能、功耗管理系统消息等功能。

Three_Types_Of_Reset: 在 Base_Specification 向量的基础上配置相关寄存器, 以及断言一些复位端口, 该向量覆盖了传统复位、热复位、功能级复位功能。

Optimized Buffer Flush/Fill 消息：OBFF 向量主要是在 PIO 的基础上配置好 OBFF 的使能端口，调用消息事务发送 OBFF 的消息，在 EP 接收端的消息接收，消息接收数据和消息接收类型端口上均收到了来自 RP 发送的 OBFF 消息数据。

Latency Tolerance Reporting 消息：此向量是在原环境上构建新的向量任务，配置关于 LTR 寄存器的值，调用 LTR 消息事务，覆盖的功能有消息请求规则、延时偏差报告机制功能。

基地址寄存器：基地址寄存器向量是在 pio_writeReadBack_test0 基础向量上配置可调基址控制寄存器仿真 PIO 的存储读写事务，向量覆盖了可调基址寄存器的功能。

4.3 验证结果分析

本文的验证工作完成了 PCI Express 3.0 协议规范规定的大部分功能，由于本次仿真采用的是直接仿真的模式，仿真的起点和终点都是 PCI Express 设备的事务层，仿真专注于 PCIE 接口的功能验证，不对挂载 PCIE 接口设备的复杂功能进行验证；本次验证的结果结合作者对协议的理解，尽量以仿真的实际波形为验证结果进行实际说明。

4.3.1 PIC Express 系统的初始化和链路训练的验证

PCI Express 系统在启动后，并不会马上进行信号的交互。根据协议规范说明，系统需要在启动初期完成系统初始化和链路训练，进行主从设备的初始化配置与识别。系统的非正常初始化、链路训练的失败都会导致链路无法连接，从而导致数据无法传输。系统初始化和链路训练是本文验证工作的基础。

我们使用 Base_Specification 模块的验证激励进行冒烟验证，仿真向量使用 pio_writeReadBack_test0 向量。系统复位和初始化的流程如下图 4-3。

电路的顶层产生一个复位信号 sys_reset 对系统进行复位，复位时间为 500 个时钟周期；一方面 PCI Express Core 会根据复位机制对系统内的资源进行初始化配置，另一方面复位信号会直接传给 PCI Express 的物理层对电气物理层进行复位同时启动链路初始化和链路训练。当链初始化和链路训练完成，user_lnk_up 信号会拉高，系统系统间可以开始发送数据包。

根据仿真结果，我们抓取了链路仿真开始的一段时间内复位信号和链路状态机信号的波形，如下图 4-4 所示：

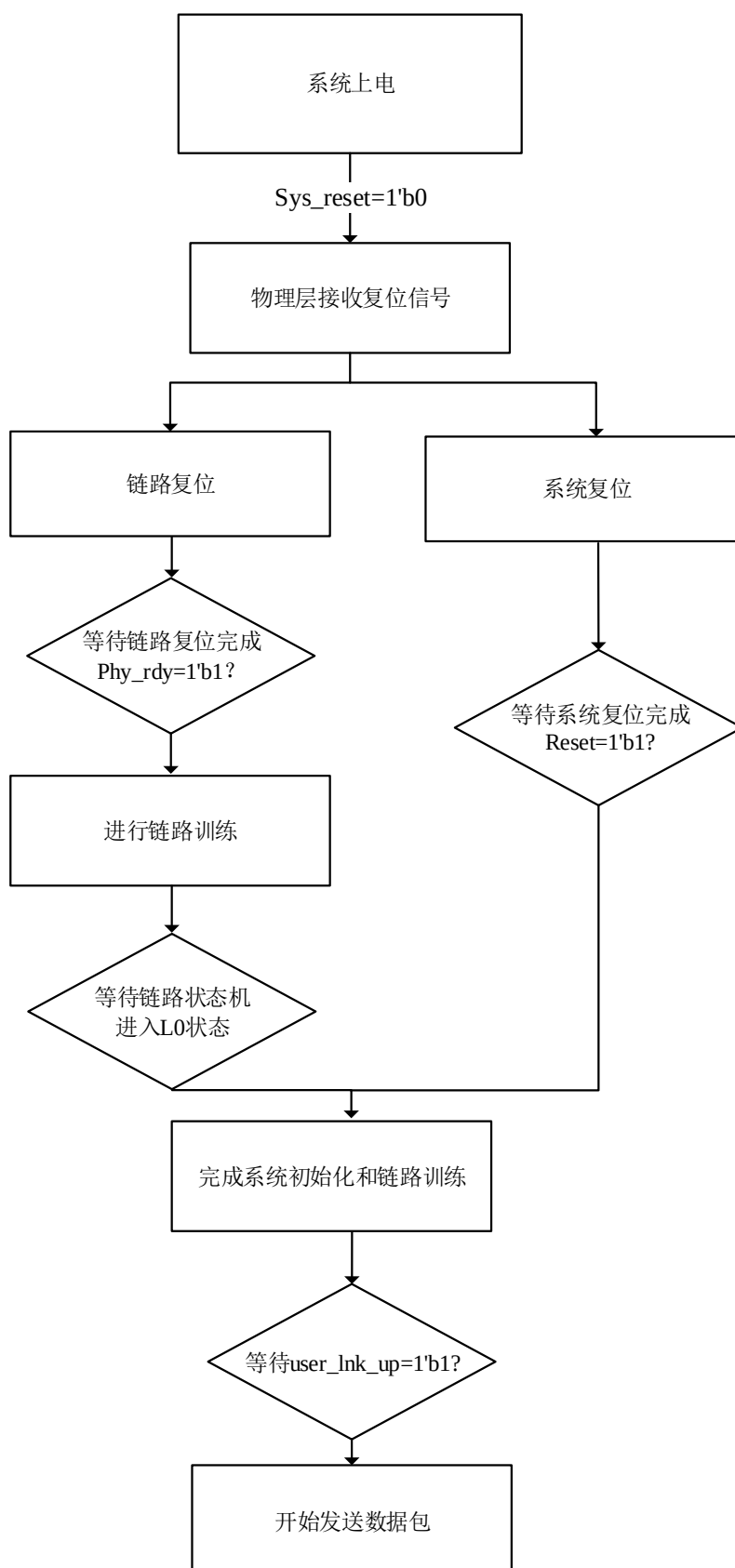


图 4-3 PCI Express 初始化流程图

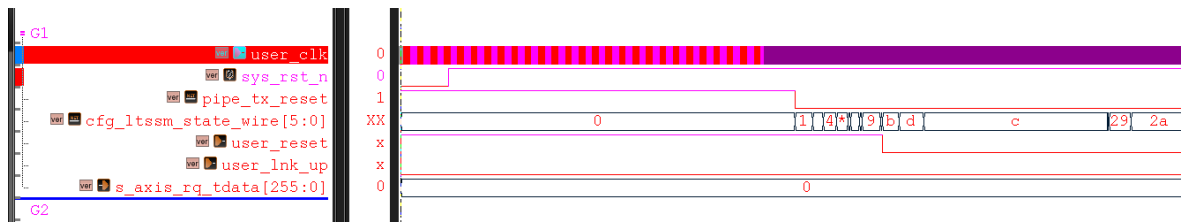


图 4-4 仿真复位信号图

一段时间后，Pipe_tx_reset 信号置 0，phy_rdy 信号置 0，随即 Cfg_ltssm_state 信号开始跳变，开始进入链路训练状态。此时表示物理层复位完成，开始进行链路训练（对于链路训练的验证将在后续进行说明），此时 Reset 信号值保持 1'b1，说明此时 PCI Express Core 的复位还未完成；物理层还未有数据发送。这一阶段表明物理层复位完成，同时进行链路训练，DUT 的物理层复位成功功能验证正确。

当 Cfg_ltssm_state 信号跳转至 10 状态，表明链路进入 L0 状态，此时，Reset 信号已经置 0，进行了复位释放，表明 PCI Express Core 的复位完成，逻辑物理层开始发送相关有序集通知全局系统链路训练完成，等待 user_lnk_up 信号拉高，逻辑物理层开始发送数据包。由此可得，系统的初始化功能正常，能够正确建立链路上设备间的识别和连接。

在系统初始化中，最重要的是物理层链路训练机制。链路训练需要完成设备的识别、传输模式的识别和建立、链路时钟补偿等功能，具体实现是通过链路训练状态机来实现。表 4-1 是 PCI Express Core 中 Cfg_ltssm_state 信号值与状态机状态的映射关系。

我们抓取了链路训练状态机 Cfg_ltssm_state 信号的波形图，如下图 4-5 所示。

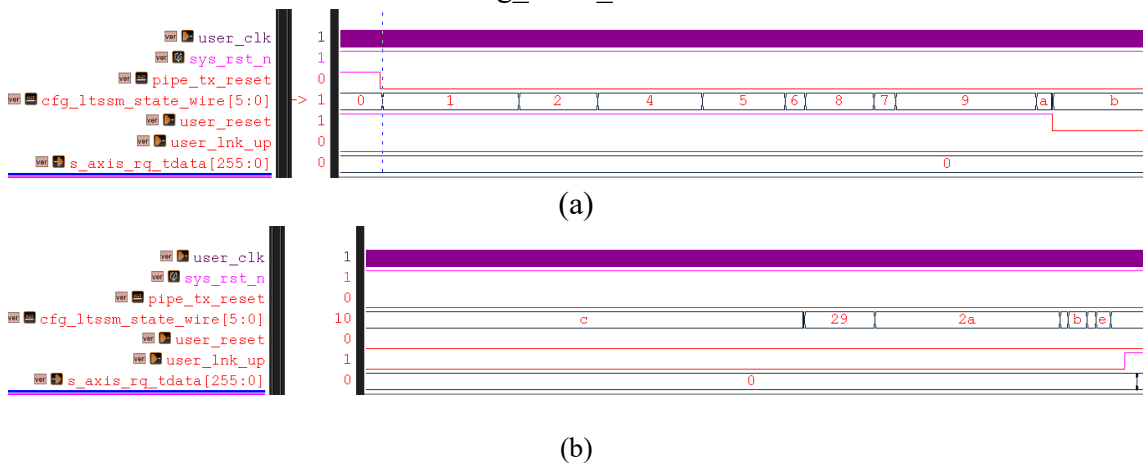


图 4-5 链路训练状态机 Cfg_ltssm_state 信号波形图。(a) 链路训练状态机

Cfg_ltssm_state 信号波形图；(b) 链路训练状态机 Cfg_ltssm_state 信号波形图续

表 4-1 链路训练状态映射关系表

端口名	数值	状态
CFG_LTSSM_STATE	00	Detect.quiet
	00	Detect.Active
	02	Polling.Active
	03	Polling.Compliance
	04	Polling.Configuration
	05	Configuration.Linkwidth.Start
	06	Configuration.Linkwidth.Accept
	07	Configuration.Lanenum.Accept
	08	Configuration.Lanenum.Wait
	09	Configuration.Complete
	0A	Configuration.Idle
	0B	Recovery.RcvrLock
	0C	Recovery.Speed
	18	L1.Idle
	19	L2.Idle
	1A	L2.TransmitWake
	20	DISABLED
	21	LOOPBACK_ENTRY_MASTER
	22	LOOPBACK_ACTIVE_MASTER
	23	LOOPBACK_EXIT_MASTER
	24	LOOPBACK_ENTRY_SLAVE
	25	LOOPBACK_ACTIVE_SLAVE
	26	LOOPBACK_EXIT_SLAVE
	27	HOT_RESET
	28	RECOVERY_EQUALIZATION_PHASE0
	29	RECOVERY_EQUALIZATION_PHASE1
	2A	RECOVERY_EQUALIZATION_PHASE2
	2B	RECOVERY_EQUALIZATION_PHASE3

链路训练状态机依次进入了 Detect.quiet、Detect.Active、Polling.Active、Polling.Configuration、Configuration.Linkwidth.Start、Configuration.Linkwidth.Accept、Configuration.Lanenum.Wait、Configuration.Lanenum.Accept、Configuration.Complete、Configuration.Idle、Recovery.RcvrLock、Recovery.RcvrCfg、Recovery.Speed、RECOVERY_EQUALIZATION_PHASE1、RECOVERY_EQUALIZATION_PHASE2、RECOVERY_EQUALIZATION_PHASE3、Recovery.RcvrLock、Recovery.RcvrCfg、Recovery.Idle、L0 状态，符合验证功能点要求，验证通过。

4.3.2 基本事务的验证

4.3.2.1 Memory 事务包

Memory 事务是对 PCI Express 的存储单位进行读写操作，Memory 事务分为读写两种，采用地址路由的方式寻址访问，存储器读事务不带数据、存储器写事务带数据。Memory 事务的验证采用了验证激励 Base_Specification，其中 pio_writeReadBack_test0 向量的工作内容包含了存储器事务的验证。验证 Case 的数据模型是由验证平台将请求通过验证平台发送至上端设备，并在 PCI Express Core 事务层中产生相应的事务包，按照发包协议进行发包。仿真结束后，我们抓取物理层端口的数据包，即可验证相应事务协议是否准确执行。

如图 4-6 所示，CC_RP 是上游设备根复合体设备（Root Complex Point，RP）的事务包发送端口。我们抓取了其中一帧数据包的内容：仿真波形是在 X8 通道 5GT/s 的速率下仿真的（5GT/s 速率采用的 8b/10b 编码模式容易区分数据包的类型以及数据结构）。截取数据通路上 RP 发送通路的物理层上某时刻的数据包通路，可以分析得出 fb 为事务包的开始帧符号，fdh 为数据包的结束帧符号，001ch 为序列号，6000_2001_0006_000f_0000_0001_0020h 为通用数据包头字段的格式，可以从头字段中知道该数据包是 64 比特的带数据 6162_6364h 的 memory_write 写事务包，使用的是地址路由，地址是 20，其中的 e95a_b961h 是 32 比特的 LCRC 字段。

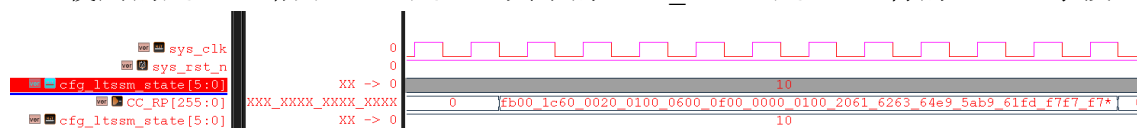


图 4-6 Memory 事务包波形图

4.3.2.2 配置事务包

配置事务是 PCI Express 设备访问设备配置空间的操作手段，具体包括配置读事务和配置写事务。配置读事务不带数据、配置写事务带数据。配置数据的路由

方式是 ID 路由。存储器事务的验证使用的激励是 Base_Specification，其中 sample_smoke_test0 向量的工作内容包含了 PCI Type 0 配置读 TLP。配置读请求访问的重点是设备的配置空间，可以是 Root Complex，亦可以是 EndPoint。本次激励采用了通过 Root Complex 发送配置读事务包访问 EndPoint 的配置空间。

图 4-7 在终端 PCI Express Core 端口截取的一帧 Type 0 配置读事务包。fb 为包的开始帧符号，fd 为包的结束帧符号；序列号是 0000h，fb29_81a0h 为 LCRC 字段；头包上度为 3DW 的数据：0400_0001_0000_000f_01a0_00d0h，其中地址为 d0h。所以这个包是访问 d0 编号的配置寄存器的一个配置读事务包。

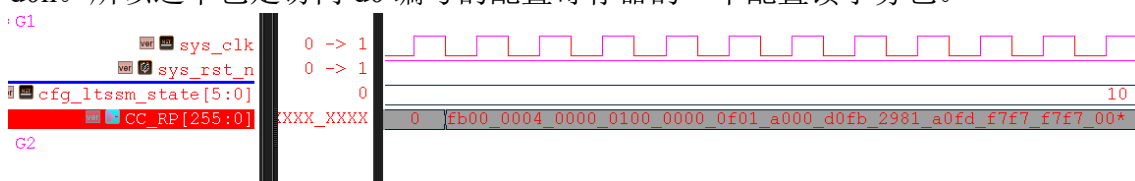


图 4-7 配置事务包波形图

4.3.2.3 IO 事务包

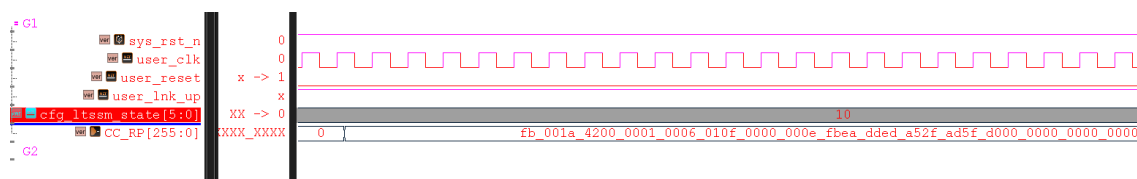
IO 事务是 PCI Express 向下兼容 PCI 协议的一类事务。对于 IO 的验证，需要搭建基于传统 PCI 端点的终端设备验证电路结构，基于 PCI Express 的协议的端点是被设备支持 IO 事务。IO 事务的验证调用的是 pio_writeReadBack_test0 激励，通过发送 IO 读写包覆盖 DUT 的 IO 事务功能。

图 4-8 别截取了 Root Complex 的发送端口和 EndPoint 的接收端口的事务包。根据图 4-6(b)，可以看到 RP 发送端的物理层通路上可以看到发送了一个 32 bit 的 IO 读事务，读地址是 00h，然后图 4-6(d)中是 EndPoint 收到了这一帧读 IO 包；图 4-6(a)Root Point 端发送了一帧 32 bit 的写事务，写地址是 00h，写数据是 efbe_addeh，图 4-6(c)显示下游设备（End Point, EP）端成功收到了 IO 写的请求，并在 IO 设备中写入数据 efbe_addeh。

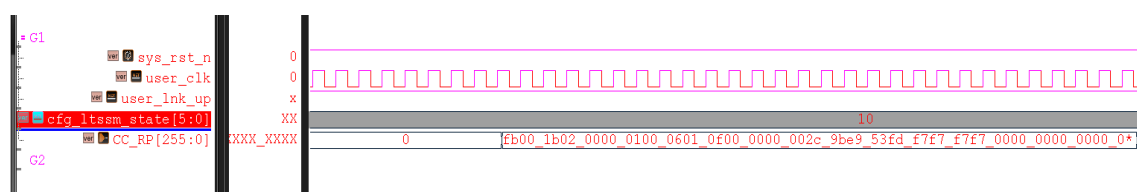
4.3.2.4 错误的事务包的验证

ECRC (Error Cyclical Redundancy Check) 规则是 PCI Express 事务层的重要部分。其功能事务层在产生 TLP 包时，为了防止后面的设备接收这一帧事务包时发生错误，可以选择性的在事务包的末端 1DW 加入 ECRC 校验码，而后的设备根据编解码规则对 TLP 的数据进行校验，从而判断收到了 TLP 的数据是否发生错误。本次验证的激励采用 Advanced_Error_Reporting 激励，主要验证在错误发生时是否产生了 ECRC 校验码。Advanced_Error_Report 向量仿真中使用的仿真向量为 sample_smoke_test0、sample_smoke_test0、pio_writeReadBack_test0 联合仿真的向

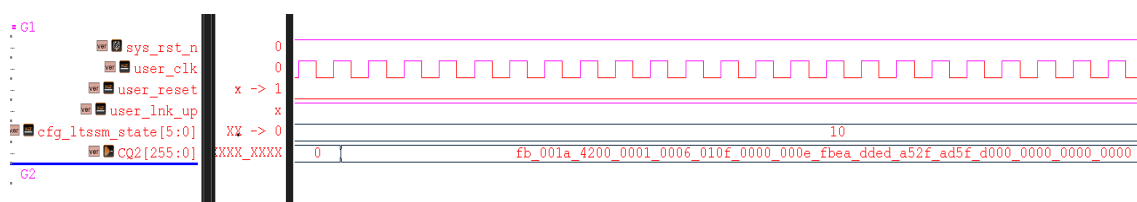
量，通过在环境中植入不同类别的错误类型，来完成验证激励的创建。



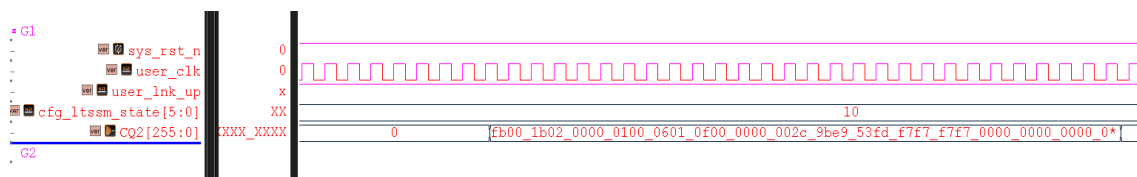
(a)



(b)



(c)



(d)

图 4-8 IO 数据包格式。(a)RP 端写 IO 数据包；(b)RP 端读 IO 数据包；(c)EP 端写 IO 数据包；(d)EP 端读 IO 数据包

如图 4-9 示，我们截取了 RP 的一帧事务包数据。这是一个 3DW 的存储器写事务，根据事务包的规则，e574_166h 为 ECRC 字段。TLP 包中产生了相应的 ECRC 字段，符合预期行为规范。

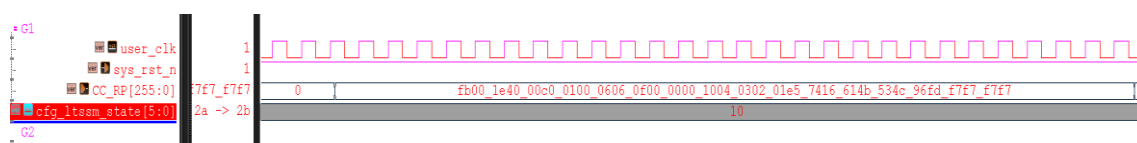


图 4-9 带 ECRC 字段的存储器事务包的结构

当设备提供了一个不受支持的请求事务时，判断终端设备是否能正确识别错误、PCI Express 是否处理错误，是本项功能点的验证的目标。根据验证计划，本次验证采用 pio_test 的 pio_timeoutFailureExpected 向量，通过 pio 的 32 bit 的存储写任务，修改存储写任务的地址为 32'h8000_0000，这是一个非法的存储空间地址。当仿真启动，仿真环境完成了系统初始和储存空间的初始化，我们尝试往地址为 32'h8000_0000 的存储器地址中写入全 0；然后我们发送一帧 32 位的读请求包，读地址为刚刚写入的地址位 32'h8000_0000。然后观察仿真结果。

根据图 4-10，我们发送一个 32 位的写事务请求包，写地址为 8000h，写数据是 0102_0304h；由图 4-11，可以看到我们发送了一个 32 位的错误读请求包，读地址为 8000h。

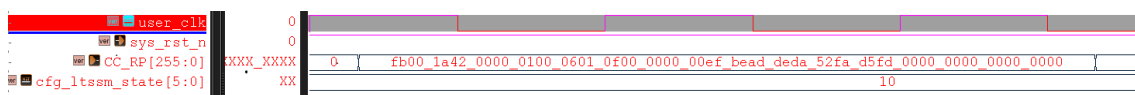


图 4-10 错误写请求事务包的发起

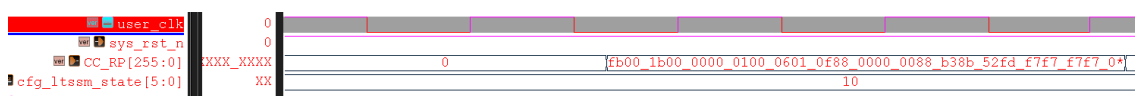


图 4-11 错误读请求事务包的发起

我们打印了系统等到的消息报告，如下图 4-12 所示。可以看到系统未读到相应的数据，且仿真结束，说明系统检测出了 8000h 地址错误，从而中断了仿真。用例的验证通过。

```

78 [ 108747413] : TSK_PARSE_FRAME on Transmit
79 [ 108875258] : TSK_PARSE_FRAME on Receive
80 [ 109155397] : TSK_PARSE_FRAME on Transmit
81 [ 109287196] : TSK_PARSE_FRAME on Receive
82 [ 109563287] : TSK_PARSE_FRAME on Transmit
83 [ 109695225] : TSK_PARSE_FRAME on Receive
84 [ 109971261] : TSK_PARSE_FRAME on Transmit
85 [ 110103306] : TSK_PARSE_FRAME on Receive
86 [ 110379199] : TSK_PARSE_FRAME on Transmit
87 [ 110511349] : TSK_PARSE_FRAME on Receive
88 [ 110787225] : TSK_PARSE_FRAME on Transmit
89 [ 110919409] : TSK_PARSE_FRAME on Receive
90 [ 111195320] : TSK_PARSE_FRAME on Transmit
91 [ 111339308] : TSK_PARSE_FRAME on Receive
92 [ 111731286] : TSK_PARSE_FRAME on Receive
93 [ 115195355] : NOTHING: to Memory 64 Space BAR 0
94 [ 115195355] : Transmitting TLPs to Memory 32 Space BAR 2
95 [ 115203343] : TSK_PARSE_FRAME on Transmit
96 [ 115251409] : TSK_PARSE_FRAME on Transmit
97 [ 115783199] : TSK_PARSE_FRAME on Receive
98 [ 135243225] : TIMEOUT WARNING in usrappp tx:TSK_WAIT_FOR_READ_DATA. Completion data never received.
99 [ 135243225] : TEST PASSED --- Test correctly timed out. CPLD not received
    
```

图 4-12 错误请求下的验证报告

4.3.2.5 首末位字节使能字段

由前文的介绍，在 PCI Express 的包中，最小的数据单元是 DW，即 4Byte；但是在数据传输过程中，传送的数据的单位都是由 Byte 为单位。为了进行精确的数据传输，PCI Express 采用首末字节使能的方式使 PCI Express 的数据分辨率达到字节位。通过对 PCI Express 事务包包头的第 2DW 的高 8 位使能，映射 PCI Express 的数据的首末 8 个字符。

首末位字节使能字段的验证通过 pio_test 中的 pio_testByteEnables_test0 向量进行。具体行为是在发送事务包时打开字节使能位，然后回读相应的数据。检查使能位的数据是否回读正确。

如图 4-13，环境连续发送了四个 32 位的存储器写请求并读出每一次写入的值，依次使能第 1 到 8 位字节，预期的结果应该是四次写入的数据的依次排列值 0123_4567h，可以看到仿真打印的结果和预期一致。

```

98 [ 111727343000] : Transmission to Mem 32 Space BAR 0
99 [ 111727343000] : MEMWRITE32, Address = 00000000, Write Data 01ffffff
100 [ 111735405000] : TSK_PARSE_FRAME on Transmit
101 [ 111775392000] : MEMWRITE32, Address = 00000000, Write Data ff23ffff
102 [ 111783392000] : TSK_PARSE_FRAME on Transmit
103 [ 111823366000] : MEMWRITE32, Address = 00000000, Write Data ffff45ff
104 [ 111831302000] : TSK_PARSE_FRAME on Transmit
105 [ 111871250000] : MEMWRITE32, Address = 00000000, Write Data ffffffff67
106 [ 111879257000] : TSK_PARSE_FRAME on Transmit
107 [ 111927184000] : TSK_PARSE_FRAME on Transmit
108 [ 112403315000] : TSK_PARSE_FRAME on Receive
109 [ 113919404000] : MEMREAD32, Address = 00000000, Read Data 01234567
110 [ 113919404000] : TEST PASSED --- Mem 32 for pio_memTestByteEnables_test1
111 [ 113919404000] : Finished transmission of PCI-Express TLPs

```

图 4-13 字节使能功能的验证报告

4.3.3 时钟和复位仿真

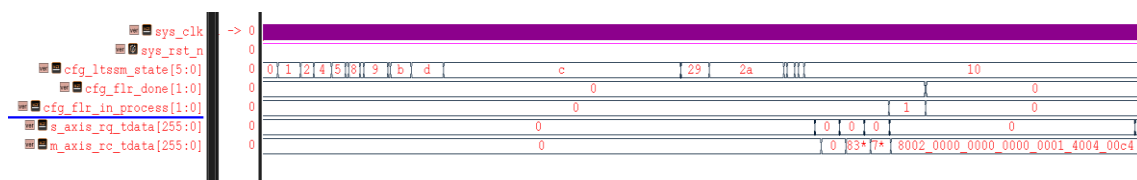
本次仿真主要是为了验证 PCI Express Core 的复位功能。仿真中包含冷复位（Fundamental Reset），热复位（Hot Reset）和功能复位（Function Level Reset，FLR）三种复位情况。

Fundamental Reset 即为软件的系统初始复位，在软件上电后开始基本复位，在环境中用信号 sys_rst_n 表示。如图 4-14 所示，在系统启动阶段，sys_rst_n 信号为低，链路训练状态机处于 Detect.Quit 状态，表示系统所有的信号传输停止，准备开始进行链路训练；当系统施加电源、系统初始复位完成，sys_rst_n 信号拉高，等待 RC 端和 EP 都收到 sys_rst_n 信号的变化值，开始进行链路训练，链接训练状态机依次进入 Detect.Active、Polling.Active、Polling.Configuration 等状态，系统开



Hot Reset 热复位为主要复位中的一种，又称作带内复位。上游设备发出热复位 TSI 有续集指令，下游设备接收并复位整个端点设备。热复位一般是在链路训练完成之后进行的复位。如图 4-15 所示，在热复位信号 `cfg_hot_reset_in` 信号拉高前，系统复位信号 `sys_rst_n` 为高，表明系统已经完成了系统初始复位。链路训练状态机信号 `cfg_ltssm_state[5:0]` 的值为 `6'h10`，此时链路已经完成链路训练，处于 L0 状态，整个系统电源全开、处于完全操作的链路状态；当热复位信号 `cfg_hot_reset_in` 拉高，系统进入热复位状态，链路训练状态机依次进入恢复锁定、恢复配置、恢复空闲、热复位状态，系统启动热复位机制完成。

图 4-15 热复位波形图



FLR Reset 为功能性复位，该复位可以让 PCIE 的某个特定事务功能进入复位状态。FLR Reset 也是带内复位的一种，`cfg_flr_in_process` 或 `cfg_vf_flr_in_process` 信号有效表示该设备已进入 FLR 状态，可以通过断言 `cfg_flr_done` 或 `cfg_vf_flr_done` 信号值解除 FLR 复位状态。如图 4-16 所示，在链路进入 L0 状态后，`m_axis_rc_tdata` 端口开始正常接收来自终端设备的配置读请求者完成信号，到 `cfg_flr_in_process` 信号拉高，表示功能复位开始，可以看到 `m_axis_rc_tdata` 的数据停止接收，表明系统的配置读请求功能能被复位。当 `cfg_flr_done` 断言后，`cfg_flr_in_process` 信号置 0，系统检测一段时间后，确定功能复位解除，

m_axis_rc_tdata 的配置读请求完成数据包可以正常接收，验证通过。

4.3.4 电源管理的验证

电源管理用例是为了验证 PCI Express Core 是否支持电源管理相关的功耗管理功能。此次仿真中使用 pio_writeReadBack_test0 向量，通过软件命令配置 PMCSR 寄存器将设备的电源状态调节到 D3_hot 状态，此时设备处于全关闭节能状态，设备不再发起各种事务协议的传输；链路功耗状态经历了从 L1 到 L0 状态的转变，同时状态机从 L0 跳到 L1 状态再到 L0 状态；等待 500 个时钟周期后，通过软件命令配置 PMCSR 寄存器将电源状态重新调节为 D0 状态链路状态。

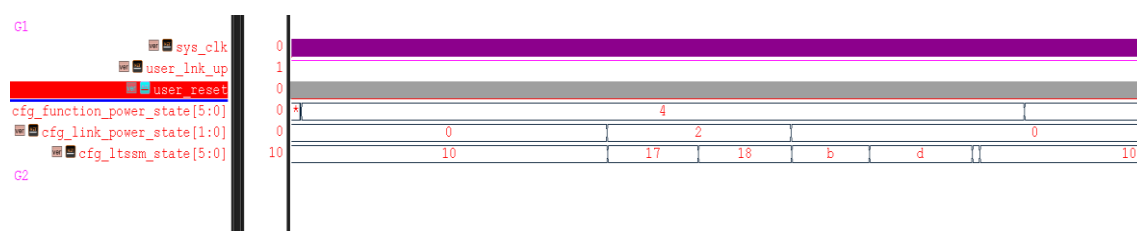


图 4-17 电源管理状态机信号

如上图 4-17 所示，cfg_function_power_state 的状态由 D0_uninitialized 到 D0_active 再到 D3_hot，最后又回到 D0 状态，同时 cfg_link_power_state 的状态由 L0 到 L1，然后又回到 L0 状态，LTSSM 状态机经历了从 10(正常工作状态)到 L1.Entry、L1.Idle、恢复锁定、恢复配置、恢复空闲又到正常 L0 状态，如下图 4-18 所示。EP 端连续发送了 5c20_0000_0065_adfdh，该 DLLP 包为 PM_Enter_L1 电源管理包，这表明系统实现电源管理的方式是通过发送电源管理类的 DLLP 包。图 4-19 是 RP 端点的接收通道波形图，在 RP 接收通路上的物理层通路上 5c20_0000_0065_adfdh 被正确接收到，RP 开始屏蔽所有事务包数据的发送。

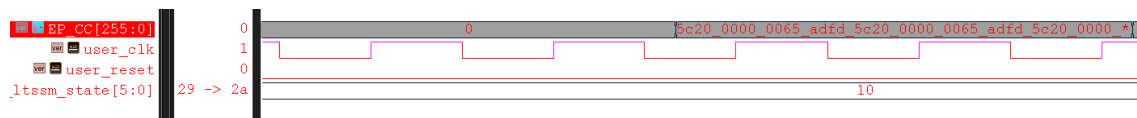


图 4-18 EP 端发送的 DLLP 包

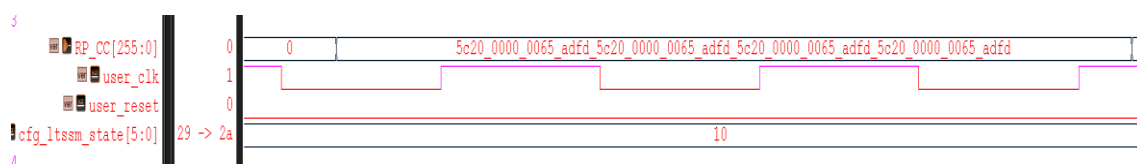


图 4-19 RP 端接收的 DLLP 包

图 4-20 为 RP 的发送通路上物理层的部分数据通路，RP 连续发送了 5c2400000930cfdh，该 DLLP 包为 PM_Request_Ack 至 EP 的数据包接收确认握手信号。至此 PCIE 电源管理状态、电源管理事件和电源管理消息的功能可以得到验证。

4.3.5.1 INTX 中断的验证

如图 4-21 所示, End Point 端口的 Cfg_interrupt_sent 端口上可以观察到发送了中断消息(即信号拉高), Cfg_interrupt_int 为 1 时发送的是 Assert_INTA 消息;同样可以从图 4-22 看出, EP 的物理层数据通路上的总线数据可以得出发送的事务包为 fb_001a_3400_0000_0100_0020_0000_0000_00c1_4e1b_7efd_h, 其中 34 表示消息事务类型, 20 是 Assert_INTA 的消息代码, Assert_INTA 中断消息事务发送成功, Cfg_interrupt_int 为 0 时发送的是 Deassert_INTA 消息, Deassert_INTA 中断消息事务发送成功。当断言中断时 Cfg_interrupt_pending 的开启保证中断状态能够被正确的更新到状态寄存器中去。

44

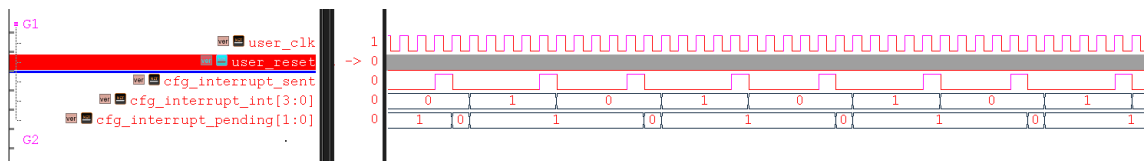


图 4-21 EP 端端口的中断信号

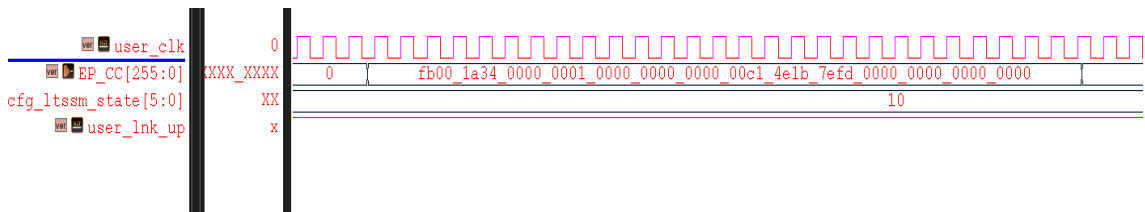


图 4-22 EP 端数据通路的中断响应

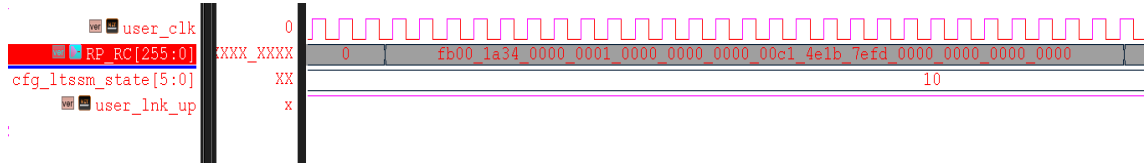


图 4-23 RP 端的中断响应信号

4.3.5.2 MSI 中断

MSI 的仿真依旧使用的仿真向量 pio_writeReadBack_test0。其具体行为是：首先通过软件对 Status Register，MSI Control Register 和 Root Control Register 三个和 MSI 中断功能相关的配置寄存器进行配置；然后上端设备根复合体 RP 端发送一个存储器写事务，目的是执行存储器写事务；同时，EP 开始发送 MSI 中断请求，MSI 中断事务并不是消息事务，其发送方式是 EP 端点向 RP 端点发送的存储写事务（32 bit/64 bit）从而声明 MSI 中断；观察 EP 端 cfg_interrupt_msi_enable、cfg_interrupt_msi_int、cfg_interrupt_msi_sent、cfg_interrupt_msi_function_number、cfg_interrupt_msi_pending_status 端口信号是否产生响应。最后由下游设备 EP 向 RP 端发送中断事务包生成中断请求，RP 执行中断相应。

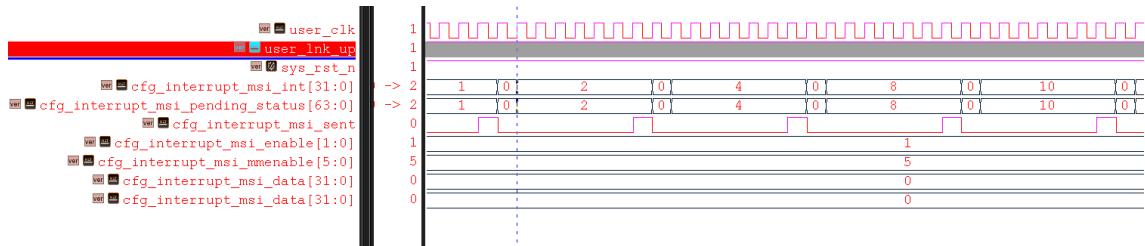


图 4-24 EP 端发起的中断响应波形

图 4-24 中所示为 EP 发起的 MSI 中断仿真的波形，Cfg_interrupt_msi_int 配置寄存器信号表明本次仿真声明的是 MSI 的中断类型，Cfg_interrupt_msi_enable 开启，说明中断状态能够被正确的更新到状态寄存器中去；Cfg_interrupt_msi_data 说明配置的 MSI 能力寄存器中写数据类型为 32 位、写数据为 32'b0；Cfg_interrupt_sent 拉高，表明中断请求产生。



(a)



(b)

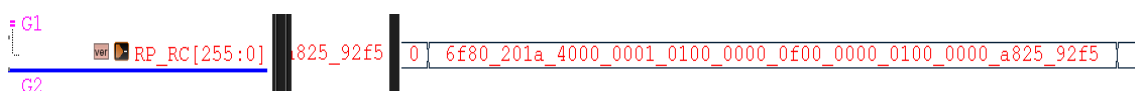


(c)

图 4-25 MSI 响应储存写事务包。(a) 第一帧 MSI 响应储存写事务包；(b) 第二帧 MSI 响应储存写事务包；(c) 第三帧 MSI 响应储存写事务包



(a)



(b)

图 4-26 RP 端的 MSI 包响应。(a) RP 端的第一帧 MSI 包接收响应；(b) RP 端的第二帧 MSI 包接收响应

图 4-25 中为 EP 端点发送的 MSI 中断事务包，6f80_201a_4000_0001_0100_0000_0f00_0000_0000_0000_a825_92f5h 为一个 32 位存储器写事务包，是 EP 端响应的 MSI 中断事务包；分析一帧包的内容可以得出：包头标志位 40 为 32 bit 存储写事务，地址为 32'h0，地址位为 MSI 能力寄存器中定义的 Message 地址，写数据为空，数据来自于软件定义的 MSI 能力寄存器组中

的数据，说明这是一个 MSI 中断请求的写事务包。图 4-25(b)、4-25(c)说明 EP 连续发送了几个 32 bit 的存储写事务，写地址依次递增，写数据不变。

图 4-26 为 RP 端点接收到的 MSI 中断事务包，能够断定 RP 端点准确的接收到了 EP 的 MSI 中断请求。MSI 中断成功的由 EP 端发起，且通过写事务包的形式传递给了 RP 端，RP 正确接收该事务，具体的 MSI 中断由设备完成，这里不做验证；由上可知，PCI Express Core 的 MSI 中断功能执行正确，验证通过。

4.3.5.3 MSI-X 中断

MSIX_Interrupt 是 MSI 中断的改进型，不同于 MSI 中断，MSI-X 中断支持 64 位中断请求，并且不要中断请求连续。本次仿真中使用的仿真向量为 pio_writeReadBack_test0。具体的仿真行为为：首先对 Status Register，MSI-X Control Register 和 Root Control Register 进行配置；然后从根复合体 RP 端发送一个存储器写事务，观察 EP 端 `cfg_interrupt_msix_enable` 信号，`cfg_interrupt_msix_int` 信号，`cfg_interrupt_msix_sent` 信号，`cfg_interrupt_msix_data` 信号，`cfg_interrupt_msix_address` 信号产生响应。最后 EP 向 RP 端发送存储器写事务，判断 RP 是否接受到 MSI-X 中断请求。

如图 4-27 所示，RP 端的 MSI-X 能力寄存器的配置数据地址是 `dead_beefh`，数据地址为 `aaaa_bbbb_cccc_ddddh`；在 EP 端点的输出端口 `Cfg_interrupt_msix_sent` 可以看到 MSI-X 中断发送成功的激励响应。



图 4-27 MSI-X 能力寄存器的配置波形

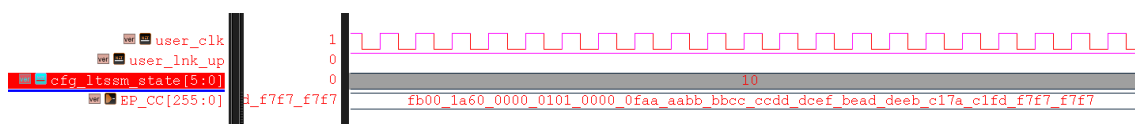


图 4-28 EP 发送的 MSI-X 事务包

由图 4-28 可知，EP 端点发送了 MSI-X 事务包类型为 64 bit 的存储写 MSI-X 事务类型，存储地址为 `aaaa_bbbb_cccc_ddddh`，写的数据为 `dead_beefh`，匹配 MSI-X

能力寄存器组的配置值，说明这是一个 MSI-X 中断请求。

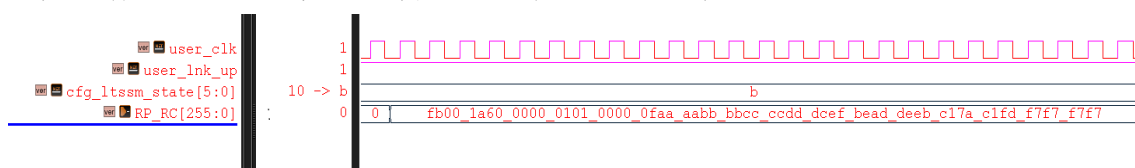


图 4-29 RP 接收的的 MSI-X 事务包

由图 4-29 所示，由 EP 端点发送的 MSI-X 消息事务包被 RP 端点正确的接收到。MSI 中断成功的由 EP 端发起，且通过写事务包的形式传递给了 RP 端，RP 正确接收该事务，具体的中断响应行为由设备完成，这里不做验证；由上可知，PCI Express Core 的 MSI-X 中断功能执行正确，验证通过。

4.3.6 Latency Tolerance Reporting 消息的验证

延迟容忍报告（Latency Tolerance Reporting, LTR）机制使 PCI Express 端点能够报告其各类请求事务对根复合体的内存读写的延迟要求，以便中央平台资源（如主存储器、RC 内部）的管理策略可以实现互连和侦测资源来考虑端点服务需求。本次验证需要验证 PCIE 的 RP 端点能够正确接收到 LTR 的消息事务。LTR 验证用例中使用的仿真向量为 pio_writeReadBack_test0。具体行为是：首先对 PCI Express Link Capabilities Register 和 No-Snoop/ Snoop Register 寄存器组进行配置进行配置；然后从设备端 EP 端发送一个 LTR 消息事务，根复合体 RP 发送内存读写事务。观察 EP 端 cfg_ltr_enable 信号被声明，RP 端 cfg_msg_received 信号有一个 6 周期的脉冲。cfg_msg_received_type 信号数值为 16，cfg_msg_received_data 信号显示数据无误。

当打开 AXISTEN_IF_ENABLE_RX_MSG_INTFC 使能后，在 RP 端点的 cfg_msg_received， cfg_msg_received_data， cfg_msg_received_type 端口均能检测到 LTR 消息的属性值。

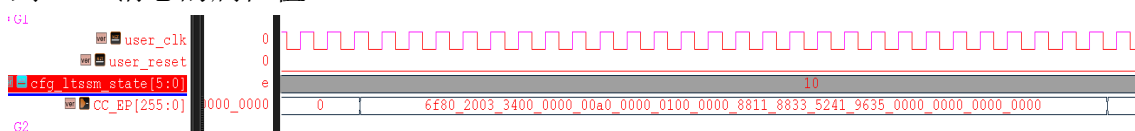


图 4-30 EP 端口发送的 LTR 消息事务

首先验证 LTR 相关的数据包收发功能。图 4-30 是 EP 物理层发送端上的通路数据，在完成了 LTR 相关寄存器组的配置后，EP 端发送了一帧 LTR 的消息事务。数据载荷为 6f80_2003_3400_0000_00a0_0000_1000_0000_8811_8833_5241_9635h，其中 6f_8020 为 gen3 模式下 128/130b 编码方式 PCI Express 事务包的开始帧标志

位，与 gen2 和 gen1 模式下的 8b/10b 的编码方式存在区别；03h 为序列号，34h 表示不带数据的消息事务，10h 表示 LTR 的消息代码，8811_8833h 为写入的数据，5241_9635h 为 LCRC 数据链路层 CRC 数据。图 4-31 是 RP 端口接收这一帧消息事务的波形图，说明 LTR 消息事务在 PCI Express 系统内能正确的产生、发送和接收。

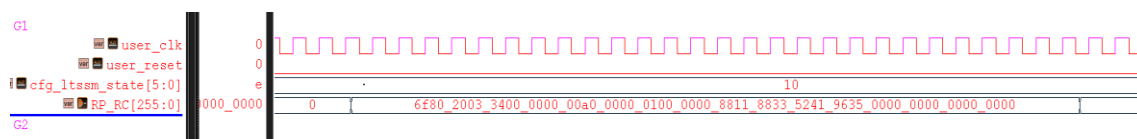


图 4-31 RP 端口接收的 LTR 消息事务

如下图 4-32 所示，图中打印了整个仿真 RP 设备的 `cfg_msg_received`、`cfg_msg_received_data`、`cfg_msg_received_type` 端口的激励响应情况。`cfg_msg_received_type` 为 10h，`cfg_msg_received` 为一个高电平脉冲响应，`cfg_msg_received_data` 的数据依次为 0(Requester ID, Bus Number)，a0(Requester ID, Device/Function Number)，88(侦测延时)，11(侦测延时)，88(不侦测延时)，33(不侦测延时)。RP 准确的接收到了 LTR 消息，用例验证通过。

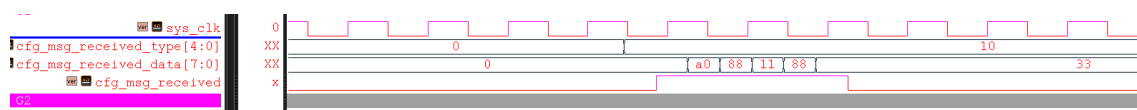


图 4-32 RP 设备的 LTR 消息响应

4.3.7 Alternative Routing-ID Interpretation 功能验证

Alternative Routing-ID Interpretation (ARI) 是一种扩充设备支持功能数量的功能。在传统设备路由规则中，16 位 `Bus_number+function_number+device_number` 中功能号只有 3 位，即一个设备只能支持 8 种功能。ARI 功能将设备的 `Bus_number+function_number+device_number` 改变为 `Bus_number+function_number`，支持 ARI 功能的设备可支持的最大功能个数增加至 256 个，用户可以对更改的 `Bus_number+function_number` 进行预编号。

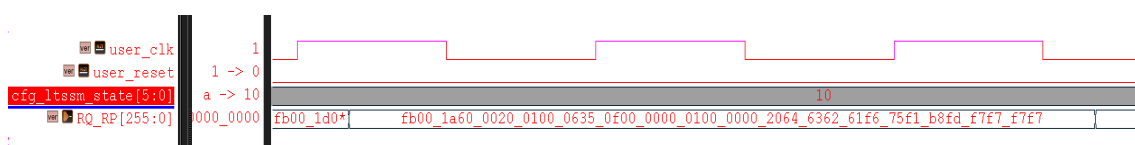
对于 ARI 功能的验证，我们采用的仿真向量为 `pio_writeReadBack_test0`。具体行为为：首先通过配置相关寄存器，打开 ARI 功能后，并在发送 AXI 传输包中打开 RID 使能，将 64 bit 的存储写和存储读的完成数据包中未打开 ARI 功能之前的 `Busnumber+functionnumber+devicenumber` 值 16'h0006 改变为打开后 `Bus_number+function_number` 值 16'hdde; 而 32 bit 的存储器读写事务包不打开 ARI 功能，系统将按照默认的 `Busnumber+functionnumber+devicenumber` 值 16'h0006

进行事务响应。

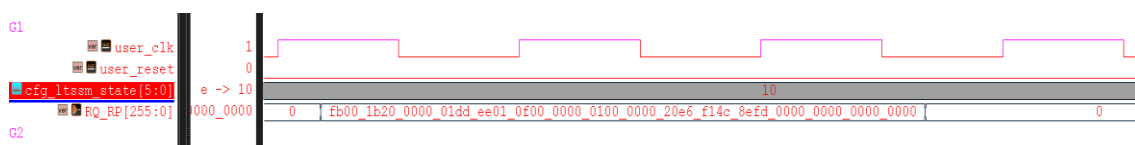
我们从上游设备 RP 中发送 32 位、64 位配置读写事务包，其包的数据内容见下图 4-33。

图 4-33 是 RP 端发送的 ARI 功能打开的 64 bit 存储器事务和 ARI 功能关闭的 32 bit 存储器事务，我们分别往 64 bit 存储器读写空间里写入 6463_6261h；往 32 bit 对应地址位写入数据 0403_0201h；然后发送读请求，将对应位置的数据读出。

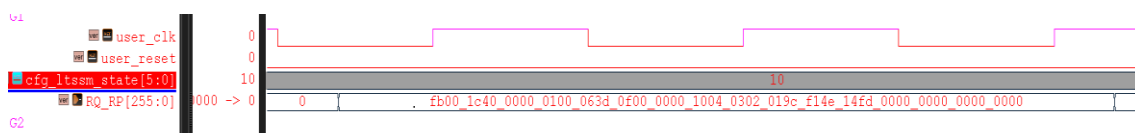
图 4-34 是 EP 端点发送的数据为带数据的完成返回包，且 64 bit 存储读事务的完成返回包的功能设备号为 ddeeh，而 32 bit 存储读事务的完成返回包的功能设备号仍为 0006h。



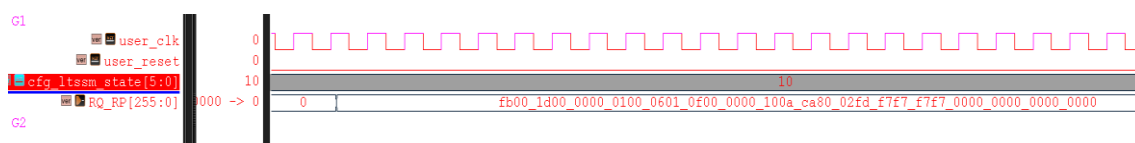
(a)



(b)

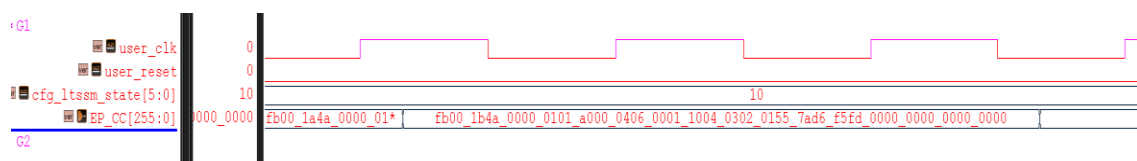


(c)

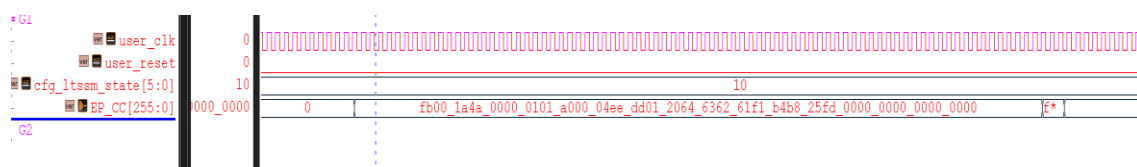


(d)

图 4-33 RP 发送的 32 位 64 位配置读写事务包。(a) ARI 打开前的 64 bit 存储器写请求；(b) ARI 打开后的 64 bit 存储器读请求；(c) 32 bit 存储器写请求；(d) 32 bit 存储器读请求



(a)



(b)

图 4-34 EP 端读出的带数据完成请求包。(a) EP 端读出的 32 bit 带数据完成请求包;

(b) EP 端读出的 64 bit 带数据完成请求包

图 4-35 是软件层 m_axis_rc_tdata 端口的数据，在 RP 端点的软件层接收端，可以看见 64 bit 读请求的反馈数据 6162_6364h 的完成返回包的功能号、设备号更改为 ddeeh, 32 bit 读请求的反馈数据 0102_0304h 的完成返回包的总线号、功能号、设备号仍为 0006h。用例验证通过。

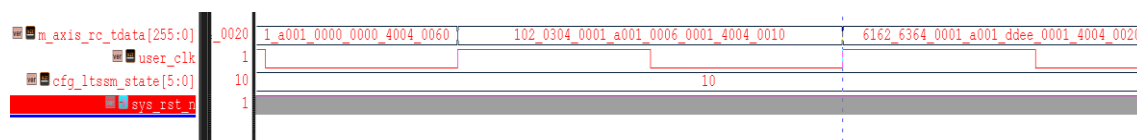


图 4-35 软件层读出的完成数据包

4.3.8 Optimized Buffer Flush/Fill 消息的验证

优化缓存刷新/填充功能（Optimized Buffer Flush/Fill, OBFF）是 PCI Express 规范提供的一种上游设备给下游器件报告窗口缓存期的机制。在本此验证中，由根联合体 RP 端向端点 EP 报告有关总线等活动的时间窗口。OBFF 有“CPU 活跃”，“OBFF”，“空闲”三种窗口事件，OBFF 发起可以由 WAKE 信号唤醒，也可以由消息协议发送。OBFF 向量仿真中使用的仿真向量为 pio_writeReadBack_test0，该向量主要是用来验证消息协议的发送。在 PIO 的基础上配置好 OBFF 的使能端口，调用消息事务发送 OBFF 的消息，在 EP 接收端的消息接收。

图 4-36 是截取的上游设备 RP 端的一帧 Optimized Buffer Flush/Fill 消息事务的波形图，可以看到 RP 的发送端发送了一帧 fb0_0033_4000_0000_0000_0120_0000_0000_0fcb_ea21_03fdh 数据包，该事务包为

RP 处理层发送的消息事务，34h 表示不带数据的消息事务，12h 为 OBFF 的消息代码，fh 为 OBFF 代码代表 CPU 处于活跃状态。

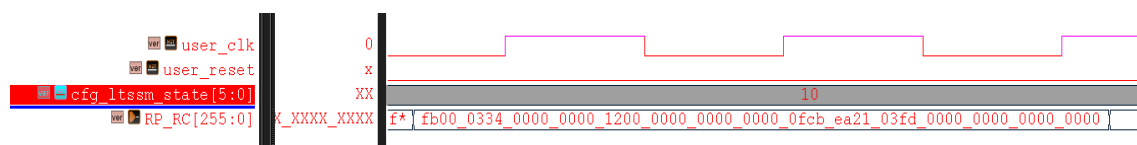


图 4-36 RP 端发送的 OBFF 消息事务

图 4-37 截取的下游设备物理层端口的数据，可以看到 RP 端口发送的 OBFF 消息已经被成功接收。

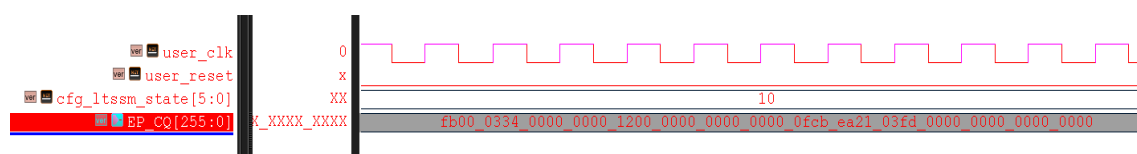


图 4-37 EP 端接收的 OBFF 消息事务

PCIE 的 EP 端点能够正确检测接收到的 OBFF 的消息事务。打开 AXISTEN_IF_ENABLE_RX_MSG_INTFC 使能后，如图 4-38 所示，OBFF 消息被物理层端口接收后，被数据链路层和事务层解包后，将相应事务数据传递到事务层，EP 端点的 `cfg_msg_received` 拉高，表示 OBFF 消息 EP 端成功接收，`cfg_msg_received_type` 表示接收到的 OBFF 类型为 11h，`cfg_msg_received_data` 为接收到的 OBFF 数据类型 fh，为 CPU Active 状态，符合预期要求。

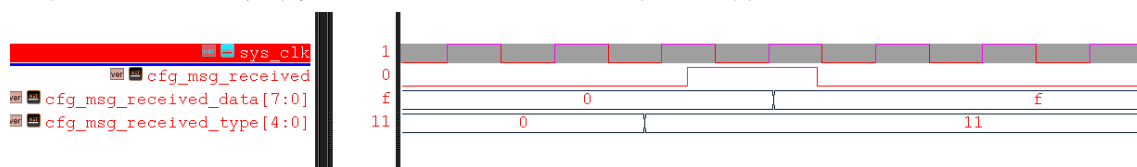


图 4-38 EP 端口的 OBFF 消息响应

4.3.9 基地址寄存器的验证

PCI Express 规范给出了 IO 基地址寄存器和存储器基地址寄存器两种基地址寄存器的使用规范。基地址寄存器存在三种格式，如下图 4-39、4-40、4-41 所示。

在基地址寄存器空间中，基地址空间标志位最低位为 0 时表示这是一个存储器基地址寄存器，为 1 时表示这是 IO 基地址寄存器；基地址寄存器类型标志位[2:1]位是存储器基地址寄存器特有的，当为 00b 时表示这是 32 位存储器基地址寄存器，为 10b 时表示这是 64 位基地址寄存器；可预取位表明了该存储器块是否可预取，

为 0 是不可预取，为 1 时可预取；基地址位为剩余的高位数据，用来申请基地址空间容量或者接受配置软件分配的地址。

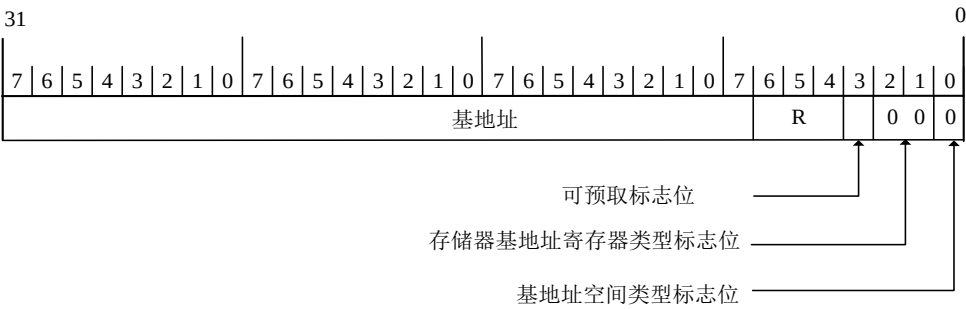


图 4-39 32 位存储器基地址寄存器



图 4-40 64 位存储器基地址寄存器

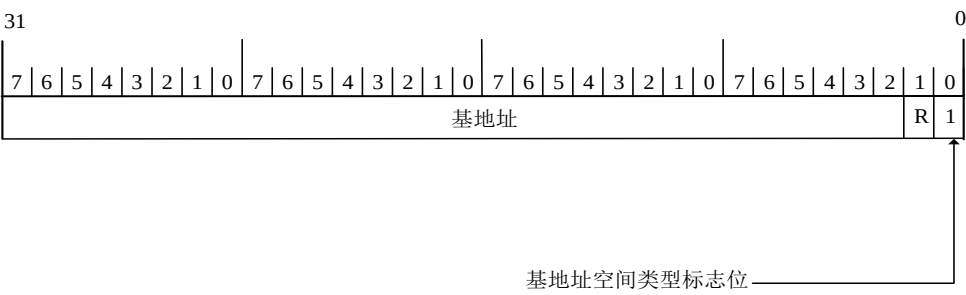


图 4-41 32 位 IO 基地址寄存器

对于基地址寄存器的仿真我们使用仿真向量为 `pio_writeReadBack_test0`。当链路初始化过程中，会初始化基地址空间，这时会扫描每个基地址寄存器的空间地址以及存储容量大小以及读写的方式。我们通过更改不同地址的配置端口基地址寄存器的[12:8]位数据来将基地址寄存器配置成用户定义的存储容量。

图 4-42 是用户通过配置端口配合基地址寄存器的波形图。我们通过下游设备 EP 端的 `cfg_mgmt_write_data` 将地址为 62 的基地址寄存器配置为 800h 的容量，根据设计说明，800h 的基地址空间大小为 256MB。

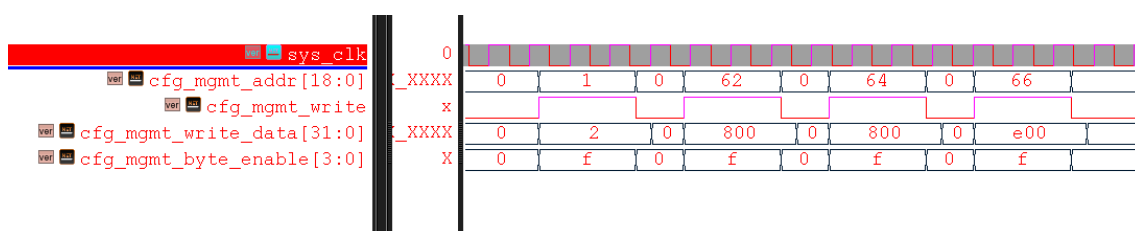


图 4-42 基地址寄存器配置波形图

根据图 4-43 基地址寄存器的初始化仿真打印报告，在系统扫描 BAR0 的时候，返回的基地址寄存器的值为 `f000_0004h`，由上文基地址寄存器的格式可以，低四位 `4'b0100` 表示这是一个 64 位存储器基地址寄存器，Range 位的最高可写 1 位是第 28 位，所以该基地址寄存器的容量为 2^{27} b，即 256MB。扫描结果和配置结果一直，验证通过。

```

9805541b] : 15K_PXKSE_FKXME on Receive
99495397] PCI EXPRESS BAR MEMORY/IO MAPPING PROCESS BEGUN...
BAR 0: VALUE = 00000000 RANGE = f0000004 TYPE = MEM64 MAPPED
BAR 1: VALUE = 00000001 RANGE = ffffffff TYPE = DISABLED
BAR 2: VALUE = 00000000 RANGE = fff00000 TYPE = MEM32 MAPPED
BAR 3: VALUE = 00000000 RANGE = 00000000 TYPE = DISABLED
BAR 4: VALUE = 00000000 RANGE = 00000000 TYPE = DISABLED
BAR 5: VALUE = 00000000 RANGE = 00000000 TYPE = DISABLED
EROM : VALUE = 00000000 RANGE = 00000000 TYPE = DISABLED

```

图 4-43 基地址寄存器的初始化仿真扫描结果

4.3.10 极性倒置的验证

极性反转功能是 PCI Express Core 差分信号传输的特性功能。在 PCI Express 的传输链路中，采用了差分传输信号进行信号传输。对于设备的某一个通道的发送和接收端，都是由一组差分信号对来进行信号传输。如图 4-44 所示。

设备 A 的 Tx 和 Rx 的通道上都由 D+和 D-信号组成的差分对信号进行发送。这样做的好处是可以避免信号在高速传输过程中一些外部信号对链路产生干扰，同时降低了系统功耗。而不同设备链路通道的差分信号对的高低位信号位并不是

确定的、唯一的，在实际应用过程中容易出现如图设备 B 所示的情况：在设备未链路极性倒置前，设备 A 的高位差分信号同设备 B 的低位差分信号链接在一起，设备 A 的低位信号和设备 B 的高位信号链接在一起，这样就会严重扰乱链路的信号发送，而设备 B 进行链路的极性倒置后，这一问题得到解决。

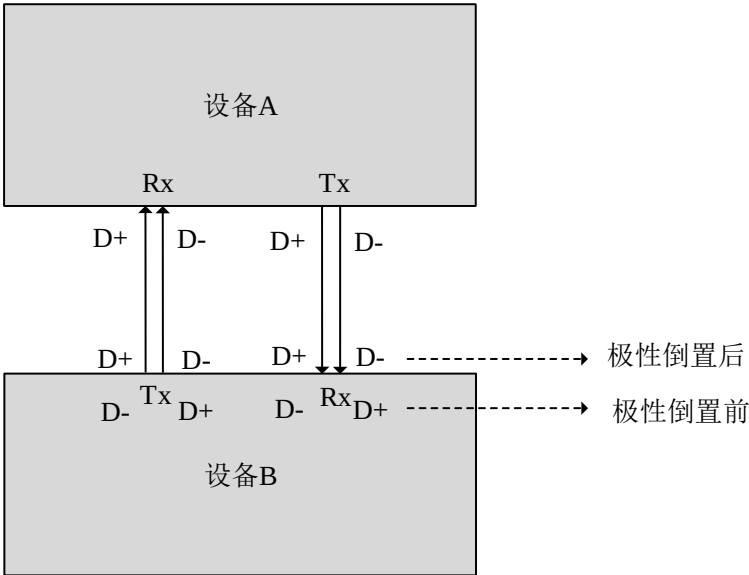


图 4-44 极性倒置功能架构图

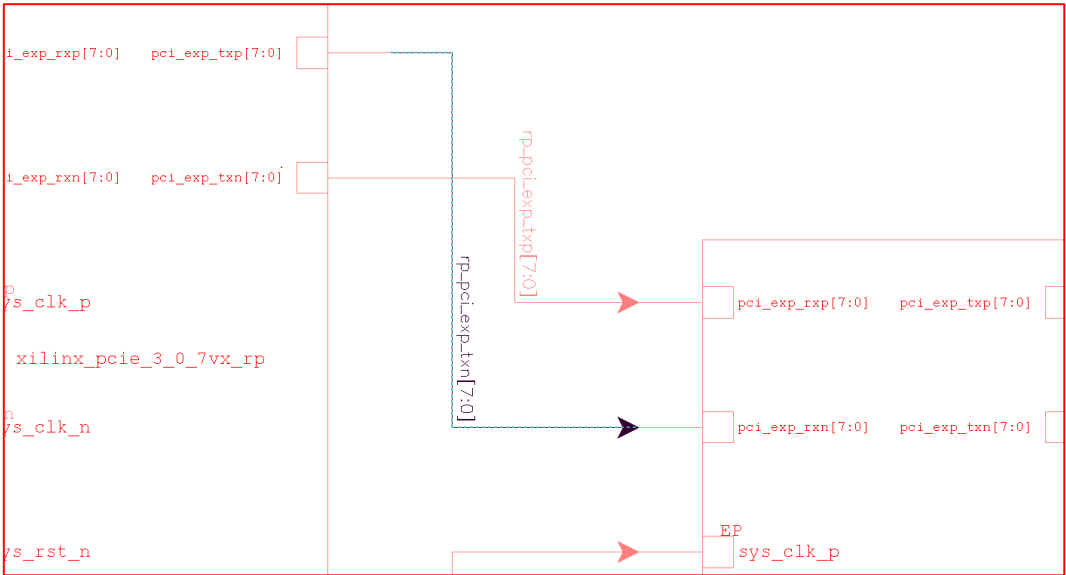


图 4-45 仿真环境中的极性反接

本验证用例使用的仿真向量为 sample_smoke_test0。任务开始仿真之前，在顶

层文件中将两个 PCIE 的链路端口极性反接，通道数进行打乱相互连接，即通道序列号不是一一对应相连接。根据验证波形分析在通道没有正确连接的情况下，PCIE 内置的极性反转和极性倒置功能是否自动纠正这些通道连接错误，RP 发送的数据是否能够被 PCIE 的 EP 正确接收，从而验证 PCIE 的极性倒置和极性反转功能。

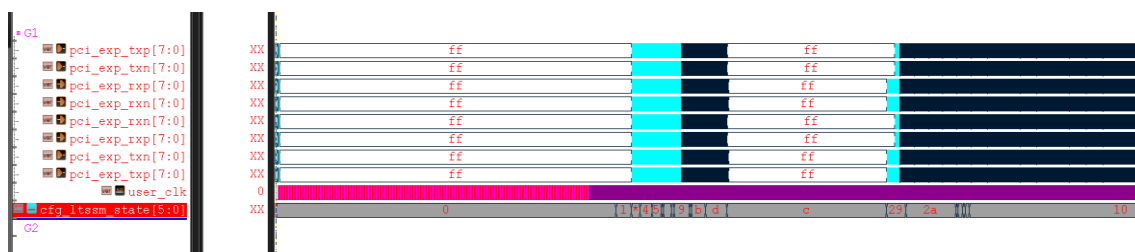


图 4-46 极性倒置后的整片数据流

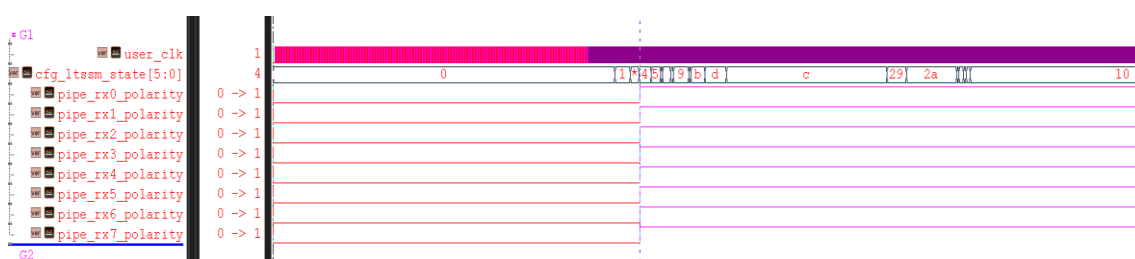


图 4-47 PIPERX 通路的极性跳转

图 4-45 是在仿真的顶层中，PCI Express 的链路上的 P、N 信号进行反接设置，即 pci_exp_txp 接到了 pci_exp_rxn 上，pci_exp_txn 接到了 pci_exp_rxp 上，两个总线端口正负极性均反接。图 4-46 显示是这种仿真环境下的仿真结果，RP 和 EP 之间仍然能够建立正常的信号传输交互。

由图 4-47 可以看出 PIPERX[7:0]POLARITY 端口的极性状态均发生跳变，即每条通道因为极性反接而发生了极性倒置的功能。极性倒置功能得到验证。

4.4 本章小结

本章主要论述对 PCI Express Core 的功能验证工作。分别从验证方式、验证环境、验证工具和验证结果等方面进行了详细的阐述。本章是 PCI Express Core 研究的重要组成部分，通过搭建的仿真环境，对实际的电路文件进行行为仿真，充分验证了 PCI Express 协议规范中提到各项主要功能，是 PCI Express 总线技术在高性能 FPGA 芯片中应用的重要检验方式。本章的验证结果分析结合第二章的 PCI Express 介绍的体系架构、分层结构、数据传输模型等理论知识，验证了实际电路中对规范协议的实现，功能覆盖率达到 100%。本章提出了一种复用性较强的可通

用于验证和应用开发的仿真平台，可有效提升第五章应用测试开发相关工作的效率。

第五章 千万门级 FPGA 中 PCI Express 接口应用测试

在完成了上文的工作后,本文后续完成了一项基于 PCI Express 接口的千万门级 FPGA 中 DDR3 读写访问应用测试工程。工程使用了本次项目中流片后的测试样片,作者实习单位的替代性自研软件对 FPGA 芯片的应用测试工程提供了软件支持。同时,这一部分的工作移植了上文第四章中搭建的验证平台中的数据流架构和激励,提高了工程的效率。

5.1 架构设计

测试工程的架构设计充分借鉴了第四章使用的端对端的直接访问架构,其架构模型如图 5-1 所示。和验证平台的架构类似,应用平台使用了端对端的直接访问架构,上游设备执行 PCI Express 体系中根复合体的逻辑功能,下游设备执行终端设备的功能。由于本文使用的 FPGA 芯片集成了三个 PCI Express3.0 的 IP 核,工程以节约资源、最大化利用单一芯片资源的原则,应用场景选择在一块 FPGA 芯片中进行实现。

该测试平台主要由 JTAG 接口电路、收发模块(Transceiver)、PCIE EP 核、PCIE RP 核组成和中断设备组成。工程中的软件部分提供了外围的 JTAG 指令集的数据,用户可以通过 iMPACT 工具和 JTAG 下载器,下载编写了指令和数据的 SVF 文件给 FPGA 内部,同时接收来自 FPGA 内部的数据并通过软件读出;JTAG 模块接收来自软件的指令集数据,根据指令给出不同的标志信号并且将指令翻译成相应的信号发送至收发模块,系统反馈的信号将在这个模块中进行处理,反馈有效信息给外部软件设备;收发模块(Tranceiver)的作用是生成 AXIS 接口数据传输格式给到 FPGA 中的 PCI Express 核接口,拆解来自 PCIEExpress 核接口返回的基于 AXIS 协议的有效数据;调用的两个 PCI Express 核将构成构成的主体架构,数据在核间进行相应的高速传输;终端设备是通过调用 FPGA 资源和开发板的 DDR3 颗粒,组成的 DDR3 设备,下文将对各个模块进行详细说明。

本次工程使用了三种形式的数据线,这里进行说明。首先是数据输入端,我们使用了 FPGA 测试模式的 JTAG 数据端口,外部软件的数据通过 JTAG 下载数据线和 FPGA 芯片进行数据交流;终端设备由于使用了开发板的资源(DDR3 颗粒),其数据流通使用了开发板的内部走线,这是另外一个工程领域的问题,这里不作具体描述;最后是 PCI Express 核间的信号传输问题,由于我们是直接调用了 FPGA 芯片的两个 IP 核,根据芯片的说明,FPGA 的内部走线无法直接连接两个 PCI

Express 核。由于核间传输的带宽接近 8GT/s，本文使用了高速差分信号线在开发板上对 PCI Express 核的输入输出端口进行了连接。

5.2 子模块设计

5.2.1 JTAG 接口模块

5.2.1.1 IEEE 1149.1 协议简介

本项目的在系统编程采用 IEEE1149.1 (JTAG) 标准协议，JTAG 协议接口最初是用于芯片测试，基本原理是在器件内部定义一个测试访问端口 (Test Access Port, TAP)，通过专用的 JTAG 测试软件及工具对内部节点进行测试。JTAG 测试允许多个器件通过 JTAG 接口串联在一起，形成一个 JTAG 链，能实现多个器件的测试。

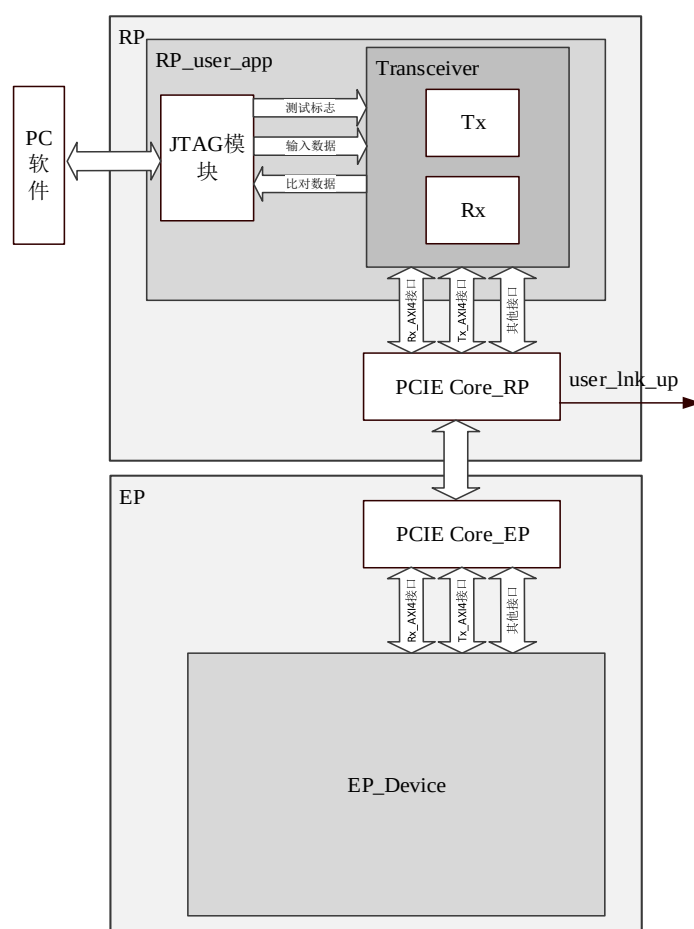


图 5-1 测试平台架构

JTAG 的协议架构如下图 5-2 所示：

从上图中可以看出, JTAG 架构主要包括 TAP 控制器、边界扫描寄存器、指令寄存器、数据寄存器、IDCODE 寄存器以及 Bypass 寄存器等模块组成, 其中边界扫描寄存器主要用于测试器件内部逻辑; 指令寄存器及指令译码主要用于接收外部指令、并译码产生相应的控制信号; 数据寄存器主要用于加载器件的配置信息; IDCODE 寄存器主要用于储存器件的型号、版本以及加工等信息; Bypass 寄存器主要用于 JTAG 链; TAP 控制器是一个模为 16 的有限状态机, 主要用于控制 JTAG 状态, 其状态图如下图 5-3 所示。

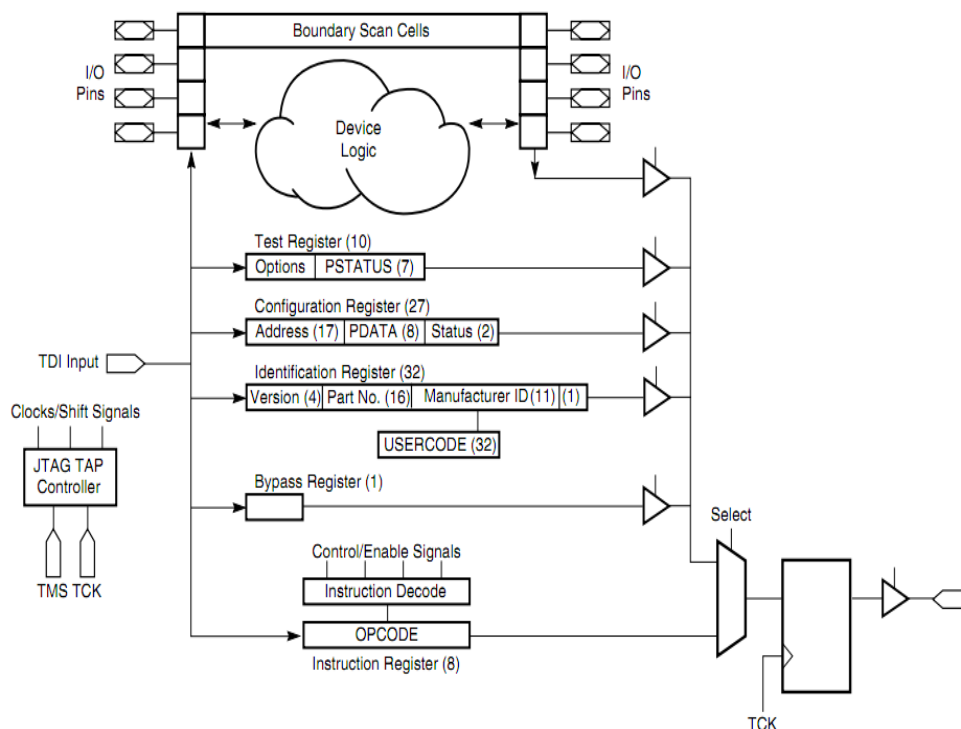


图 5-2 JTAG 协议框架^[43]

5.2.1.2 JTAG 指令设计

本项目中, 使用了基于 JTAG 协议的 SIR 和 SDR 指令集。SDR 指令输入位宽是 32 bits; 定义的 SDR 指令可用做特殊的功能请求, SDR 指令定义见附录表附-1; 另一方面, 由于 SDR 指令的单个指令数据位宽为一个 DW, 在项目中, 我们同样适用 SDR 指令进行 TLP 数据的手动输入; SIR 指令输入位宽是 10 bits, SIR 指令的主要作用是唤醒定义在 Transceiver 中的 PCIe 操作数据流, 系统每发出一个 SIR 有效指令, 系统都会根据 Transceiver 中的功能定义, 进行一系列的读写操作, 这一部分的工作继承了第四章的测试向量集的内容; SIR 和 SDR 指令的功能定义见附录表附-1 TLP 包的 SDR 指令说明、表附-2 JTAG SIR 指令说明。

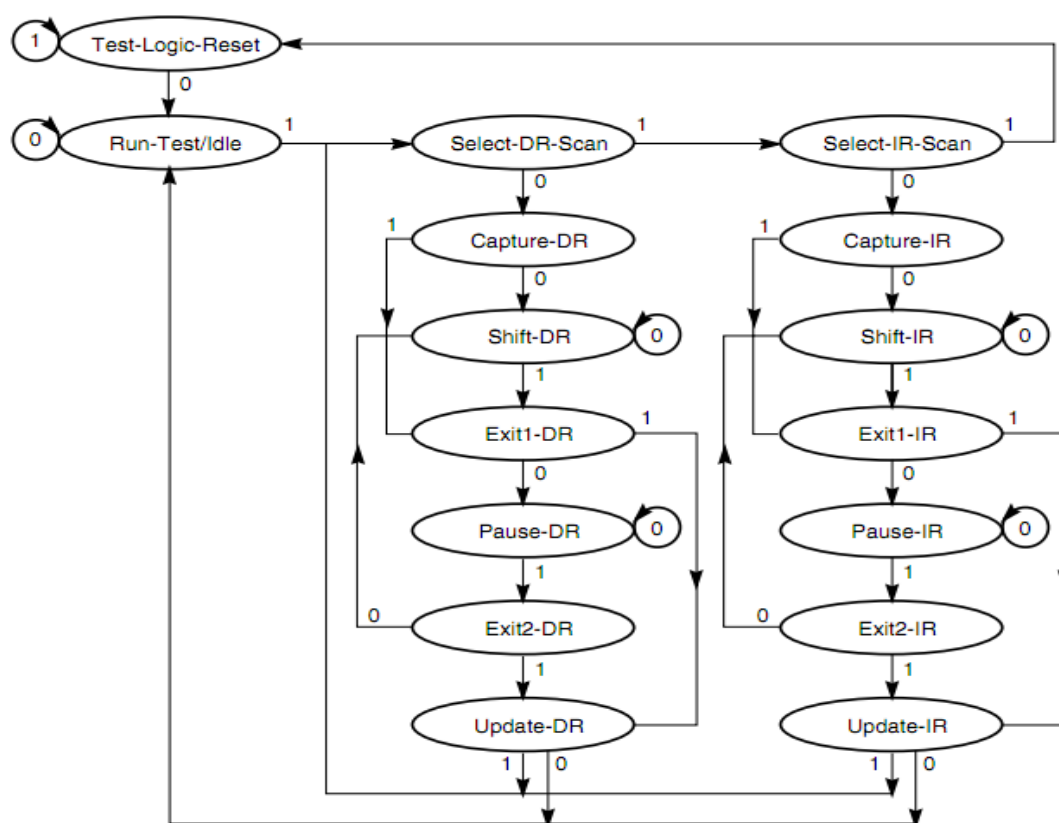


图 5-3 TAP 状态图

5.2.1.3 TLP 输入简化

用户在访问 PCIe 时需要根据 PCIe 的 TLP (事务层数据包) 的格式发送数据, 为了降低操作难度, 作者根据不同 PCIe 事务的类型、TLP 结构, 结合设计逻辑对 PCIe 发送接口数据包进行了简化, 设置成指令集的形式, 结合 SDR 指令选择的不同模式, 用户在发送时在对应的位中配进数据即可。指令集其中主要分为配置请求、32 位 memory/IO 请求、64 位 memory 请求、消息请求、完成请求五类。其数据结构如图 5-4 至 5-8 所示。

1) 配置请求

适配 Type0 与 Type1 型的配置读、写 TLP。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SDR0	tag[7:0]								First BE[3:0]				R								addr[11:0]											
SDR1	data[31:0]																															

图 5-4 配置请求测试指令集

2) 32 bit Memory/IO 请求

适配 32 bit memory 与 IO 请求的读、写 TLP。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																				
SDR0	tag[7:0]								First BE[3:0]				Last BE[3:0]				TC[2:0]		R			addr[9:0]																														
SDR1	addr[31:0]																																																			
SDR2	data[31:0]																																																			

图 5-5 32 bit Memory/IO 请求测试指令集

3) 64 bit Memory 请求

适配 64 bit memory 请求的读、写 TLP。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																				
SDR0	tag[7:0]								First BE[3:0]				Last BE[3:0]				TC[2:0]		R			addr[9:0]																														
SDR1	addr[63:32]																																																			
SDR2	addr[31:0]																																																			
SDR3	data[31:0]																																																			

图 5-6 64 bit memory 请求测试指令集

4) 完成请求

适配 PCIe 完成 TLP。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SDR0	tag[7:0]								R								TC[2:0]		compl_sta		length[9:0]											
SDR1	lower addr[6:0]								R																byte_count[11:0]							
SDR2	data[31:0]																															

图 5-7 完成请求测试指令集

5) 消息请求

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SDR0	tag[7:0]								message code[7:0]								TC[2:0]			message rt			length[9:0]									
SDR1	R																															
SDR2	R																															

图 5-8 消息请求测试指令集

5.2.2 EP 模块

5.2.2.1 模块概述

本文根据搭建了一款基于千万门级 FPGA 芯片的外挂 DDR3 颗粒的读写访问 Endpoint 模型，其主要结构如下图 5-9 所示。

Endpoint 模块由 PCI Express Core、PCI Express to DDR3_Ctrl、DDR3_Soft_IP 和 FPGA 外挂的 DDR3 颗粒组成。PCI Express Core 是本文主要研究的对象，集成在 FPGA 芯片中，可以通过软件配置 PCI Express Core 形成不同链路宽度、传输带宽的模式，形成具体功能；PCI Express to DDR3_Ctrl 的功能是对 PCI Express Core 和 DDR3_Soft_IP 进行协议包转换、跨时钟域处理；DDR3_Soft_IP 存储器接口发

生器是 DDR3 的逻辑 IP，负责将接受的数据转换成可以和 DDR3 颗粒进行读写操作的格式，我们在工程中调用了 Xilinx 公司给出的整体的解决方案。

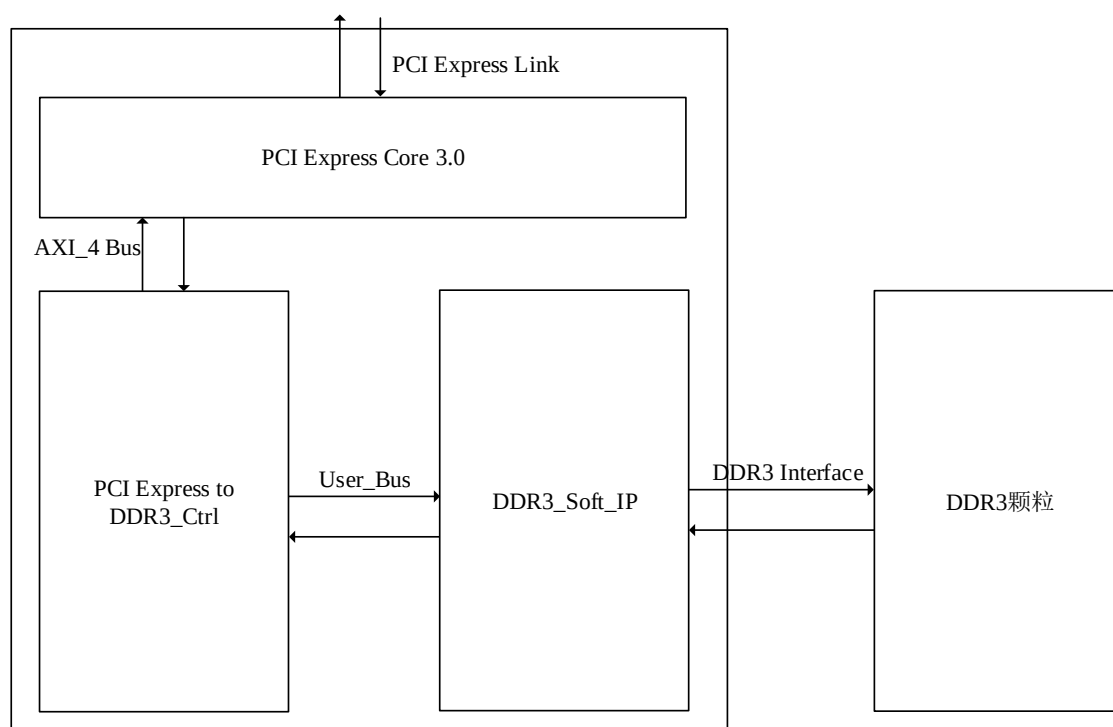


图 5-9 EP 模块结构图

5.2.2.2 PCI Express to DDR_3_Ctrl 模块的设计

PCI Express to DDR_3_Ctrl 模块是本次工程的主要组成部分，图 5-10 给出了其内部的结构。由 PCI Express 事务层解包而来的数据包经过 RX 状态机，CQ_RX_DESC0、CQ_RX_DESC1、CQ_RX_data 为 RX 状态机的三个状态，具体行为分析：当 RST_N 为低，表示系统复位，一直处于初始状态 CQ_RX_DESC0，等待 RST_N 信号拉起；当 EP 端设备接收到请求事务，系统根据接收包的类型分别进入 CQ_RX_DESC1、CQ_RX_data 状态，CQ_RX_DESC1 为指令输入，CQ_RX_data 为数据输入；OPER 状态机是根据状态机规则，将 AXI 格式的数据指令包处理为 DDR 格式的数据指令格式；EP 端发送过来的数据经过处理后，根据 DDR3_UI_GEN 模块发送至 DDR3_soft_IP 模块；由 DDR3_soft_IP 回读的模块的数据都是读操作的指令，读地址由发送的事务解析后发送，Read_cnt_ctrl 模块负责计算读回的数据量，回读的数据由 Rdata_fifo 存储后发送到 trans 状态机，trans 状态机完成回读 DDR3 格式数据转换为 AXI 格式的数据，再传递给 EP 设备，完成

后续完成事务包的打包发送。

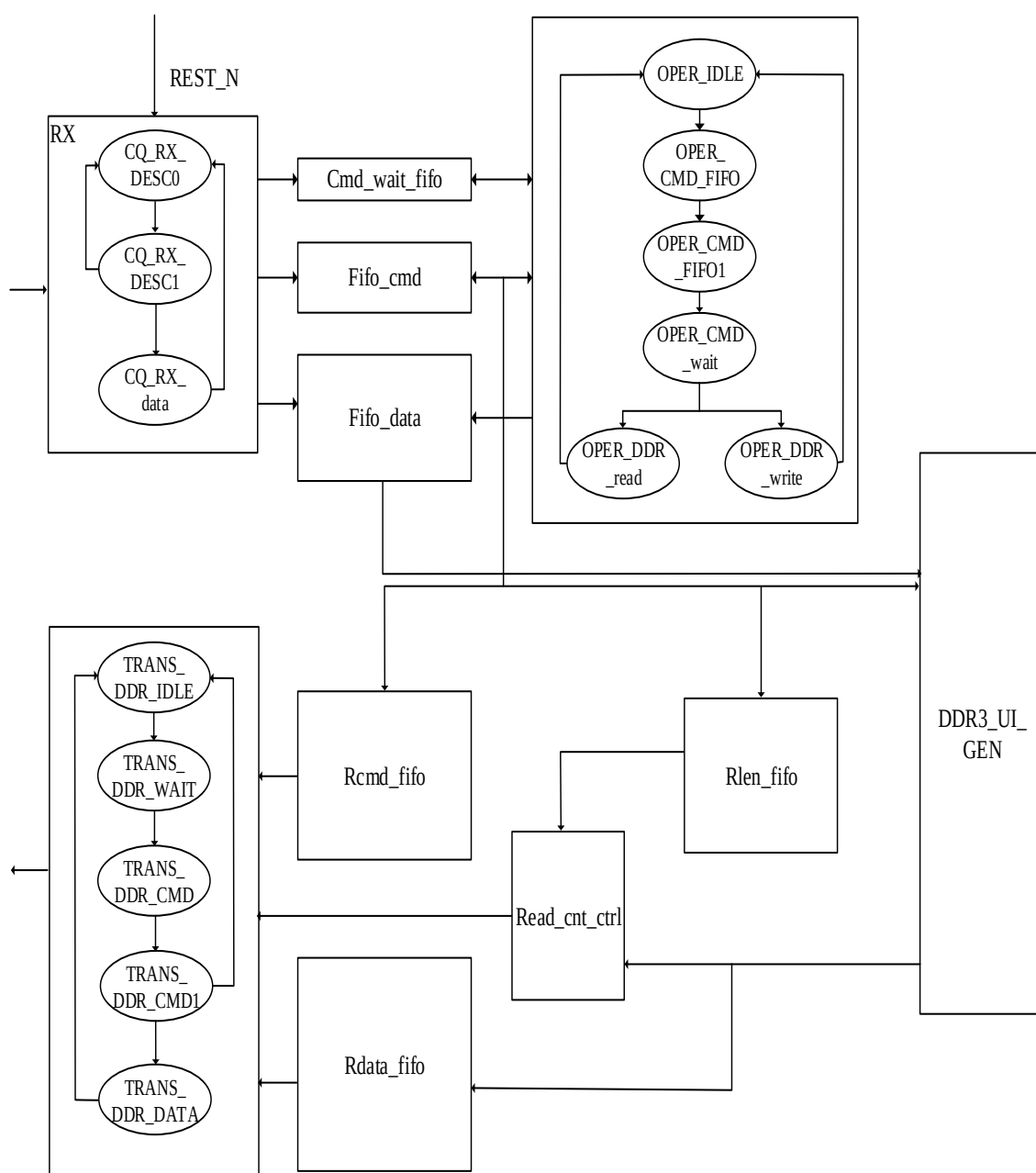


图 5-10 PCI Express to DDR3_Ctrl 的状态机示意图

5.2.3 RP 模块的设计

工程的上游设备 RP 端由 Transceiver 模块、PCI Express Core 模块组成，其中 Transceiver 模块中包括了 Tx_port 和 Rx_port。Tx_port 模块接收来自上游的数据处理后向 PCI Express 模块发送 TLP，然后再经由 PCI Express 链路传送至 EP 端。对端设备 EP 模块器件发送的 TLP 经过 PCI Express 链路到达 RP 的 PCI Express 模块，

然后再传送至 Rx_port 模块。Tx_port 和 Rx_port 通过 PCI Express Core 模块来共享各种功能，例如 TLP 处理和日志文件输出。Tx_port 模块发出事务序列或测试程序，以驱动端点器件的架构接口。Rx_port 模块接收来自端点器件的 TLP 响应。Tx_port 与 Rx_port 模块间的可通信性允许 Tx_port 模块在 Rx_port 模块收到来自端点器件的 TLP 时，验证正确的行为并相应地作出响应。

5.2.3.1 Tx_port 模块设计

Tx_port 模块接收端需要接收来自外部用户输入的 JTAG 信号以及芯片的全局时钟信号和复位信号，然后转换成 PCI Express Core 支持的包的格式发送到上游设备的 PCI Express Core 中，设计框图及端口列表如图 5-11 所示，表 5-1 是 Tx_port 端口列表。

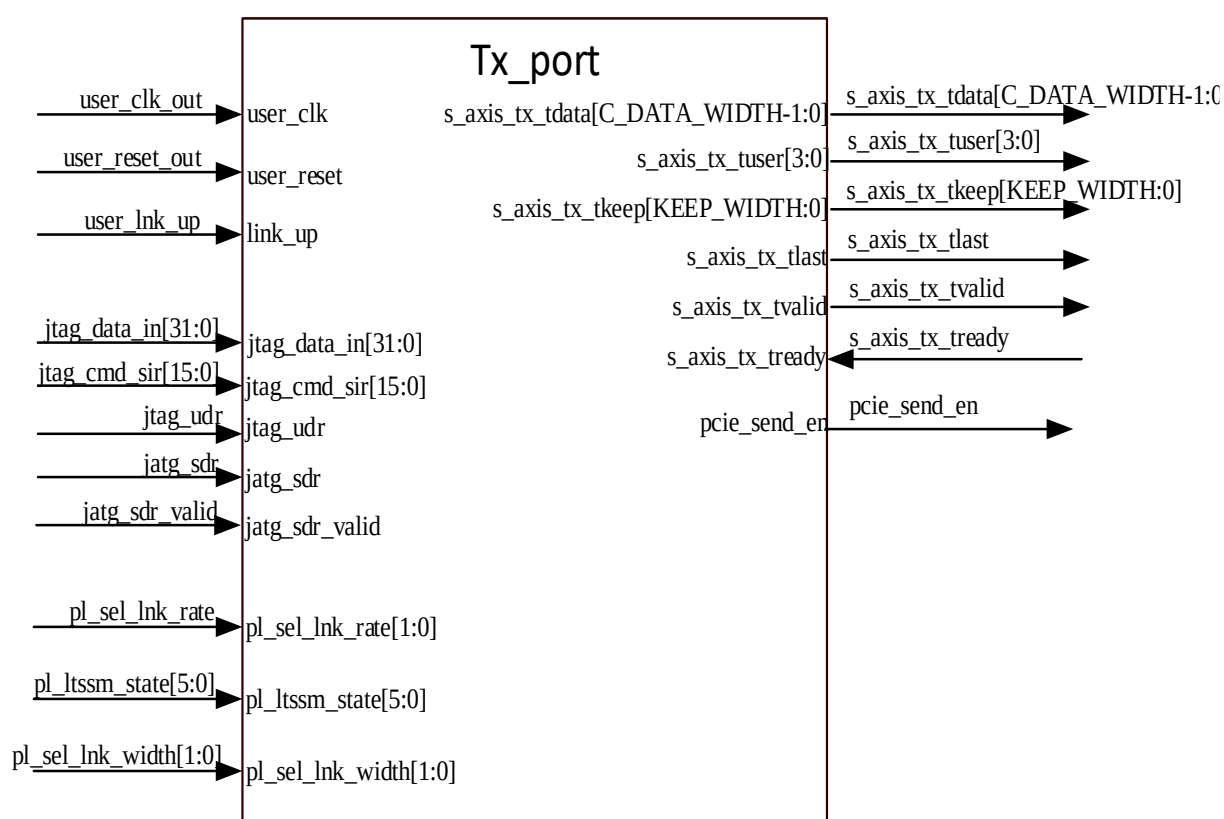


图 5-11 Tx_port 端口

5.2.3.2 Rx_port 模块设计

Rx_port 模块主要是接口来自 PCI Express 核的事务响应数据，接收的数据格式是 AXI；Rx_port 模块需要对传递过来的响应数据进行转换，由 AXI 格式转换为 JTAG 格式，然后将数据放到相应的存储 RAM 中，等到系统提取数据，进行校验；Rx_port 模块的设计框图如图 5-12 所示，端口列表如表 5-2 所示。

表 5-1 Tx_port 端口列表

端口名称	位宽	方向	源	描述
user_clk	1	input	rp 核	用户时钟
user_reset	1	input	rp 核	复位信号
user_link_up	1	input	rp 核	链路使能信号
pl_sel_lnk_rate	2	input	rp 核	当前链路速度： 00: 2.5Gb/s 01: 5.0Gb/s 10: 8.0Gb/s
pl_ltssm_state	6	input	rp 核	当前链路训练状态
pl_sel_lnk_width	2	input	rp 核	当前链路的宽度： 00: 1 通道； 01: 2 通道； 10: 4 通道； 11: 8 通道。
s_axis_tx_tdata	C_DATA_WIDTH	output	-	AXI4 接口信号
s_axis_tx_tuser	4	output	-	AXI4 接口信号
s_axis_tx_tlast	1	output	-	AXI4 接口信号
s_axis_tx_tkeep	KEEP_WIDTH	output	-	AXI4 接口信号
s_axis_tx_tvalid	1	output	-	AXI4 接口信号
s_axis_tx_tready	1	input	rp 核	AXI4 接口信号
jtag_cmd_sir	16	input	jtag_ctrl	JTAG 控制信号
jtag_data_in	32	input	jtag_ctrl	JTAG 控制信号
jtag_udr	1	input	jtag_ctrl	JTAG 控制信号
jatg_sdr	1	input	jtag_ctrl	JTAG 控制信号
jatg_sdr_valid	1	input	jtag_ctrl	JTAG 控制信号
pcie_send_en	1	output	-	TLP 发送使能

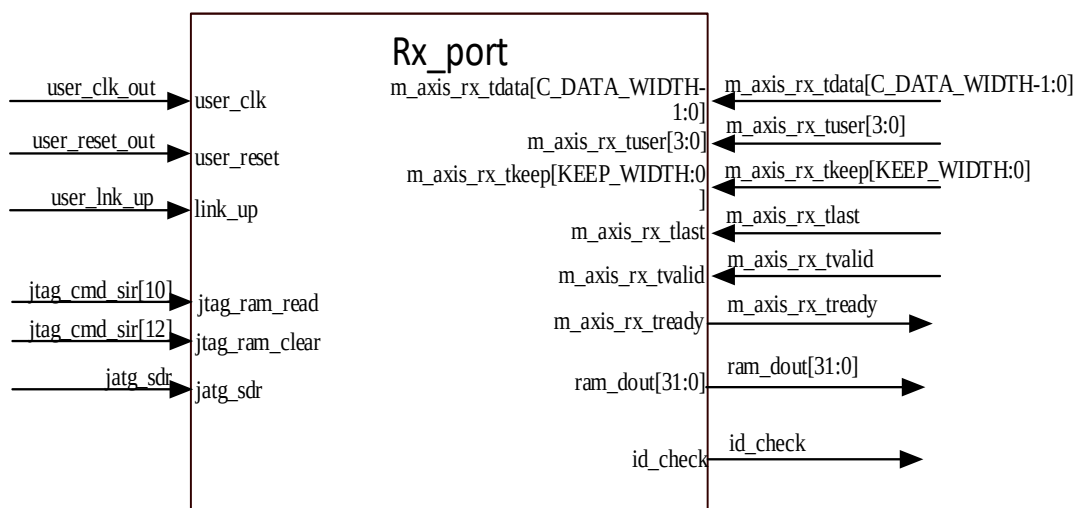


图 5-12 Rx_port 端口

表 5-2 Rx_port 端口列表

端口名称	位宽	方向	源	描述
user_clk	1	input	rp 核	用户时钟
user_reset	1	input	rp 核	复位信号
user_link_up	1	input	rp 核	链路使能信号
m_axis_rx_tdata	C_DATA_WIDTH	input	rp 核	AXI4 接口信号
m_axis_rx_tuser	4	input	rp 核	AXI4 接口信号
m_axis_rx_tlast	1	input	rp 核	AXI4 接口信号
m_axis_rx_tkeep	KEEP_WIDTH	input	rp 核	AXI4 接口信号
m_axis_rx_tvalid	1	input	rp 核	AXI4 接口信号
m_axis_rx_tready	1	output	-	AXI4 接口信号
jtag_ram_read	1	input	jtag_ctrl	JTAG 控制信号
jtag_ram_clear	1	input	jtag_ctrl	JTAG 控制信号
jtag_sdr	1	input	jtag_ctrl	JTAG 控制信号
ram_dout	32	output		读 ram 中的数据
id_check	1	output		设备 ID 检查

5.3 应用测试流程简介

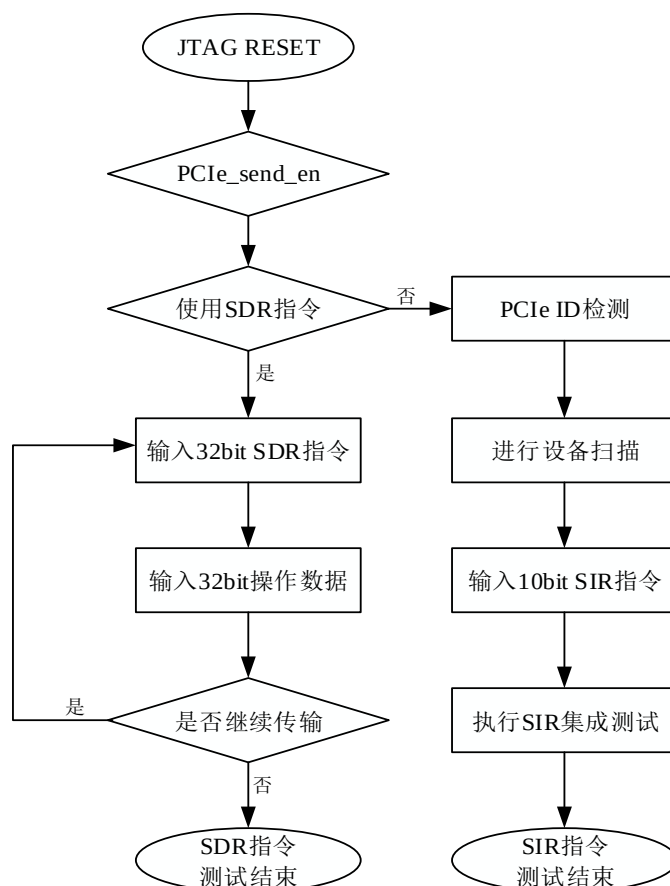


图 5-13 平台测试流程图

图 5-13 是本次工程的运行流程。当系统上电，首先进行系统的复位，包括 JTAG 模块的复位和 FPGA 的复位，当设备复位完成，JTAG 信号复位释放，等待 FPGA 系统复位完成，其标志是 PCIE 核发送传输准备完成信号，即 PCie_send_en 信号拉高。

为了保证测试的准确性，本次工程设计了两种测试指令下发模式。当 SIR 模式启用时，使用 10 bit SIR 寄存器传输指令的集成测试模式，该模式将所要执行的操作以指令集的形式直接在代码中体现，其操作步骤已经固定，输入指令仅作为执行相应测试的开关。当 SDR 模式开启时，使用 32 bit SDR 寄存器传输指令及数据的动态测试模式，针对 PCIE 的每一项操作都严格根据输入执行。可由人工配备成不同顺序进行测试。

5.4 应用工程的建立

5.4.1 约束策略

时序约束

工程可运行的最高系统时钟频率为 250Mhz，所以将系统时钟 sys_clk 的周期约束为 4ns（250Mhz），将 JTAG 时钟 tck 的周期约束为 166.66ns（6Mhz）。

物理约束

工程中调用两个 PCIe IP 核，跟上片上资源和封装对 IP 核进行位置约束。作为 Endpoint 的 PCIe 核约束为 PCIE_X1Y2；作为 Root Port 的 PCIe 核约束为 PCIE_X1Y1。

IO 约束

根据开发板资源分布和 FPGA 引脚原理图对 I/O 进行约束，I/O 标准为 LVCMOS18。

5.4.2 布局布线结果

根据前文的工程创建方案和约束策略，如下图所示，我们选择了 High 等级的自动布局布线，在这种情况下布线工具会根据工程中的资源调用情况、FPGA 芯片的资源分布情况进行综合优化，给出最优布线方案。布线结果如图 5-14 所示。

Property Name	Value
Place And Route Mode	Route Only
Place & Route Effort Level (Overall)	High
Extra Effort (Highest PAR level only)	None
Ignore User Timing Constraints	☐
Timing Mode	Performance Evaluation
Generate Asynchronous Delay Report	☐
Generate Clock Region Report	☐
Generate Post-Place & Route Simulation Model	☐
Generate Post-Place & Route Power Report	☐
Power Reduction	☐
Power Activity File	
Enable Multi-Threading	Off
Other Place & Route Command Line Options	

图 5-14 布线策略

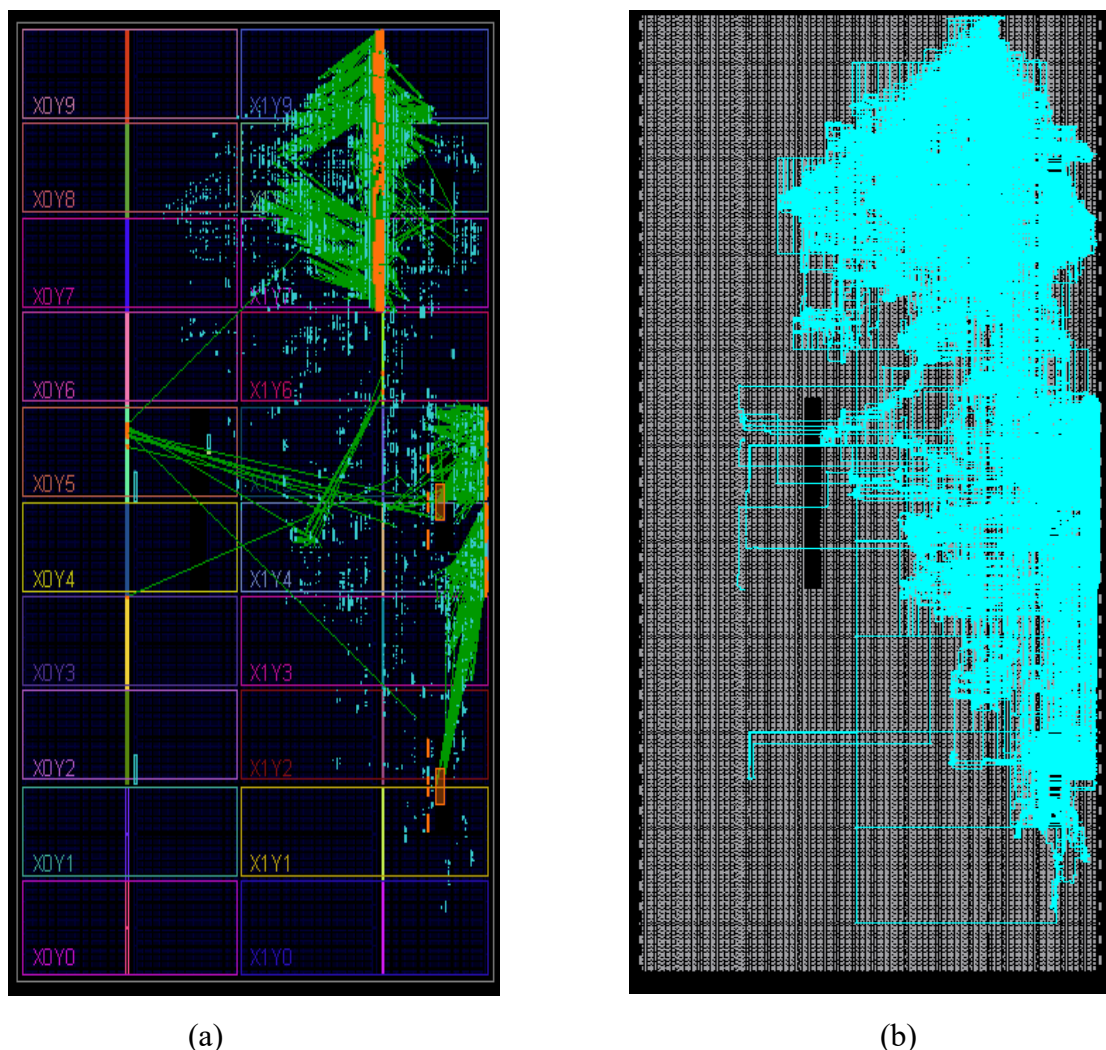


图 5-15 工程资源使用及布局布线图。(a) 工程资源使用图；(b) 工程布局布线图

图 5-15 (a)为本次工程调用的 PCI Express Core 的资源使用情况，橘红色部分为调用的片内资源；图 5-15 (b)为本次工程的 FPGA 芯片片内布局布线情况，可以看到软件的布局布线算法优先使用的调用资源附近的布线资源。

图 5-16 是本次工程使用片内资源情况的报告。本次工程我们一共使用了 FPGA 芯片中 8564 个 LUT 资源，352 个分布式 RAM 资源，8412 个触发器资源，32Kb 容量的 BRAM14 个。

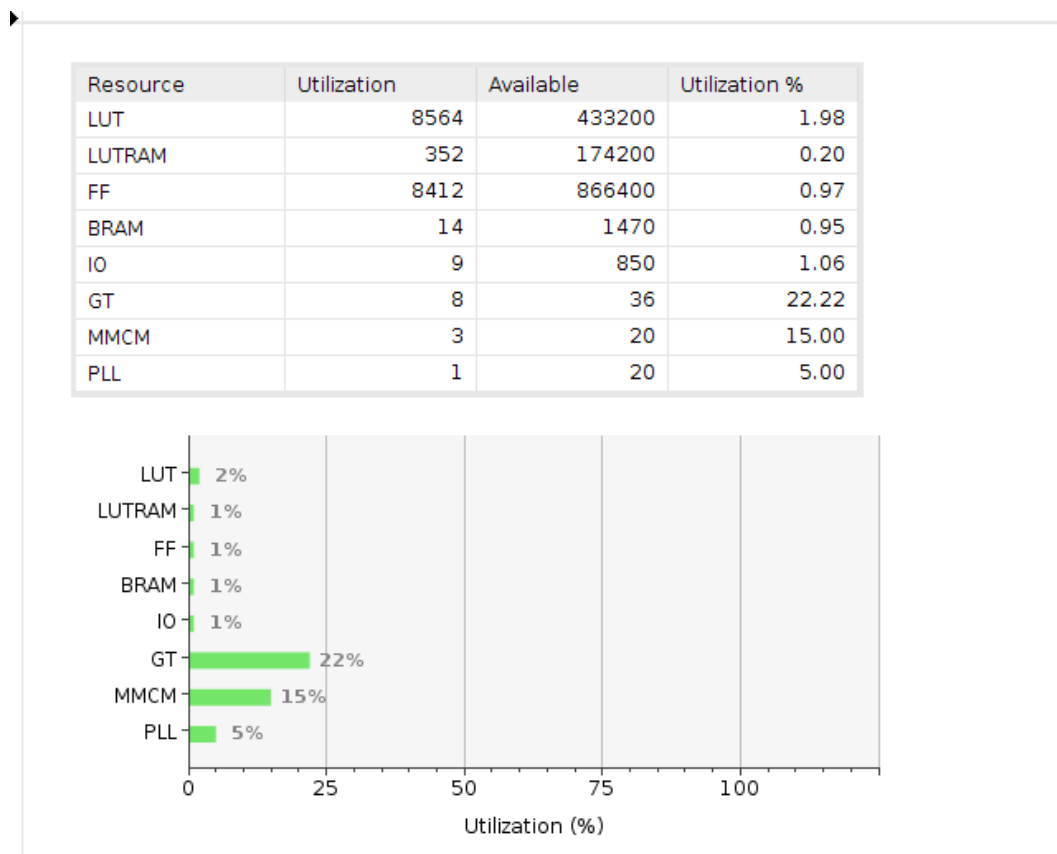


图 5-16 工程资源使用情况

5.4.3 时序报告和功耗分析

软件工具会在综合实现后给出工程相关的时序分析和功耗分析概况。图 5-17 是布线后软件生成的静态时序分析报告，布线后系统时钟最小周期可达 3.070ns (325.732MHz)，可满足对最大系统时钟 250Mhz 的 IP 测试要求，实际时序满足时序约束要求；图 5-18 是系统各个模块的功耗分析情况，可以看到由于布局布线资源复杂，布线资源的功耗占动态功耗比例最大。

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3.049 ns	Worst Hold Slack (WHS): 0.021 ns	Worst Pulse Width Slack (WPWS): 0.064 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 3535	Total Number of Endpoints: 3535	Total Number of Endpoints: 1217

All user specified timing constraints are met.

图 5-17 时序报告概述

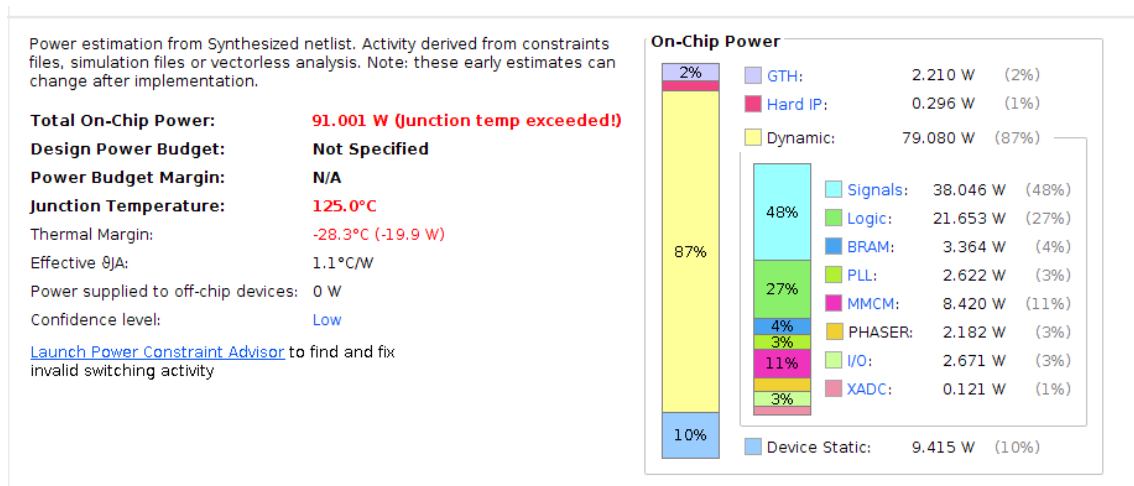


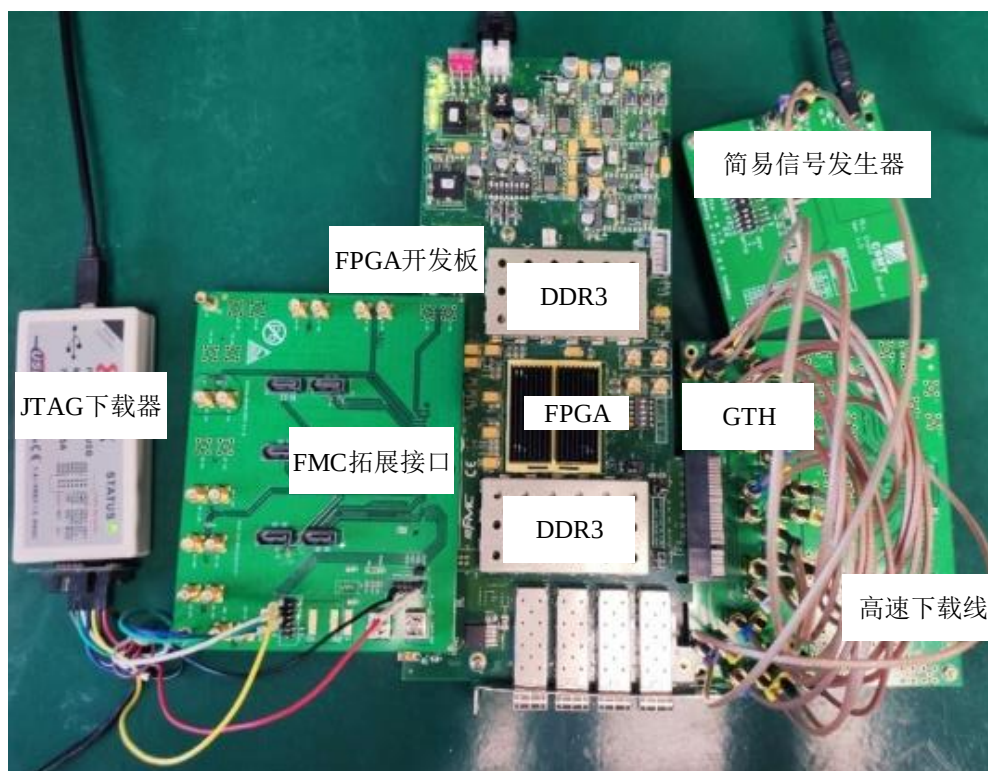
图 5-18 功耗分析报告

5.5 应用平台的搭建和测试

基于 PCI Express 的千万门级 FPGA 中 DDR3 读写访问测试应用平台的硬件测试环境如图 5-19 所示。包含 Xilinx 公司的信号下载器，用来连接主机进行 JTAG 命令的下发；FMC 拓展接口是基于开发板的 FMC 接口进行了开发资源的拓展，JTAG 下载器的输入输出通过左侧的 FMC 拓展接口和平台连接；FPGA 开发板是实习公司自制的高性能应用测试平台级开发板，其中包括电源模块、核心的 FPGA 芯片为本文所研究和核心芯片，其中集成了 PCI Express3.0 串行接口，同时开发板内嵌了上下两组共 8 颗 DDR3 颗粒，支持本章研究的 DDR3 应用工程；同时平台搭载了简易的信号发送器模块，可以发送 100MHz、125MHz、250MHz 的时钟信号；右侧的 FMC 拓展接口通过在 SERDES 端口上连接的高速差分信号线连接了本次工程的两个 PCI Express Core；泰克 DPO72304SX ATI 高性能示波器用来采集 PCIE 接口的波形信息质量。

5.5.1 功能应用

本次应用工程输入 FPGA 芯片的基础冒烟用例，实现的功能集成了第四章中的验证激励，应用工程平台的结构和代码也大部分集成了验证平台中的组分；根据之前的工作，我们配置好了相关功能的配置文件、同时搭建了适合本次应用工程的测试平台，功能应用阶段的主要工作是配置点的下发，使得 FPGA 芯片在开发板电路中形成具体功能，之后向形成功能的电路中下发激励，采集反馈信息，校验比对。



(a)



(b)

图 5-19 应用测试环境。(a) 测试平台中的 FPGA 开发板；(b) 测试平台使用的示波器

应用测试的行为主要是通过上游设备发起读写事务包，完成相应的设备操作；由于本次应用测试平台的搭建使用的是 JTAG 接口，我们的测试激励的发送采集和接收都是在 SVF 文件中预先定义。实习公司设计的 JTAG 回读软件可以识别 SVF 文件，对指令进行下发，对采集的数据进行校验。

图 5-20 给出了部分功能的 SVF 指令代码，其中 29 行至 42 行是 JTAG 接口的系统初始化过程，34 行给出了本次测试的频率是 100MHz，在完成系统初始化后，

首先需要获取系统的设备 ID_CODE，其实现方式是往 JTAG 内部写入一帧 32 位 SDR 指令 0000_0000h，JTAG 中的移位寄存器单元会将系统中默认的设备 ID_CODE 按位读出，通过 TDI 端口输出，软件检测通过后继续执行后续的指令；44 行是发送的第一帧测试指令集，SIR 为指令集类型，10 为指令数据位宽，TDI 为指令输入端口，3f1h 为指令内容；通过查阅表附-1，可知这帧指令 PCIE_ID_TEST0 测试向量，完成了 PCI Express Core 默认的设备 ID 检测功能；指令间的 RUNTEST 指令是不同操作间的系统等待时间。如 45 行，TCK 是时钟输入端口，500000 表示等待五十万个时钟周期；第 53 行是系统发送的 SDR 指令，JTAG 的行为完全有指令内容描述，这一帧指令是通过 TDI 端口向 JTAG 端口写入 ffff_ffffh，并且给出了预期的 TDO 输出内容，MASK 中的 ffff_ffffh 表示预期 TDO 的 32 位输出和实际输出进行全部比较。如图 5-21 所示，表示读写操作测试通过。

```
TRST OFF;
ENDIR IDLE;
ENDDR IDLE;
STATE IDLE;
FREQUENCY 1E6 HZ;
TIR 0;
HIR 0;
TDR 0;
HDR 0;
TIR 0;
HIR 0;
HDR 0;
TDR 0;
SDR 32 TDI(00000000);
SIR 10 TDI(3F0);
RUNTEST 500000 TCK;
SIR 10 TDI(3F1);
RUNTEST 10000000 TCK;
SIR 10 TDI(3F5);
RUNTEST 500000 TCK;
SIR 10 TDI(3D2);
SDR 32 TDI(FFFFFFFF) TDO(00130000) MASK(FFFFFFFF)
```

图 5-20 SVF 示例代码

根据工程需要在现阶段不用考虑测试资源的功能覆盖率问题，所以本文我们只对部分功能进行了测试，包括对 PCI Express 的读写操作、字节使能操作、PCI

Express 设备的 ID 检测、极性反置功能等。

5.5.2 不同模式下的传输质量分析

传输质量的分析主要是对 FPGA 芯片的数据传递过程中, 对高速串行接口传输信号质量进行采样、分析。测试工作依靠上文搭建的基于 DDR3 的读写访问模型, 通过使用带宽示波器和测试输出平台, 来完成传输通道信号质量的数据收集。

5.5.2.1 传输质量信号采集分析平台

我们使用了图 5-19(b)中的泰克 DPO72304SX ATI 高性能示波器进行 PCIE 接口的传输质量测试, 该示波器带宽达到 23 GHz, 性能符合项目要求。我们在软件中设置好下载到 FPGA 芯片中的数据类型、下载速率; 程控直流电源用来控制输入的电电压, 是系统的供电模块; 板级测试版如图 5-19 所示, 其中挂载了本文研究的 FPGA 芯片; 宽带示波器是信号传输质量的结果输出端, 我们通过宽带示波器来获得通道信号的各方面性能指标。

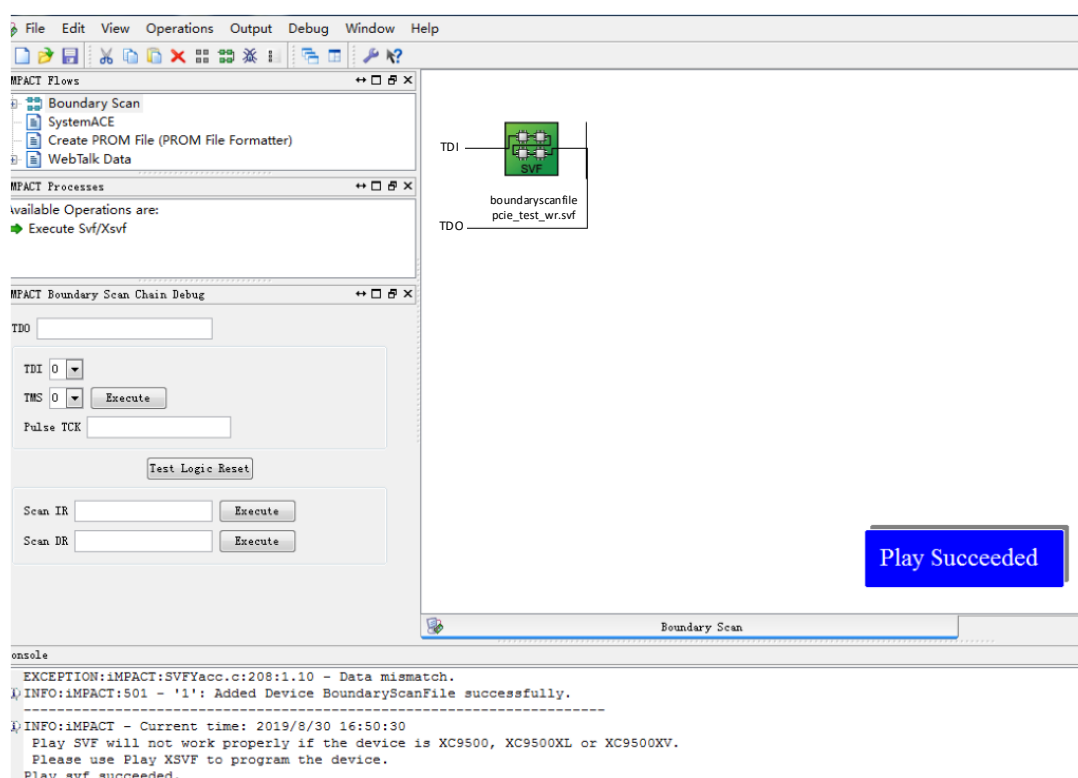


图 5-21 读写操作成功界面

由于 FPGA 芯片搭载的 PCI Express IP 核支持 PCIE3.0 传输协议, 其支持的传输模式向下兼容之前的版本, 故本次传输信号质量的测试我们分别对系统在 2.5 GT/s、5 GT/s、8 GT/s 传输速率下的进行测试。由于设备的限制, 本次测试我们只

测试单通道模式下的信号传输质量。根据 PCI Express 协议规范，单通道模式下差分信号的摆幅应该在 800 mV 至 1200 mV 之间，我们的测试环境设置信号摆幅为 1000 mV。

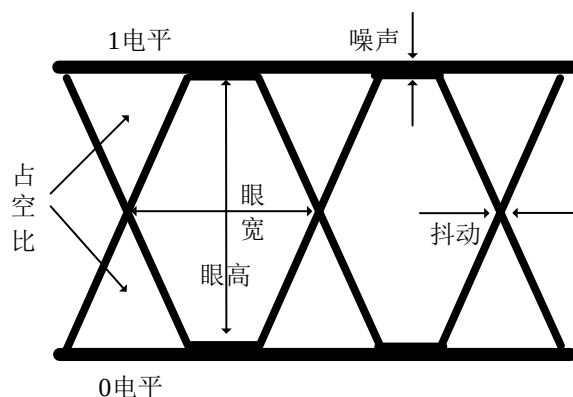


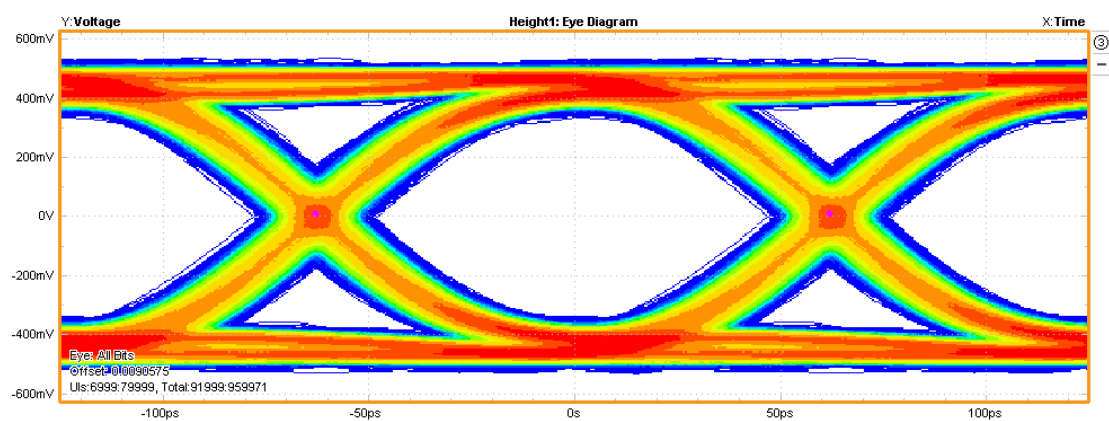
图 5-22 眼图的参数模型

5.5.2.2 眼图分析

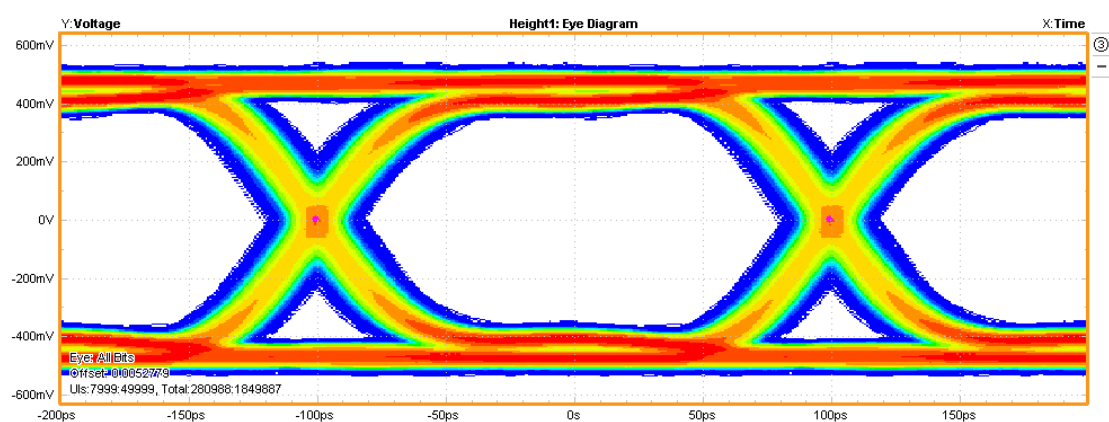
眼图分析是通信系统信号质量分析的重要手段。本文中使用了眼图分析的方法对 FPGA 芯片串行接口的信息传输质量。眼图本质上是对数字信号以时钟为基准在示波器上进行叠加产生最终的统计分布情况，以此反应出信号的整体质量情况。相较于传统观测方法使用示波器观测实时波形，我们只能分析出在示波器显示的那一帧/段波形的质量情况，而实际的 PCI Express 通路上的信号传输都不是瞬时的，所以实时波形的观测并不能充分反映信号整体质量；眼图分析通过在整体数据传输的所有信号进行采样输出，能够直观的显示通道信号的传输质量。

对于眼图的评价，通常有一些基础的可视衡量标准。如图 5-23 所示，眼宽和眼高一般可以反映系统的噪声 (Voltage Noise)、抖动 (Jitter)。当纵轴的眼高变矮，说明系统的噪声过大；当横轴的眼宽变窄，说明系统的抖动增大；所以，直接观察眼图时，眼的大小可以直观的描述信号的质量；两个眼之间的空白部分是数字信号的占空比，一般来说，信号的上下沿的占空比面积应该对称，若果占空比面积不对称，表明系统的存在信号失真部分。

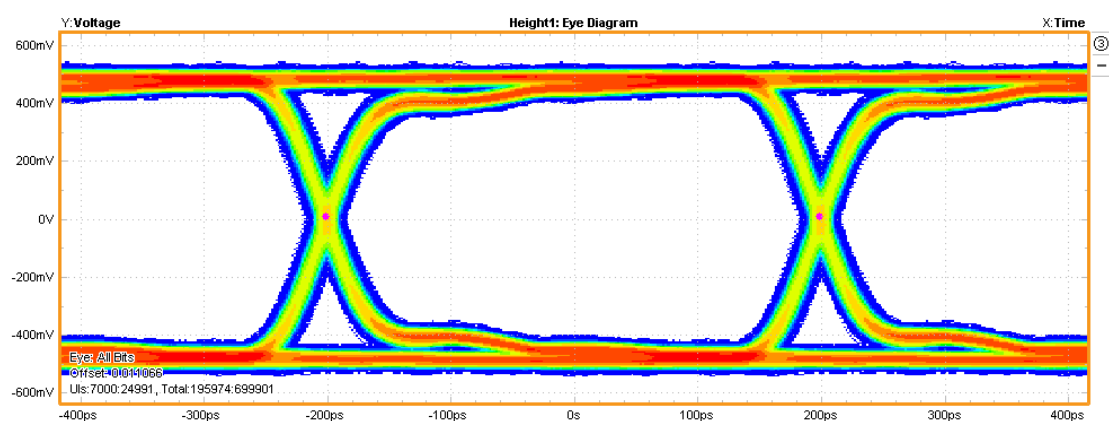
同时，眼图张开的幅度能反映信号的码间串扰问题。当眼图的纵向曲线斜率很大，这时候眼图的张开幅度大，信号的建立时间短，码间串扰较小；而反之，当纵向曲线斜率变小，眼图张开幅度变缓，说明信号的建立时间长，码间串扰变大。



(a)



(b)



(c)

图 5-23 不同模式下的眼图。(a) 8 GT/s 模式下的眼图；(b) 5 GT/s 模式下的眼图；(c) 2.5 GT/s 模式下的眼图

我们通过采样得到了 8GT/s、5GT/s、2.5GT/s 模式下的信号传输质量眼图，如图 5-24 所示。

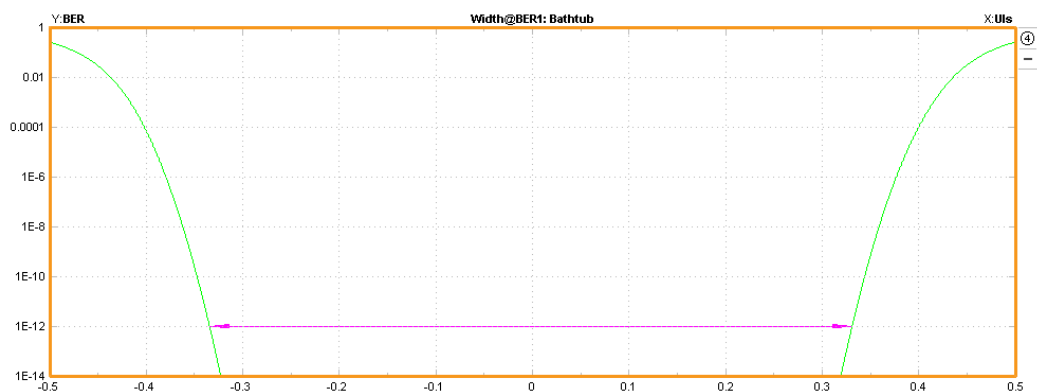
在 2.5GT/s 模式下，眼图的眼面积最大，说明 2.5G/s 模式下噪声和抖动最小；

眼图的占空比面积对称, 2.5GT/s 模式下张开幅度最大, 说明在 2.5GT/s 模式下码间串扰最小; 而随着信号传输速率增高, 眼图的张开幅度变小, 眼宽和眼高减小, 这是因为随着传输速率的提升, 由于工艺的限制, 信号的建立保持时间固定, 周期变小, 导致码间串扰增大, 噪声增加, 抖动加剧。

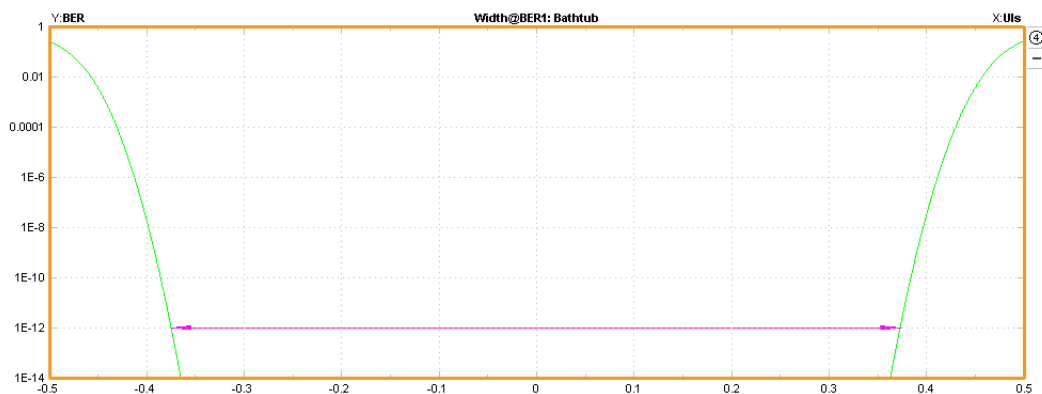
5.5.2.3 误码率分析

通过前文的眼图测试, 我们可以统计出系统的误码率 (bit Error Ratio, BER) 数据。误码率是指在一段时间内, 系统所发送的比特数据量总和与发生错误的比特数据的比值, 是通信系统中的重要参数。系统中的抖动、噪声、信道损耗等都可以影响误码率。在工程实践中, 通常统计系统误码率在 10^{-12} 情况下眼图的宽度, 眼图的宽度单位是 UI, 一个眼图的宽度为一个 UI; 在 BER 一定的情况下, 两边曲线直接的距离为眼图的张开程度; 一般情况, 误码率越低, 眼张开越小。

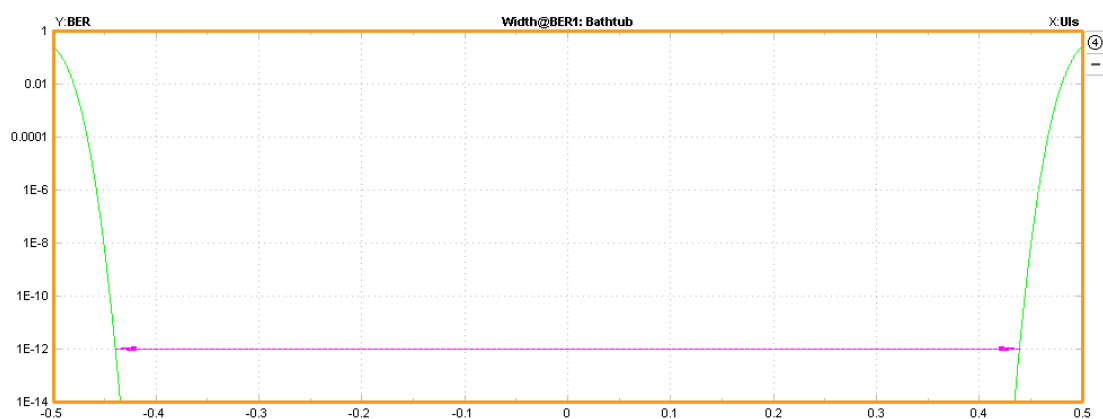
图 5-25 是 8 GT/s、5 GT/s、2.5 GT/s 模式下系统的误码率统计图。可以看到, 在 8 GT/s 模式下, 10^{-12} 误码率下的眼图宽度大于 0.6 个 UI, 随着传输速率降低, 10^{-12} 误码率下的眼图宽度越来越大, 2.5 GT/s 模式下眼图宽度大于 0.8 UI, 符合设计要求。



(a)



(b)



(c)

图 5-24 不同模式下的误码率分析图；(a) 8 GT/s 模式下的误码率分析图；(b) 5 GT/s 模式下的误码率分析图；(c) 2.5 GT/s 模式下的误码率分析图

5.6 本章小结

本章完成了 FPGA 芯片流片后的应用测试的工作。作者利用高性能通用 FPGA 测试开发板，搭建了基于 PCI Express 接口的千万门 FPGA 芯片对 DDR3 颗粒的读写访问应用工程，工程的搭建和激励的产生均继承了第四章验证仿真平台的部分工作内容，有效提高了工作效率。最终，本项测试顺利完成了预期的功能；同时，我们利用应用测试平台采集了 PCI Express 接口的传输信号的质量分析眼图和误码率分析图，对信号传输质量进行了有效分析，信号传输质量满足设计要求。

第六章 总结与展望

6.1 文章总结

本文通过对高性能 FPGA 芯片的高速串行接口的研究，在项目中完成了基于 28nm 工艺的千万门级 FPGA 芯片中 PCI Express 模块的集成、验证工作，并且在芯片流片后结合开发软件工具对芯片进行了整体应用测试，通过实践证明了此项技术研究的先进性和可靠性。

本文所作的主要工作有：

(1) 查阅了相关技术资料，分析了高性能可编程器件的发展、应用等；同时分析、总结了 PCI Express 串行总线的各项技术参数，包括 PCI Express 的经典拓扑结构、PCI Express 的分层架构、PCI Express 内部的通信机制和 PCI Express 的协议数据包，以及一些 PCI Express 规范中规定的其他技术规范。这部分工作是本文的理论依据。

(2) 在调研理论知识的基础上，完成了一款基于 28nm 工艺的千万门级高性能 sram 型 FPGA 芯片设计项目中 PCI Express 模块的集成的工作。PCI Express Core 作为定制 IP 进行集成。文章对 FPGA 芯片的设计方案、架构设计进行了研究，同时分析了 PCI Express Core 相关的片内资源，给出了 PCI Express Core 的集成技术路线。本文完成了定制 IP 的直接验证工作。由于 PCI Express 是定制的 IP，本文采用了灰盒验证的思想，搭建的直接验证的验证平台，对 PCI Express Core 的主要行为功能进行仿真验证，验证结果符合要求，功能覆盖率达到 100%，符合项目要求。

(3) 以上的工作集成设计和模块验证工作支撑了 28nm 工艺的千万门级高性能 sram 型 FPGA 芯片的流片工作。本文基于芯片流片后返回的样片，结合软件部门提供的配套软件，完成了一款包含 DDR3 终端设备的读写访问应用测试。作者首先在软件工具中创建了应用模型的工程，给出相关器件的物理约束、时序约束；搭建包含项目流片后的 FPGA 芯片应用模型平台，然后进行 FPGA 芯片的功能开发工作，即 bit_stream 的下发、功能创建、实行板级功能验证；此项工作通过全片测试的模型，验证了本文集成设计的 PCI Express Core 在 FPGA 芯片的实际工作中的可靠性；同时结合带宽示波器，得到了实际工作中 PCI Express 串行接口的信号传输质量分析数据；根据分析，本次集成的 PCI Express 接口的工作性能指标符合预期。

综上所述，本文完了 PCI Express 接口在高性能 FPGA 芯片中的集成设计、功能

验证和应用测试的全流程研究与实现。由于国内芯片厂商开发的高速接口的应用产品主要是基于上一代通信接口 PCI 接口或者是低版本的 PCI Express 接口，本次工作作为高性能 FPGA 芯片中 3.0 版本的 PCI Express 串行接口的集成开发提供了较为完善的思路和技术路线。

6.2 未来展望

本文的工作涉及 FPGA 芯片的 IP 集成、核心模块的模块级验证和应用测试。由于项目的目标是保证工程的全流程、高效率和可靠性，本文所涉及的研究方法是较为初级的研究方法，将来随着工程经验的增加和对工程器件认识的加深，作者将把研究方法的创新作为重要突破方向；另一方面，本文采用了定制的 PCI Express IP Core，其内部是基本门电路单元，由于项目的进度要求，并未对 IP 进行层次化建模，PCI Express 层次化的验证方案未实现；将来的工作中，如何高效完成对于定制 PCI Express IP Core 的层次化验证将是一大重要的研究方向。

文章中所完成的应用测试部分的工作仅仅完成了 FPGA 芯片中 PCI Express 接口的基本数据通道的测试，大量其他的专用功能的测试验证将在后期进行，其实现方式并未在文中详述。

本文实现的 PCI Express 接口应用工程是基于 PCI Express 规范架构中端对端传输基本模型，应用工程中链路设备只有上、下游两类设备，对于 PCI Express 规范中端口拓展设备的应用场景并未实现，未实现基于 PCI Express 规范地多设备应用场景开发；在未来的项目中，作者将着力于搭建多应用设备的应用环境，实现图 2-1 所示的完整的 PCI Express 传输架构。

致 谢

在硕士论文即将完成之际，首先要感谢我的导师王治国教授，带领我步入研究生学习的殿堂，引导我找到感兴趣的科研方向，教授我科学的科研工作方法。王老师渊博的学术修养、严谨的治学态度、平易近人的师者风范让我敬仰万分，令我受益匪浅，亦将在毕业后的学习工作生活中时刻激励我。

能够完成我的学业，我要感谢我的国家。祖国伟大富强、社会和谐稳定，我的国家赋予我学习权利，让我在和平的年代里遨游于浩瀚的学海二十余载，追求兴趣与爱好，锻炼专业技术能力。如今即将学成步入工作岗位，希望能为国家繁荣昌盛献上自己的智慧与力量。

特别感谢成都华微电子科技有限公司提供的实习实践机会。在成都华微的两年，我的专业技术水平不断提高；成功完成了第一款芯片设计验证工作并流片，亦顺利完成了毕业设计课题的相关工作。十分感谢我的企业导师谢洪波工程师，是他让我接触 IC 设计与验证这个广袤无垠、丰富多彩的技术领域；感谢公司领导丛伟林工程师在实习过程中提供了丰富的技术指导，生活上亦是关怀备至；同时感谢我的同学刘自祥、李明谦、庞文倩、叶磊、程俊明，成都华微的同事郭华伦、李建秋、冯萍、邓雯方、邢亚楠、湛伟、陶琼、何晓桐、王浩宇、胡路易等在论文写作过程中提供的帮助与指导。

感谢我的家人。感谢我的父母、奶奶，你们是最坚强的后盾；感谢我的未婚妻唐潇艺女士，陪伴我一路走来，相信我并支持我的每一个决定。

最后感谢电子科技大学，给了我研究生学习的机会，让我在三年的时间里不断进步，专业技术水平显著提高；即将走出校园，老师们的谆谆教诲我将铭记于心，激励我努力创造自己的事业。祝福母校蓬勃发展、再创辉煌、桃李满天下。

参考文献

- [1] L. Rota, M. Caselle, S. Chilingaryan, et al. A PCIe DMA Architecture for Multi-Gigabyte Per Second Data Transmission[J]. IEEE Transactions on Nuclear Science, 2015, 62(3): 972-976.
- [2] H. Kaviani-pour, S. Muschter and C. Bohm. High Performance FPGA-Based DMA Interface for PCIe[J]. 2012 18th IEEE-NPSS Real Time Conference, 2012: 1-3.
- [3] 李大琳. 基于 FPGA 的高性能算法实现的设计模式及其应用研究[D]. 长春:吉林大学, 2020.
- [4] 张荣生. SRAM 型 FPGA 故障注入及刷新技术研究[D]. 哈尔滨:哈尔滨工业大学, 2019.
- [5] A. Fernandes, R. C. Pereira, J. Sousa, et al. FPGA Remote Update for Nuclear Environments[J]. IEEE Transactions on Nuclear Science, 2016, 63(3): 1645-1649.
- [6] Y. Thoma, A. Dassatti, D. Molla, et al. FPGA-GPU Communicating through PCIe[J]. Microprocessors and Microsystems, 2015, 39(7): 565-575.
- [7] 雷玉飞. 基于 FPGA 的高速、高精度 FFT 处理方案研究与实现[D]. 西安:西安电子科技大学, 2019.
- [8] 陈君杰. 基于 FPGA 和 DSP 的高清全景视频采集与处理系统[D]. 长春:长春理工大学, 2019.
- [9] Y. Li, F. Zhang, J. Lv, et al. A 5Gb/s Multi-Mode Transmitter with De-Emphasis for PCI Express 2.0/USB 3.0[J]. Analog Integrated Circuits and Signal Processing, 2014, 81(2): 503-513.
- [10] R. Budruk, D. Anderson, T. Shanley. PCI Express 系统体系结构标准教材. PCI Express 系统体系结构标准教材, 北京:2005.
- [11] Z. Lang, T. Lu and Y. Zong. MAC Design and Verification Based on PCIe 2.0[C]. 中国香港, 2016, 5.
- [12] H. Jiang, S. Li, H. Li, et al. A High-Performance and Hardware-Efficient PCIe Transmission for a Multi-Channel Video Using Command Caching and Dynamic Splicing on FPGA[J]. Journal of Circuits Systems and Computers, 2018, 27(4).
- [13] 李鹏飞, 钱伟康. 基于 FPGA 的 PCIe 总线 DMA 传输研究[C]. 第十一届全国信号和智能信息处理与应用学术会议, 中国贵州贵阳, 2017, 4.
- [14] 郑维维. 基于 PCI Express 的多路存储系统的设计[D]. 北京:北京理工大学, 2016.
- [15] K. Chandra, A. P. Jagtap, N. Ranjan, et al. Design of PCIe-DMA Bridge Interface for High Speed Ethernet Applications[C]. 2019 Second International Conference on Advanced Computational and Communication Paradigms (ICACCP), 2019, 1-5.

- [16] Y. Cao, Y. Zhu, X. Wang, et al. An FPGA Based PCI-E Root Complex Architecture for Standalone Sopcs[C]. 2013 IEEE 21st Annual International Symposium on Field-Programmable Custom Computing Machines, 2013, 149-152.
- [17] 胡远明. 基于 X86 处理器的高可用 SMP 服务器的设计与实现[D]. 北京:中国科学院大学(中国科学院工程管理与信息技术学院), 2017.
- [18] 王志奇. 基于 PCIe 接口的固态硬盘控制器设计与实现[D]. 成都:电子科技大学, 2017.
- [19] 朱奇凡. 基于 PCI Express 的光纤通信卡研制[D]. 哈尔滨:哈尔滨工业大学, 2019.
- [20] 师凯. 可扩展网络化多根 PCI Express Switch 的研究与设计[D]. 长沙:国防科学技术大学, 2017.
- [21] J. Regula, M. Subramaniyan and J. Dodson. Integrating Rack Level Connectivity into a PCIe Switch[C]. 2013 IEEE Hot Chips 25 Symposium (HCS), 2013, 1-26.
- [22] 李全孟. 双向 PCI-PCIe 桥接电路的验证[D]. 西安:西安电子科技大学, 2018.
- [23] 许孟. 应用于 SOC 的 PCIe 与 fc 协议高速信息传输模块实现与验证[D]. 西安:西安电子科技大学, 2017.
- [24] 田静. 4Gbps FC 网络通信中 PCIe 接口研究[D]. 西安:西安电子科技大学, 2017.
- [25] 陈杨. 基于 PCIe 总线的高速数据采集系统设计与实现[D]. 杭州:浙江大学, 2018.
- [26] M. A. Khelif, J. Lorandel, O. Romain, et al. Toward a Hardware Man-in-the-Middle Attack on PCIe Bus for Smart Data Replay[C]. 2019 22nd Euromicro Conference on Digital System Design (DSD), 2019, 230-237.
- [27] 郑乾. 基于 PCI Express 总线的物理编码子层设计[D]. 长沙:湖南大学, 2014.
- [28] 崔冰. 应用于 PCI-Express 2.0 的双通道锁相环的研究与设计[D]. 厦门:华侨大学, 2017.
- [29] 陈路. 基于 PCIe Gen4 协议的高速串行链路信号完整性分析与优化设计技术研究[D]. 苏州:苏州大学, 2019.
- [30] 母芥滨. 基于 PCIe 总线的高速数据传输通道设计[D]. 成都:电子科技大学, 2020.
- [31] E. Barrera, M. Ruiz, D. Sanz, et al. Test Bed for Real-Time Image Acquisition and Processing Systems Based on Flexrio, Camera Link, and Epics[J]. Fusion Engineering and Design, 2014, 89(5): 633-637.
- [32] A. Byszuk, J. Kolodziejski, G. Kasprowicz, et al. Implementation of PCI Express Bus Communication for FPGA-Based Data Acquisition System[J]. Photonics Applications in Astronomy, Communications, Industry, and High-Energy Physics Experiments 2012, 2012, 8454.
- [33] 许鹏. 通信 SOC 芯片 PCIe 2.0 接口设计与验证[D]. 西安:西安电子科技大学, 2018.

- [34] R. Ammendola, A. Biagioni, O. Frezza, et al. ASIP acceleration for virtual-to-physical address translation on RDMA-enabled FPGA-based network interfaces[J]. *Future Generation Computer Systems-the International Journal of Escience*, 2015, 53: 109-118.
- [35] 张云. 基于 PCIe3.0 的视频传输系统的设计与实现[D]. 成都:电子科技大学, 2019.
- [36] 腾孝东. 基于 FPGA 的 PCI Express 高速数据交换技术研究[D]. 大连:大连海事大学, 2016.
- [37] 王帅. 基于 FPGA 的 PCIe 多路高速数据采集系统的设计[D]. 济南:济南大学, 2017.
- [38] 叶庭祥. PCI Express 2.0 物理层关键模块的设计与验证[D]. 南京:东南大学, 2016.
- [39] T. Qian, L. Chen, X. Li, et al. A 1.25gbps Programmable FPGA I/O Buffer with Multi-Standard Support[C]. 2018 IEEE 3rd International Conference on Integrated Circuits and Microsystems (ICICM), 2018, 362-365.
- [40] 于祥苓. 反熔丝 FPGA 器件抗辐照性能测试及方法研究[D]. 西安:西安电子科技大学, 2019.
- [41] 谢丁. FPGA 互连结构评估系统研究与实现[D]. 上海:复旦大学, 2011.
- [42] 柴磊. 基于 PCIe 总线的高速数据传输技术研究[D]. 西安:西安电子科技大学, 2018.
- [43] P. S. Mutha and Y. M. Vaidya. FPGA Reconfiguration Using Uart and Spi Flash[C]. 2017 International Conference on Trends in Electronics and Informatics (ICEI), 2017, 59-63.
- [44] Z. Qi, Q. Duan, L. Hu, et al. Timing Model for Grm FPGA Based Routing[C]. 2018 14th IEEE International Conference on Solid-State and Integrated Circuit Technology (ICSICT), 2018, 1-3.
- [45] 李金龙. PCIe 交换芯片 FPGA 原型验证平台设计[D]. 西安:西安电子科技大学, 2017.
- [46] 黄丹萍. 基于 PCIe 的高速数据采集模块的软件设计与实现[D]. 成都:电子科技大学, 2018.

附录

附 1 SDR 指令集设计表

表附-1 TLP 包的 SDR 指令说明

指令	数值	描述
TYPE0_CFG_READ	32'hffff100	Type 0 配置读请求使能
TYPE1_CFG_READ	32'hffff101	Type 1 配置读请求使能
TYPE0_CFG_WRITE	32'hffff102	Type 0 配置写请求使能
TYPE1_CFG_WRITE	32'hffff103	Type 1 配置写请求使能
MEMORY_READ_32	32'hffff104	Mem 32 读使能
MEMORY_READ_64	32'hffff105	Mem 64 读使能
MEMORY_WRITE_32	32'hffff106	Mem 32 写使能
MEMORY_WRITE_64	32'hffff107	Mem 64 写使能
COMPLETION_NODATA	32'hffff108	不带数据的完成包使能
COMPLETION_DATA	32'hffff109	带数据的完成包使能
MESSAGE_NODATA	32'hffff10A	消息请求使能
MESSAGE_DATA	32'hffff10B	带数据的消息请求使能
IO_READ	32'hffff10C	IO 读请求使能
IO_WRITE	32'hffff10D	IO 写请求使能
TYPE0_CFG_READ_DONE	32'hffff200	Type 0 配置读请求完成
TYPE1_CFG_READ_DONE	32'hffff201	Type 1 配置读请求完成
TYPE0_CFG_WRITE_DONE	32'hffff202	Type 0 配置写请求完成
TYPE1_CFG_WRITE_DONE	32'hffff203	Type 1 配置写请求完成
MEMORY_READ_32_DONE	32'hffff204	Mem 32 读完成
MEMORY_READ_64_DONE	32'hffff205	Mem 64 读完成
MEMORY_WRITE_32_DONE	32'hffff206	Mem 32 写完成
MEMORY_WRITE_64_DONE	32'hffff207	Mem 64 写完成
COMPLETION_NODATA_DONE	32'hffff208	不带数据的完成包请求完成
COMPLETION_DATA_DONE	32'hffff209	带数据的完成包请求完成
MESSAGE_NODATA_DONE	32'hffff20A	消息请求使能完成

表附-1 TLP 包的 SDR 指令说明（续）

指令	数值	描述
IO_READ_DONE	32'hffff20C	IO 读请求使能完成
MESSAGE_DATA_DONE	32'hffff20B	带数据的消息请求使能完成
IO_WRITE_DONE	32'hffff20D	IO 写请求使能完成
CFG_ID_ENABLE	32'hffff301	配置完成者 ID 使能
CFG_ID_DISABLE	32'hffff302	配置完成者 ID 结束
CFG_DATA_ENABLE	32'hffff303	配置 memory 空间写数据使能
CFG_DATA_DISABLE	32'hffff304	配置 memory 空间写数据使能结束

附 2 SIR 指令集设计表

表附-2 JTAG SIR 指令说明

指令	数值	描述
PCIE_ID_TEST0	10'h3f0	发出 0 型 PCI 配置读 TLP 并等待完成型 TLP；然后将返回值与预期的器件/供应商 ID 值进行比较。
PCIE_BAR_SCAN	10'h3f1	扫描 BAR 寄存器，检查 BAR 寄存器可用性
PCIE_BAR_PROG	10'h3f2	对 BAR 寄存器写固定值
PCIE_BAR_WRITE	10'h3f3	对 BAR 寄存器 编程
PCIE_BAR_READ	10'h3f4	从 BAR 寄存器读出数据
PCIE_PIO_TEST0	10'h3f5	向示例设计的各个有效 BAR 相继发送 1 DWORD 写 TLP 和 1 DWORD 读 TLP，然后等待完成型 TLP 并确认写和读数据匹配。此测试会根据 BAR 的地址类型（如 32 位或 64 位）和空间类型（如 IO 或存储器）向每个 BAR 发送相应的 TLP。
PCIE_PIO_BYTE_ENA	10'h3f6	发送四个连续的写 TLP，并为每个写 TLP 赋予唯一的字节使能，然后发出 1 DWORD 读 TLP，以确认数据已正确写入示例设计。

表附-2 JTAG SIR 指令说明（续）

指令	数值	描述
PCIE_PIO_DATA_TEST	10'h3f7	通过对示例设计中的第一个可用 BAR 执行 32 位 Walking One 数据测试，确定 PIO 设计的 FPGA Block RAM 数据总线接口是否已正确连接。
PCIE_PIO_ADDR_TEST	10'h3f8	通过执行 Walking One 地址测试，确定 PIO 设计的 FPGA Block RAM 的地址总线接口是否已正确连接。只有成功完成 pio_memTestDataBus 后才应调用此测试。
PCIE_PIO_DEVICE_TEST	10'h3f9	通过执行递增/递减测试，检查 PIO 设计的 FPGA Block RAM 每个位的完整性。只有成功完成 pio_memTestAddrBus 后才应调用此测试。
PCIE_READ_BACK	10'h3d2	从 RX 端读出数据，判断写操作是否成功
RAM_RESET	10'h3d4	读回数据存储 RAM 重置
SDR_CMD_TRANS	10'h3eb	发送 Sdr 指令集前需要发送的的使能指令